Lời nói đầu

Mục đích của cuốn sách này để giúp sinh viên nắm được các kiến thức cơ bản trong phát triển ứng dụng cho thiết bị di động. Giáo trình tập trung vào nền tảng Android, một trong các nền tảng thiết bị di động phổ biến nhất hiện nay.

Nội dung cuốn sách trình bày lần lượt các kỹ thuật cơ bản trong Android để giúp sinh viên có thể xây dựng một ứng dụng đầy đủ. Một số thuật ngữ nếu dịch ra tiếng Việt sẽ không diễn tả được hết nội hàm, do đó nhóm tác giả sẽ giữ nguyên thuật ngữ tiếng Anh gốc. Nội dung bao gồm các bài sau:

- Bài 1. Cơ bản về Kotlin
- Bài 2. Hàm trong ngôn ngữ Kotlin
- Bài 3. Lớp và đối tượng trong Kotlin
- Bài 4. Xây dựng ứng dụng Android đầu tiên
- Bài 5. Bố cục - Layout
- Bài 6. Điều hướng các màn hình trong Android
- Bài 7. Activity và Fragment.
- Bài 8. Kiến trúc chương trình.
- Bài 9. Lưu trữ dữ liệu trong Android.
- Bài 10. Một số giao diện nâng cao.
- Bài 11. Lập trình kết nối Internet.
- Bài 12. Lập trình native với ngôn ngữ C trên Android.
- Bài 13. Giới thiệu phương pháp lập trình giao tiếp với các thiết bị ngoại vi trên Android.

Giáo trình dành cho các bạn có mong muốn học phát triển ứng dụng cho thiết bị di động trên nền tảng Android. Toàn bộ nội dung của giáo trình được tham khảo chính từ bộ slide hướng dẫn lập trình Android của Google. Link đến tài liệu gốc có thể tìm thấy ở đây: `https:/drive.google.com/drive/folders/1eu-LXxiHocSktGYpG04PfE9Xmr_pBY5P`

Hà Nội, ngày 25 tháng 08 năm 2025

Hoàng Văn Hiệp, Nguyễn Hồng Quang, Lê Bá Vui, Nguyễn Đình Thuận

Mục lục

Lời nói đầu 3
Mục lục 5

Chương 1. Cơ bản về Kotlin **9**
1.1. Bắt đầu 9
1.2. Các toán tử 12
1.3. Kiểu dữ liệu 13
1.4. Biến 15
1.5. Câu lệnh điều kiện 16
1.6. Danh sách và mảng 21
1.7. An toàn null 23
1.8. Bài tập 24

Chương 2. Các hàm **25**
2.1. Các chương trình trong Kotlin 25
2.2. Các hàm trong Kotlin 29
2.3. Các hàm rút gọn 32
2.4. Lambda và hàm bậc cao 33
2.5. Bộ lọc danh sách 35
2.6. Bài tập 38

Chương 3. Lớp và đối tượng **39**
3.1. Lớp 39
3.2. Kế thừa 46
3.3. Các hàm mở rộng 49
3.4. Các lớp đặc biệt 50
3.5. Tổ chức mã nguồn 55

3.6. Bài tập 57

Chương 4. Xây dựng ứng dụng Android đầu tiên 58

4.1. Ứng dụng đầu tiên của bạn 58
4.2. Cấu trúc của một ứng dụng Android 63
4.3. Layout và resources trong ứng dụng Android 64
4.4. Activities 68
4.5. Tạo tính tương tác cho ứng dụng 70
4.6. Gradle: Xây dựng ứng dụng Android 74
4.7. Tổng kết chương 75

Chương 5. Bố cục - Layout 76

5.1. Layout trong Android 76
5.2. ConstraintLayout 78
5.3. Các chủ đề nâng cao với ConstraintLayout 84
5.4. Liên kết dữ liệu - Data binding 88
5.5. Hiển thị danh sách với RecyclerView 91
5.6. Tổng kết 94

Chương 6. Điều hướng các màn hình trong Android 96

6.1. Ứng dụng với nhiều activities 96
6.2. App bar, navigation drawer, và menus 98
6.3. Fragment 102
6.4. Diều hướng (Navigation) trong ứng dụng Android 103
6.5. Truyền dữ liệu 106
6.6. Back Stack 111
6.7. Tổng kết 112

Chương 7. Vòng đời hoạt động của Activity và Fragment 113

7.1. Vòng đời Activity 113
7.2. Ghi log (Logging) 117
7.3. Vòng đời Fragment 117
7.4. Các thành phần nhận biết vòng đời (Lifecycle-aware components) . . 119

7.5. Nhiệm vụ và ngăn xếp (Tasks & Back stack) 120
7.6. Tóm tắt 123

Chương 8. Kiến trúc ứng dụng (Tầng giao diện) 125
8.1. Kiến trúc ứng dụng Android 125
8.2. ViewModel 127
8.3. Data Binding 131
8.4. LiveData 133
8.5. Transform LiveData 137
8.6. Tóm tắt chương 137

Chương 9. Kiến trúc ứng dụng (Lưu trữ bền vững) 138
9.1. Lưu trữ dữ liệu 138
9.1.1. Cơ sở dữ liệu 139
9.1.2. Thư viện lưu trữ Room 141
9.1.3. Ví dụ Room 146
9.2. Lập trình bất đồng bộ 148
9.2.1. Coroutines 149
9.3. Kiểm thử với CSDL 154
9.4. Tóm tắt chương 155

Chương 10. Sử dụng nâng cao RecyclerView 157
10.1. Nhắc lại về RecyclerView 157
10.2. Liên kết nâng cao (Advanced binding) 162
10.3. Nhiều loại item view (Multiple item view types) 165
10.4. Headers 168
10.5. Grid Layout 169

Chương 11. Kết nối Internet 171
11.1. Android Permision 171
11.2. Kết nối, sử dụng các tài nguyên mạng 174
11.3. Kết nối đến một dịch vụ web 175
11.4. Hiển thị danh sách ảnh từ máy chủ 179

11.5. Tổng kết chương 181
Tài liệu tham khảo 182

Chương 1
Cơ bản về Kotlin

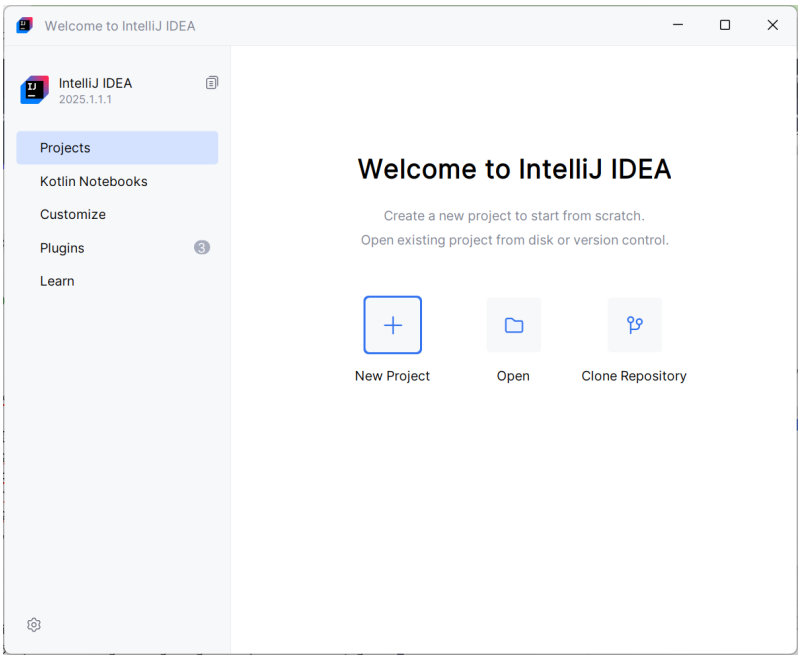
1.1. Bắt đầu 9
1.2. Các toán tử 12
1.3. Kiểu dữ liệu 13
1.4. Biến 15
1.5. Câu lệnh điều kiện 16
1.6. Danh sách và mảng 21
1.7. An toàn null 23
1.8. Bài tập 24

1.1. Bắt đầu

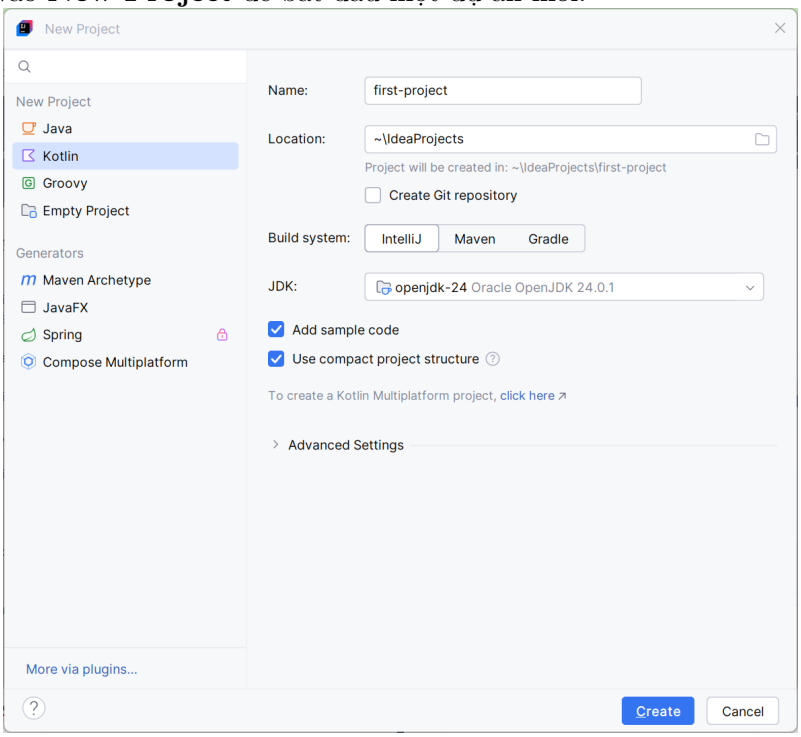
Để bắt đầu với lập trình bằng ngôn ngữ Kotlin, phần mềm IntelliJ IDEA bản Community được chọn sử dụng. Phần mềm này được tải miễn phí ở địa chỉ https://www.jetbrains.com/idea/download

IntelliJ IDEA là một Môi trường Phát triển Tích hợp (IDE) mạnh mẽ được tạo ra bởi JetBrains. Nó chủ yếu được thiết kế cho phát triển Java, nhưng cũng hỗ trợ nhiều ngôn ngữ khác như Kotlin, Scala, Groovy, JavaScript, TypeScript, SQL, Python (thông qua plugin), và nhiều ngôn ngữ khác.

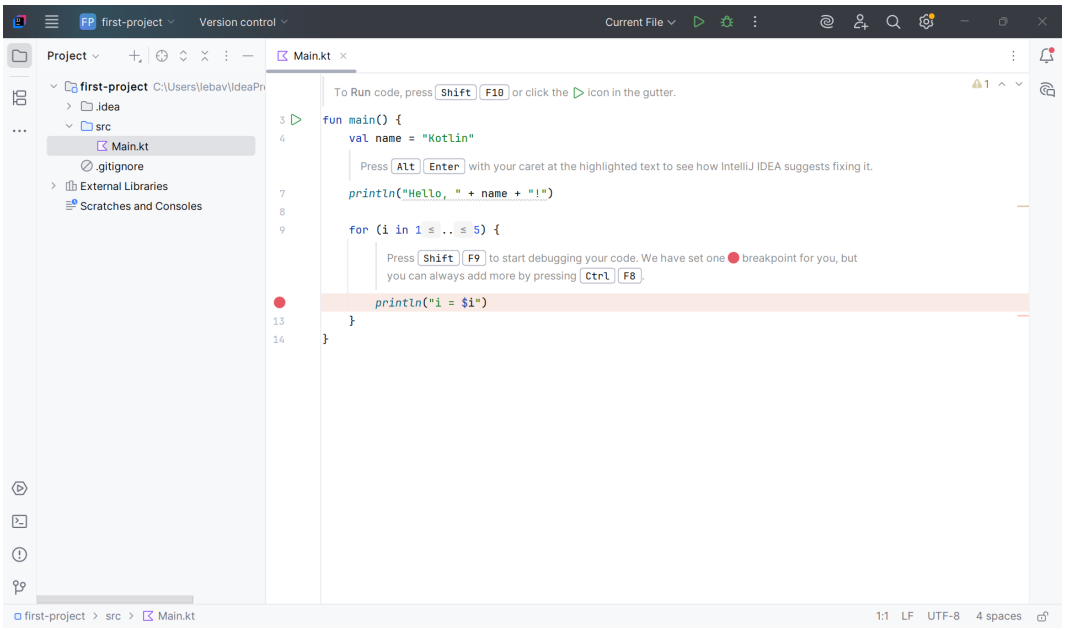
Để tạo một dự án lập trình bằng ngôn ngữ Kotlin, ta cần thực hiện theo các bước như sau:



Nhấp vào **New Project** để bắt đầu một dự án mới.



Chọn Kotlin, sau đó đặt tên cho dự án và nhấp vào **Create**.



Sau khi tạo xong project, IDE sẽ hiển thị nội dung file Main.kt với mã nguồn minh họa.

Nhập mã dưới đây vào file **Main.kt**:

```
fun printHello() {
    println("Hello World")
}

fun main() {
    printHello()
}
```

Hãy xem nhanh mã Kotlin trong ví dụ trên.

- Từ khóa **fun** chỉ định một hàm, theo sau là tên hàm.
- Như trong các ngôn ngữ lập trình khác, dấu ngoặc đơn dùng cho các tham số của hàm (nếu có), và dấu ngoặc nhọn bao quanh mã của hàm.
- Không có kiểu trả về vì hàm không trả về bất kỳ giá trị nào.
- Lưu ý rằng không có dấu chấm phẩy ở cuối các dòng. Việc thêm dấu chấm phẩy là tùy chọn, tuy nhiên nếu đặt các câu lệnh trên cùng một dòng, thì cần phải có dấu chấm phẩy để ngăn cách các câu lệnh.

1.2. Các toán tử

Hãy bắt đầu khám phá Kotlin bằng cách xem xét các thành phần chính của nó, bắt đầu với các toán tử số học. Như trong các ngôn ngữ khác, Kotlin sử dụng +, -, *, / và % cho phép cộng, trừ, nhân, chia và lấy nguyên (hoặc dư). Ngoài ra còn có các toán tử tăng và giảm, toán tử so sánh, toán tử gán, và toán tử kiểm tra bằng.

- Toán tử số học: + - * / %
- Toán tử tăng và giảm: ++ –
- Toán tử so sánh: < <= > >=
- Toán tử gán: =
- Toán tử kiểm tra bằng: == !=

Các toán tử số học rất đơn giản và hoạt động tương tự như nhiều ngôn ngữ lập trình khác.

Dưới đây là các ví dụ về toán tử với số nguyên:

1 + 1 => 2

53 - 3 => 50

50 / 10 => 5

9 % 3 => 0

Và toán tử với số thực:

1.0 / 2.0 => 0.5

2.0 * 3.5 => 7.0

Kotlin hỗ trợ các kiểu dữ liệu số khác nhau, như Int, Long, Double, và Float. Lưu ý rằng chúng đều bắt đầu bằng chữ in hoa.

1 + 1 => kotlin.Int = 2

53 - 3 => kotlin.Int = 50

50 / 10 => kotlin.Int = 5

1.0 / 2.0 => kotlin.Double = 0.5

2.0 * 3.5 => kotlin.Double = 7.0

Kotlin giữ các số dưới dạng nguyên thủy nhưng cho phép bạn gọi các phương thức trên số như thể chúng là đối tượng.

2.times(3) => kotlin.Int = 6

3.5.plus(4) => kotlin.Double = 7.5

2.4.div(2) => kotlin.Double = 1.2

1.3. Kiểu dữ liệu

Các kiểu số nguyên trong Kotlin có kích thước tiêu chuẩn giống như trong các ngôn ngữ khác.

Các kiểu dữ liệu số nguyên.

Kiểu	Số bit	Ghi chú
Long	64	Từ -2^{63} đến $2^{63}-1$
Int	32	Từ -2^{31} đến $2^{31}-1$
Short	16	Từ -32768 đến 32767
Byte	8	Từ -128 đến 127

Và các kiểu dữ liệu số khác.

Kiểu	Số bit	Ghi chú
Double	64	16-17 chữ số có nghĩa
Float	32	6-7 chữ số có nghĩa
Char	16	Ký tự Unicode 16-bit
Boolean	8	True hoặc false. Các phép toán bao gồm: || - phép hoặc lười, && - phép và lười, ! - phủ định

Kết quả của các phép toán giữ nguyên kiểu của các toán hạng.

6*50 => kotlin.Int = 300

6.0*50.0 => kotlin.Double = 300.0

6.0*50 => kotlin.Double = 300.0

1/2 => kotlin.Int = 0

1.0/2.0 => kotlin.Double = 0.5

Kotlin không tự động chuyển đổi giữa các kiểu số, vì vậy ta không thể gán trực tiếp giá trị Short cho biến Long, hoặc Byte cho Int. Chuyển đổi số ngầm định là nguồn lỗi phổ biến trong các chương trình, nhưng có thể tránh điều đó bằng cách gán giá trị của các kiểu khác nhau bằng cách ép kiểu.

Trong ví dụ dưới đây, trước tiên ta sẽ khai báo một biến và xem điều gì xảy ra nếu gán lại nó với một kiểu không khớp. Sau đó, chúng ta sử dụng toByte() để ép kiểu và in nó ra mà không có lỗi.

Gán một Int cho một Byte

```
val i: Int = 6
val b: Byte = i
```

```
println(b)
```

=> Lỗi cú pháp: Initializer type mismatch: expected 'Byte', actual 'Int'.

Chuyển đổi Int sang Byte với ép kiểu

```
val i: Int = 6
println(i.toByte())
```

=> 6

Sử dụng dấu gạch dưới để làm cho các hằng số số dài dễ đọc hơn.

```
val oneMillion = 1_000_000
val idNumber = 999_99_9999L
val hexBytes = 0xFF_EC_DE_5E
val bytes = 0b11010010_01101001_10010100_10010010
```

Chuỗi là tập hợp các ký tự được bao quanh bởi dấu ngoặc kép, có thể chứa cả khoảng trắng và số. Kotlin có hai loại chuỗi ký tự: chuỗi có ký tự thoát (có thể chứa các ký tự thoát) và chuỗi thô có thể chứa dấu xuống dòng và bất kỳ ký tự nào. Chuỗi thô được giới hạn bởi ba dấu ngoặc kép (""""), không chứa ký tự thoát và có thể chứa dấu xuống dòng và bất kỳ ký tự nào khác.

```
val s1 = "Hello world!"
val s2 = "Hello world!\n"
val text = """
  var bikes = 50
"""
```

Chuỗi trong Kotlin hoạt động tương tự như chuỗi trong các ngôn ngữ lập trình khác, sử dụng " cho chuỗi và ' cho ký tự đơn, và có thể nối chuỗi bằng toán tử +.

```
val numberOfDogs = 3
val numberOfCats = 2
"I have $numberOfDogs dogs" + " and $numberOfCats cats"
```

=> I have 2 dogs and 3 cats

Bạn có thể tạo mẫu chuỗi bằng cách kết hợp chúng với các giá trị. Trong ví dụ này, $numberOfDogs và $numberOfCats được thay thế bằng giá trị của các biến đó. Đây được gọi là nội suy biến.

Chuỗi ký tự có thể chứa các biểu thức mẫu; các đoạn mã được đánh giá và kết quả của chúng được nối vào chuỗi.

Biểu thức mẫu bắt đầu bằng dấu đô la ($) và có thể là một giá trị đơn giản:

```
val i = 10
println("i = $i")
```

=> i = 10

Hoặc một biểu thức trong dấu ngoặc nhọn:

```
val s = "abc"
println("$s.length is ${s.length}")
```

=> abc.length is 3

Như trong các ngôn ngữ khác, một giá trị có thể là kết quả của một biểu thức. Sử dụng dấu ngoặc nhọn để định nghĩa biểu thức.

```
val numberOfShirts = 10
val numberOfPants = 5
"I have ${numberOfShirts + numberOfPants} items of clothing"
```

=> I have 15 items of clothing

1.4. Biến

Kotlin có khả năng suy luận kiểu dữ liệu mạnh mẽ, và thường chỉ cần để trình biên dịch thực hiện công việc bằng cách suy ra kiểu. Tuy nhiên, bạn có thể khai báo kiểu của một biến một cách rõ ràng.

Xác định kiểu biến

Kiểu dữ liệu lưu trữ trong một biến được suy ra khi trình biên dịch có thể xác định từ ngữ cảnh. Nếu muốn, ta cũng có thể chỉ định kiểu của biến một cách rõ ràng, sử dụng ký hiệu dấu hai chấm.

Một số lưu ý về ký hiệu dấu hai chấm:

- Kiểu dữ liệu được đặt sau tên biến

- Luôn đặt một khoảng trắng sau dấu :

```
var width: Int = 12
var length: Double = 2.5
```

Quan trọng: Một khi kiểu đã được gán bởi bạn hoặc trình biên dịch, bạn không thể thay đổi kiểu hoặc sẽ nhận được lỗi.

Biến có thể thay đổi và không thể thay đổi

Kotlin hỗ trợ hai loại biến: có thể thay đổi giá trị và không thể thay đổi giá trị. Với **var**, ta có thể gán một giá trị, sau đó thay đổi giá trị đó sau này trong chương trình. Với **val**, ta chỉ có thể gán giá trị một lần. Nếu cố gắng thay đổi nó thành giá trị khác, sẽ nhận được lỗi cú pháp.

```
var score = 10
val name = "Jennifer"
```

Kotlin không bắt buộc tính bất biến, mặc dù điều này được khuyến nghị. Về cơ bản, hãy sử dụng **val** thay vì **var**.

Hãy xem một ví dụ sau đây. Ta có thể gán giá trị cho biến **count**, sau đó gán lại nó một giá trị mới, vì nó được định nghĩa với **var**. Việc cố gắng gán giá trị mới cho biến **size** sẽ gây ra lỗi vì nó được định nghĩa với **val**.

```
var count = 1
count = 2

val size = 1
size = 2
```

=> Lỗi: val không thể được gán lại

Lưu ý rằng trong Kotlin, theo mặc định, các biến không thể là **null**. Chúng ta sẽ nói về an toàn **null** sau trong bài học này.

1.5. Câu lệnh điều kiện

Kotlin cung cấp các biểu thức điều kiện sau:

- Câu lệnh If/Else

- Câu lệnh When
- Vòng lặp For
- Vòng lặp While

Câu lệnh điều kiện phổ biến nhất là câu lệnh **if/else**. Câu lệnh **if** cho phép chỉ định một khối mã (mã trong dấu ngoặc nhọn sau **if**) chỉ được thực thi nếu điều kiện đã cho là đúng. Nếu điều kiện sai, thì mã được chỉ định bởi câu lệnh **else** sẽ được thực thi. Lưu ý rằng khối **else** không bắt buộc.

```
val numberOfCups = 30
val numberOfPlates = 50

if (numberOfCups > numberOfPlates) {
    println("Too many cups!")
} else {
    println("Not enough cups!")
}
```

=> Not enough cups!

Như trong các ngôn ngữ khác, có thể kiểm tra nhiều trường hợp bằng cách sử dụng else if. Lưu ý rằng ta có thể sử dụng bất kỳ toán tử so sánh hoặc bằng nào trong các điều kiện **if/else**, không chỉ == hoặc < như đã thấy ở trên, cũng có thể sử dụng toán tử logic và ("&&"), và logic hoặc ("||").

```
val guests = 30
if (guests == 0) {
    println("No guests")
} else if (guests < 20) {
    println("Small group of people")
} else {
    println("Large group of people!")
}
```

=> Large group of people!

Phạm vi

Một phạm vi (hoặc "khoảng") xác định các ranh giới bao gồm xung quanh một khoảng liên tục của các giá trị của một số kiểu có thể so sánh, chẳng hạn như **intRange** (ví dụ, các số nguyên từ 1 đến 100 bao gồm cả hai đầu). Số đầu tiên là điểm bắt đầu, số thứ hai là điểm kết thúc.

Tất cả các phạm vi đều có giới hạn, và phía bên trái của phạm vi luôn <= phía bên phải của phạm vi.

Trong Kotlin, điều kiện kiểm tra cũng có thể sử dụng phạm vi. Phạm vi cho phép chỉ định một tập hợp con của một nhóm lớn hơn, ví dụ, chúng ta chỉ quan tâm đến các số nguyên từ 1 đến 100 (tức là, một phạm vi các giá trị kiểu Int, hoặc IntRange). Có các lớp tương đương với IntRange cho các kiểu dữ liệu khác, như CharRange và LongRange.

```
val numberOfStudents = 50
if (numberOfStudents in 1..100) {
    println(numberOfStudents)
}
```

=> 50

Lưu ý: Không có khoảng trắng xung quanh toán tử "phạm vi đến" (1..100)

Ta cũng có thể định nghĩa giá trị bước nhảy giữa các phần tử trong giới hạn của phạm vi. Ví dụ, trong 1..8, nếu chúng ta định nghĩa kích thước bước là 2, phạm vi của chúng ta sẽ bao gồm các phần tử 1, 3, 5, 7.

Ví dụ:

```
for (i in 1..8 step 2) print(i)
```

=>1357

Câu lệnh when Có một cách gọn gàng hơn để viết một loạt các câu lệnh **if/else** trong Kotlin, sử dụng câu lệnh when. Nó giống như câu lệnh **switch** trong các ngôn ngữ khác. Các điều kiện trong câu lệnh **when** cũng có thể sử dụng phạm vi.

```
when (results) {
    0  -> println("No results")
    in 1..39 -> println("Got results!")
    else -> println("That's a lot of results!")
}
```

=> That's a lot of results!

Ngoài câu lệnh **when**, cũng có thể định nghĩa một biểu thức **when** cung cấp giá trị trả về.

Ví dụ:

```
val month = 1
val result = when (month) {
    1, 3, 5, 7, 8, 10, 12 -> "31 days"
    4, 6, 9, 11 -> "30 days"
    else -> "28 or 29 days"
}
```

=> kết quả là "31 days"

Vòng lặp for

Kotlin hỗ trợ các vòng lặp **for**, **while**, **do-while**. Hãy xem xét các ví dụ của từng loại, bắt đầu với vòng lặp **for**.

Trong ví dụ dưới, chúng ta sử dụng vòng lặp **for** để lặp qua mảng và in các phần tử.

```
val pets = arrayOf("dog", "cat", "canary")
for (element in pets) {
    print(element + " ")
}
```

=> dog cat canary

Với cú pháp này, ta không cần định nghĩa một biến lặp và tăng nó cho mỗi lần lặp. Cũng có thể lặp qua các phần tử và chỉ số cùng một lúc.

```
for ((index, element) in pets.withIndex()) {
    println("Item at $index is $element\n")
}
```

=> Item at 0 is dog Item at 1 is cat Item at 2 is canary

Ta có thể chỉ định phạm vi của số hoặc ký tự, theo thứ tự bảng chữ cái, hoặc có thể định nghĩa kích thước bước, cũng có thể bước lùi bằng **downTo**.

```
for (i in 1..5) print(i)
```

=> 12345

```
for (i in 5 downTo 1) print(i)
```

=> 54321

```
for (i in 3..6 step 2) print(i)
```

=> 35

```
for (i in 'd'..'g') print (i)
```

=> defg

Vòng lặp while

Như các ngôn ngữ khác, Kotlin có vòng lặp **while**, **do...while**, và các toán tử ++ và –. Hãy xem các ví dụ sau:

```
var bicycles = 0
while (bicycles < 50) {
    bicycles++
}
println("$bicycles bicycles in the bicycle rack\n")
```

=> 50 bicycles in the bicycle rack

```
do {
    bicycles--
} while (bicycles > 50)
println("$bicycles bicycles in the bicycle rack\n")
```

=> 49 bicycles in the bicycle rack

Vòng lặp repeat

Kotlin cung cấp thêm hàm bậc cao repeat cho phép lặp lại một khối mã bên trong dấu ngoặc nhọn của nó. Số trong dấu ngoặc đơn là số lần nó nên được lặp lại. Trong ví dụ sau, lệnh in được thực thi hai lần.

```
repeat(2) {
    print("Hello!")
}
```

=> Hello!Hello!

1.6. Danh sách và mảng

Danh sách là một tập hợp có thứ tự với quyền truy cập vào các phần tử của nó thông qua các chỉ số—các số nguyên phản ánh vị trí của chúng. Các phần tử có thể xuất hiện nhiều lần trong một danh sách.

Một ví dụ về danh sách là một câu: nó là một nhóm các từ, thứ tự của chúng quan trọng, và chúng có thể lặp lại.

Danh sách bất biến sử dụng listOf()

Danh sách được tạo bằng **listOf()** không thể thay đổi (bất biến), không thêm bớt cũng như không thay đổi được giá trị của các phần tử. Ví dụ này khai báo một danh sách sử dụng **listOf()** và in nó ra.

```
val instruments = listOf("trumpet", "piano", "violin")
println(instruments)
```

=> [trumpet, piano, violin]

Danh sách có thể thay đổi sử dụng mutableListOf()

Chúng ta sẽ sử dụng **mutableListOf()** để tạo một danh sách có thể thay đổi. Trong ví dụ, phương thức **remove()** trả về true khi nó xóa thành công mục được truyền vào.

```
val myList = mutableListOf("trumpet", "piano", "violin")
myList.remove("violin")
```

=> kotlin.Boolean = true

Với danh sách được định nghĩa bằng **val**, ta không thể thay đổi danh sách mà biến tham chiếu đến, nhưng vẫn có thể thay đổi nội dung của danh sách.

Mảng

Mảng được sử dụng để tổ chức dữ liệu sao cho một tập hợp các giá trị liên quan có thể được sắp xếp hoặc tìm kiếm dễ dàng.

- Mảng lưu trữ nhiều mục.
- Các phần tử mảng có thể được truy cập thông qua chỉ số của chúng.
- Các phần tử mảng có thể thay đổi.
- Kích thước mảng là cố định.

Ví dụ về một mảng các chuỗi có thể được tạo bằng **arrayOf()**:

```
val pets = arrayOf("dog", "cat", "canary")
println(java.util.Arrays.toString(pets))
```

=> [dog, cat, canary]

Với mảng được định nghĩa bằng **val**, không thể thay đổi mảng mà biến tham chiếu đến, nhưng vẫn có thể thay đổi nội dung của mảng.

Trong Kotlin, ta có thể khai báo mảng với hỗn hợp các kiểu, hoặc chỉ một kiểu cho tất cả các phần tử. Trong ví dụ thứ hai, mặc dù chúng ta định nghĩa một mảng chỉ chứa số nguyên bằng **intArrayOf()**. Có các hàm khởi tạo tương ứng cho mảng của các kiểu khác.

Mảng có thể chứa các kiểu khác nhau.

```
val mix = arrayOf("hats", 2)
```

Mảng cũng có thể chỉ chứa một kiểu (số nguyên trong trường hợp này).

```
val numbers = intArrayOf(1, 2, 3)
```

Trong ví dụ này, chúng ta kết hợp hai mảng và sau đó in mảng kết quả.

```
val numbers = intArrayOf(1,2,3)
val numbers2 = intArrayOf(4,5,6)
val combined = numbers2 + numbers
println(Arrays.toString(combined))
```

1.7. An toàn null

Một đặc điểm nổi bật của Kotlin là nỗ lực loại bỏ nguy cơ tham chiếu **null** trong mã nguồn. Tham chiếu **null** xảy ra khi mã nguồn cố gắng truy cập một biến không tham chiếu đến bất kỳ thứ gì (mang giá trị null), có thể làm dừng thực thi mã và gây ra **NullPointerException**.

Trong Kotlin, các biến mặc định không được mang giá trị **null**. Ta không thể khai báo một biến kiểu Int và gán null cho nó.

```
var numberOfBooks: Int = null
```

=> lỗi: Null cannot be a value of a non-null type 'Int'.

Hãy nói về biến có thể mang giá trị **null** so với biến không thể mang giá trị **null**. Các lỗi lập trình liên quan đến **null** đã gây ra vô số lỗi. Kotlin tìm cách giảm lỗi bằng cách giới thiệu các biến không thể mang giá trị **null**. Vì các biến không thể là **null** theo mặc định, nên ít có nguy cơ thực thi mã thất bại do **NullPointerException**. Trong ví dụ với dòng mã ở trên, không được trình biên dịch cho phép.

Toán tử gọi an toàn (?) đặt sau kiểu dữ liệu, chỉ ra rằng một biến có thể là **null**.

Ví dụ này sẽ khai báo một biển thuộc kiểu dữ liệu **Int?** có thể mang giá trị **null**.

```
var numberOfBooks: Int? = null
```

Ta có thể kiểm tra giá trị **null** bằng toán tử **?**, giúp tránh phải viết nhiều câu lệnh if/else.

Ví dụ này sẽ kiểm tra xem biến **numberOfBooks** có phải là **null** hay không. Sau đó gọi phương thức để giảm giá trị biến đó.

```
var numberOfBooks = 6
if (numberOfBooks != null) {
    numberOfBooks = numberOfBooks.dec()
}
```

Bây giờ hãy xem cách viết theo phong cách Kotlin, sử dụng toán tử gọi an toàn.

```
var numberOfBooks = 6
numberOfBooks = numberOfBooks?.dec()
```

Mặc dù ví dụ đầu tiên được viết bằng Kotlin, nhưng nó giống cách kiểm tra **null** trong Java hoặc các ngôn ngữ khác. Nó vẫn là Kotlin, nhưng không phải là cách đặc trưng của Kotlin. Dòng mã cuối cùng trong ví dụ chỉ đơn giản nói "gọi dec() trên numberOfBooks nếu nó không phải là null."

Nếu chắc chắn một biến không phải là **null**, hãy sử dụng **!!** để ép biến thành kiểu không **null**. Sau đó, có thể gọi các phương thức/thuộc tính trên nó.

```
val len = s!!.length
```

=> Dòng mã này sẽ gây ra một NullPointerException nếu s là null.

Cảnh báo: Vì !! sẽ gây ra một ngoại lệ, nó chỉ nên được sử dụng khi việc giữ giá trị null là bất thường.

Toán tử Elvis

Toán tử **?:** đôi khi được gọi là "toán tử Elvis," vì nó giống như một khuôn mặt cười nghiêng sang một bên với kiểu tóc pompadour, giống như cách Elvis Presley chải tóc.

Ví dụ này sẽ liên kết các kiểm tra **null** với toán tử **?:**.

```
numberOfBooks = numberOfBooks?.dec() ?: 0
```

Ví dụ này được diễn giải như sau:

"Nếu **numberOfBooks** không phải là **null**, gọi hàm giảm và sử dụng nó; nếu không, sử dụng giá trị sau **?:**, tức là giá trị 0."

Nếu **numberOfBooks** là **null**, việc đánh giá sẽ dừng lại, và phương thức **dec()** không được gọi. Kết hợp các kiểm tra null với toán tử Elvis giúp giảm chúng thành một dòng duy nhất, và tránh phải sử dụng câu lệnh **if/else** dài hơn.

1.8. Bài tập

Bài tập chương 1: https://developer.android.com/courses/pathways/android-development-with-kotlin-1

Chương 2
Các hàm

2.1. Các chương trình trong Kotlin 25
2.2. Các hàm trong Kotlin 29
2.3. Các hàm rút gọn 32
2.4. Lambda và hàm bậc cao 33
2.5. Bộ lọc danh sách 35
2.6. Bài tập 38

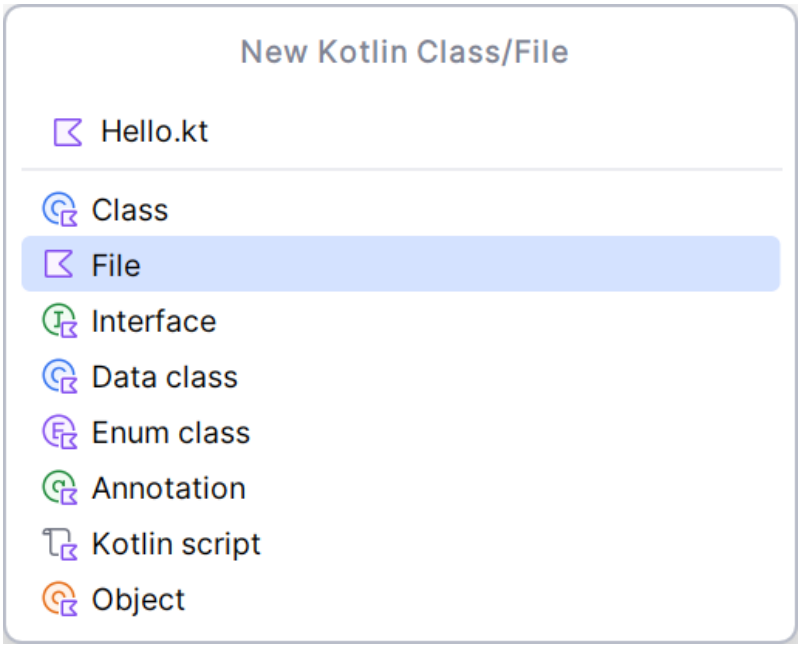
2.1. Các chương trình trong Kotlin

Trước khi có thể viết mã và chạy các chương trình, ta cần:

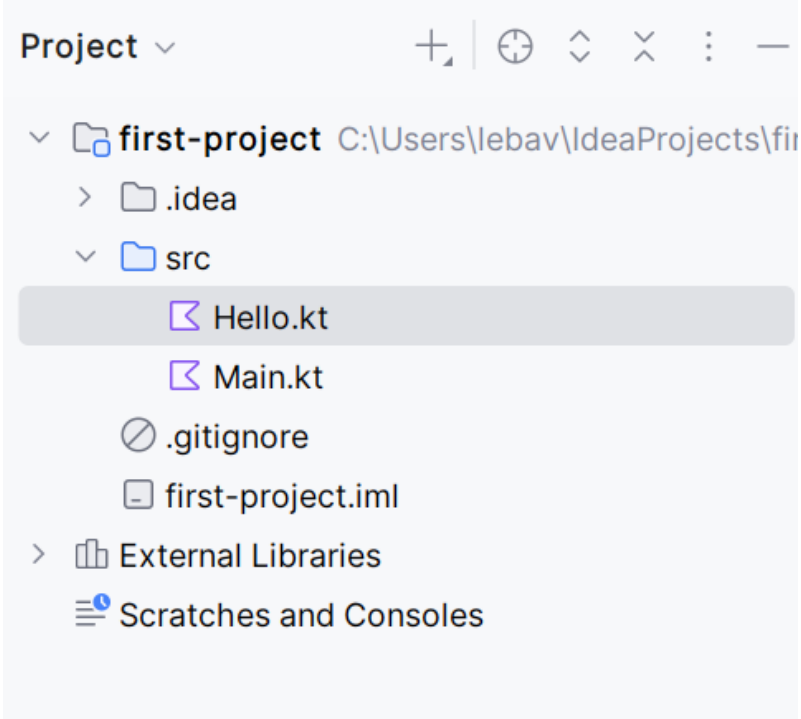
- Tạo một tệp trong dự án
- Tạo một hàm main()
- Truyền tham số cho main() (Tùy chọn)
- Sử dụng các tham số được truyền trong các lời gọi hàm (Tùy chọn)
- Chạy chương trình

Trong cửa sổ dự án, nhấp chuột phải vào thư mục **src**.

- Chọn New > Kotlin Class/File.
- Chọn File, đặt tên tệp là Hello.kt, và nhấn Enter.



Bây giờ ta sẽ thấy một tệp trong thư mục **src** có tên là Hello.kt.



Tạo một hàm main()

main() là điểm bắt đầu cho việc thực thi một chương trình Kotlin.

Trong tệp Hello.kt:

```
fun main(args: Array<String>) {
    println("Hello, world!")
}
```

Các tham số **args** trong hàm **main()** là tùy chọn.

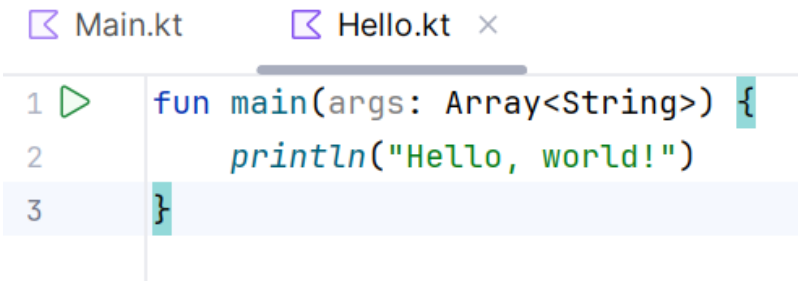
Như trong các ngôn ngữ khác, hàm **main()** trong Kotlin chỉ định điểm bắt đầu cho việc thực thi. Bất kỳ tham số dòng lệnh nào được truyền dưới dạng một mảng các chuỗi.

Tương tự như hàm **printHello()** trước đó, hàm này không có câu lệnh trả về. Mọi hàm trong Kotlin đều trả về một giá trị nào đó, ngay cả khi không có gì được chỉ định rõ ràng. Vì vậy, một hàm như hàm **main()** này trả về kiểu **kotlin.Unit**, đó là cách Kotlin nói rằng không có giá trị.

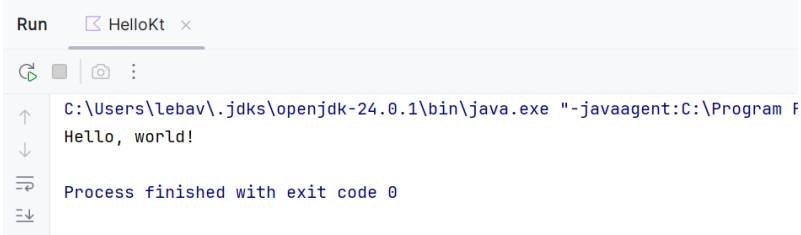
Lưu ý: Khi một hàm trả về **kotlin.Unit**, không cần phải chỉ định nó một cách rõ ràng. Điều này khác với một số ngôn ngữ khác, nơi ta phải nói rõ rằng hàm không trả về gì.

Chạy chương trình Kotlin

Để chạy chương trình, nhấp vào biểu tượng **Run** ở bên trái của hàm **main()**.

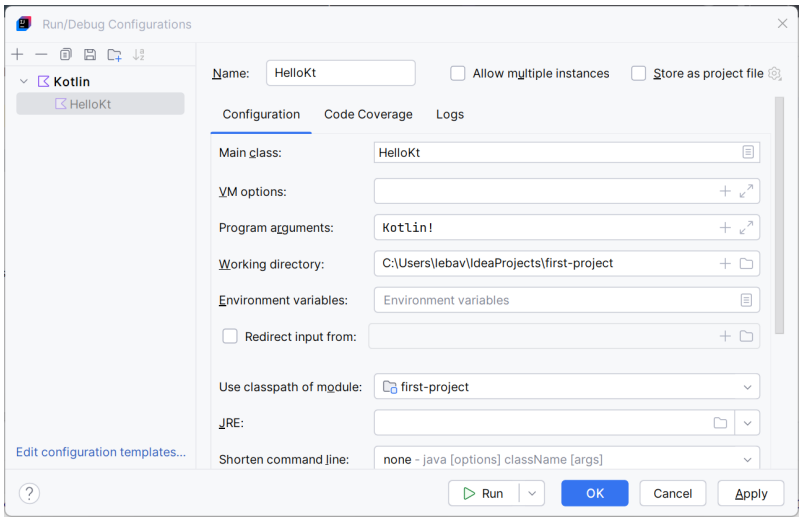


IntelliJ IDEA chạy chương trình và hiển thị kết quả.



Truyền tham số cho main()

Chọn Chạy > Chỉnh sửa Cấu hình để mở cửa sổ Cấu hình Chạy/Gỡ lỗi.



Nhập Kotlin! vào trường Program arguments và nhấp vào OK.

Vì bạn đang chạy chương trình từ IntelliJ IDEA chứ không phải từ dòng lệnh, bạn cần chỉ định bất kỳ tham số nào cho chương trình một cách hơi khác.

Sử dụng tham số trong main()

Sử dụng args[0] để truy cập tham số đầu vào đầu tiên được truyền cho main().

```
fun main(args: Array<String>) {
    println("Hello, ${args[0]}")
}
```

=> Hello, Kotlin!

Trước đó, chúng ta đã truyền "Kotlin!" làm tham số cho main().

Trong Hello.kt, thay đổi thông điệp chào hỏi để sử dụng tham số đầu tiên được truyền vào chương trình (thông qua cửa sổ Cấu hình Chạy/Gỡ lỗi ở slide trước), args[0], thay vì "world".

Ví dụ này sử dụng mẫu chuỗi. Mẫu chuỗi cho phép bạn tham chiếu các biến bên trong khai báo chuỗi.

(Hầu hết) Mọi thứ trong Kotlin đều là một biểu thức

Trong Kotlin, gần như mọi thứ là một biểu thức và có một giá trị. Ngay cả một biểu thức **if** cũng có giá trị.

```
val temperature = 20
val isHot = if (temperature > 40) true else false
```

```
println(isHot)
```

=> false

Khai báo một biến bằng từ khóa **val** có tên là **temperature** và khởi tạo nó bằng 20. Sau đó khai báo một biến val khác có tên là **isHot** và gán giá trị trả về của một câu lệnh **if/else** cho **isHot**. Bây giờ chạy chương trình, và thấy rằng giá trị của biểu thức **if** được trả về.

Lưu ý: Vòng lặp là ngoại lệ cho "mọi thứ có giá trị." Không có giá trị hợp lý cho các vòng lặp **for** hoặc **while**, vì vậy chúng không có giá trị. Nếu bạn cố gắng gán giá trị của một vòng lặp cho một thứ gì đó, trình biên dịch sẽ báo lỗi.

Đôi khi, giá trị đó là *kotlin.Unit.*

```
val isUnit = println("This is an expression")
println(isUnit)
```

=> This is an expression kotlin.Unit

Lệnh **println()** đầu tiên in chuỗi "This is an expression"; lệnh **println()** thứ hai in giá trị trả về của câu lệnh **println()** đầu tiên.

Một số ngôn ngữ khác có các câu lệnh điều khiển, là những dòng mã không có giá trị. Trong Kotlin, gần như mọi thứ là một biểu thức và có một giá trị—ngay cả khi giá trị đó là kotlin.Unit. (Unit của Kotlin tương đương với void của Java.)

Để chứng minh, trong hàm main() trong file Hello.kt, chúng ta đã viết một số mã để gán giá trị của hàm **println()** cho một biến có tên là **isUnit**, và sau đó in nó. Vì **println()** không trả về giá trị, nó trả về **kotlin.Unit**.

2.2. Các hàm trong Kotlin

Hàm là một khối mã riêng biệt thực hiện một thao tác và có thể trả về một giá trị. Trong Kotlin, các hàm được khai báo bằng từ khóa **fun** và có thể nhận tham số với giá trị được đặt tên hoặc giá trị mặc định. Một hàm liên kết với một lớp cụ thể được gọi là phương thức.

Hàm mang các đặc điểm sau:

- Một khối mã thực hiện một nhiệm vụ cụ thể.
- Phân chia một chương trình lớn thành các phần nhỏ hơn.

- Được khai báo bằng từ khóa **fun**.
- Có thể nhận các tham số với giá trị được đặt tên hoặc mặc định.

Trước đó, bạn đã tạo một hàm đơn giản in "Hello World".

```
fun printHello() {
    println("Hello World")
}
printHello()
```

Ta định nghĩa các hàm bằng từ khóa **fun**, theo sau là tên của hàm. Như trong các ngôn ngữ lập trình khác, dấu ngoặc đơn () dành cho các tham số của hàm, nếu có. Dấu ngoặc nhọn bao quanh mã của hàm. Không có kiểu trả về cho hàm này, vì nó không trả về bất kỳ thứ gì.

Nếu một hàm không trả về bất kỳ giá trị hữu ích nào, kiểu trả về của nó là **Unit**.

```
fun printHello(name: String?): Unit {
    println("Hi there!")
}
```

Unit là một kiểu chỉ có một giá trị: Unit. Khai báo kiểu trả về Unit là tùy chọn. Hai đoạn mã sau tương đương với nhau.

```
fun printHello(name: String?): Unit {
    println("Hi there!")
}
```

tương đương với:

```
fun printHello(name: String?) {
    println("Hi there!")
}
```

Tham số của hàm

Các hàm có thể có:

- Tham số mặc định

- Tham số bắt buộc
- Tham số được truyền theo tên

Tham số mặc định

Giá trị mặc định cung cấp một giá trị dự phòng nếu không có giá trị tham số nào được truyền.

Trong **Hello.kt**, hãy viết một hàm **drive()** với một tham số có kiểu dữ liệu **String** có tên là **speed** để in tốc độ của xe. Tham số **speed** có giá trị mặc định là "fast".

```
fun drive(speed: String = "fast") {
   println("driving $speed")
}
```

Sử dụng "=" sau kiểu dữ liệu để định nghĩa giá trị mặc định.

Từ hàm **main()**, gọi hàm **drive()** theo ba cách.

- Gọi hàm sử dụng giá trị mặc định.
- Gọi hàm và truyền tham số **speed** mà không có tên.
- Gọi hàm bằng cách gọi tên cho tham số **speed**.

```
drive() => driving fast
drive("slow") => driving slow
drive(speed = "turtle-like") => driving turtle-like
```

Tham số bắt buộc

Nếu không chỉ định giá trị mặc định cho một tham số, giá trị truyền vào tương ứng là bắt buộc.

```
fun tempToday(day: String, temp: Int) {
    println("Today is $day and it's $temp degrees.")
}
```

Hàm **tempToday()** nhận hai tham số, **day** và **temp**, cả hai đều là bắt buộc.

Các hàm có thể có sự kết hợp của tham số mặc định và bắt buộc.

```
fun reformat(str: String,
    divideByCamelHumps: Boolean,
    wordSeparator: Char,
    normalizeCase: Boolean = true){ }
```

Truyền vào các tham số bắt buộc.

```
reformat("Today is a day like no other day", false, '_')
```

Trong ví dụ trên, chúng ta sử dụng hàm **reformat()** để áp dụng một định dạng được chỉ định, hoặc kết hợp các định dạng cho String. Các tham số của **reformat()** chứa tên và kiểu của chúng. Tham số **normalizeCase** có giá trị mặc định.

Các tham số mặc định và được đặt tên giúp giảm thiểu việc viết chồng và cải thiện tính dễ đọc của mã nguồn.

Tham số được đặt tên

Để cải thiện tính dễ đọc, hãy sử dụng tên cho các tham số bắt buộc.

```
reformat(str, divideByCamelHumps = false, wordSeparator = '_')
```

Việc gọi tên các tham số mặc định sau các tham số bắt buộc được coi là phong cách tốt, bằng cách này, người gọi chỉ cần chỉ định các tham số bắt buộc.

Các tham số có thể được truyền vào một hàm với tên của chúng (tham số được đặt tên). Sử dụng tham số được đặt tên rất tiện lợi khi một hàm có số lượng tham số lớn hoặc có các tham số mặc định.

2.3. Các hàm rút gọn

Các hàm rút gọn, hoặc các hàm biểu thức, giúp mã nguồn ngắn gọn và dễ đọc hơn.

Khi một hàm trả về một biểu thức duy nhất, dấu ngoặc nhọn có thể được bỏ qua và phần thân được chỉ định sau ký hiệu "=".

```
fun double(x: Int): Int {
    return x * 2
}
=> Phiên bản đầy đủ
fun double(x: Int):Int = x * 2
=> Phiên bản gọn nhẹ
```

2.4. Lambda và hàm bậc cao

Các hàm trong Kotlin là các hàm bậc nhất (first-class)

- Các hàm Kotlin có thể được lưu trữ trong các biến và cấu trúc dữ liệu.
- Chúng có thể được truyền như tham số cho, và trả về từ, các hàm bậc cao khác.
- Chúng ta có thể sử dụng các hàm bậc cao để tạo ra các hàm "tích hợp" mới.

Hàm lambda

Ngoài các hàm được đặt tên truyền thống, Kotlin hỗ trợ lambda. Lambda là một biểu thức tạo ra một hàm. Nhưng thay vì khai báo một hàm được đặt tên, bạn khai báo một hàm không có tên. Điều làm cho điều này hữu ích là biểu thức lambda giờ đây có thể được truyền như dữ liệu. Trong các ngôn ngữ khác, lambda được gọi là hàm ẩn danh, hàm chữ, hoặc các tên tương tự.

```
var dirtLevel = 20
val waterFilter = {level: Int -> level / 2}
println(waterFilter(dirtLevel))
=> 10
```

Giống như các hàm được đặt tên, lambda có thể có các tham số. Đối với lambda, các tham số (và kiểu của chúng, nếu cần) được đặt ở phía bên trái của mũi tên ->. Mã để thực thi được đặt ở phía bên phải của mũi tên. Khi lambda được gán cho một biến, bạn có thể gọi nó giống như một hàm.

Cú pháp cho kiểu hàm

Cú pháp của Kotlin cho các kiểu hàm có liên quan chặt chẽ với cú pháp của nó cho lambda. Khai báo một biến chứa một hàm.

```
val waterFilter: (Int) -> Int = {level -> level / 2}
```

Đây là ý nghĩa của đoạn mã trên:

- Tạo một biến kiểu hàm có tên là waterFilter.
- waterFilter có thể là bất kỳ hàm nào nhận một tham số kiểu nguyên Int và trả về một giá trị kiểu nguyên Int.

- Gán một lambda cho waterFilter.
- Lambda trả về giá trị của tham số level chia cho 2.

Lưu ý rằng bạn không cần phải chỉ định kiểu của tham số lambda nữa. Kiểu được tính toán bằng suy luận kiểu.

Hàm bậc cao

Sức mạnh thực sự của lambda là sử dụng chúng để tạo ra các hàm bậc cao, nơi tham số cho một hàm là một hàm khác, hoặc trả về một hàm.

```
fun encodeMsg(msg: String, encode: (String) -> String): String {
    return encode(msg)
}
```

Phần thân của hàm **encodeMsg()** gọi hàm được truyền ở tham số thứ hai **encode()**, và truyền tham số đầu tiên **msg** cho nó.

Ở đây, việc cho phép bộ mã hóa được truyền dưới dạng hàm có nghĩa là ta có thể sử dụng các thuật toán mã hóa tốt hơn hoặc khác nhau mà không cần phải cố định một cái vào ứng dụng. Nó cũng cung cấp tính trừu tượng bằng cách cho phép một bộ nhận được sử dụng ở các nơi khác nhau mà không cần mã chuyên biệt.

Để gọi hàm này, truyền vào một chuỗi và một hàm.

```
val enc1: (String) -> String = { input -> input.toUpperCase() }
println(encodeMsg("abc", enc1))
```

Sử dụng kiểu hàm tách biệt việc triển khai của nó khỏi việc sử dụng, cho phép bạn sử dụng bất kỳ thứ gì thỏa mãn điều kiện khai báo, trong trường hợp này là bất kỳ thứ gì nhận một String và trả về một String: (String) -> String.

Truyền tham chiếu hàm

Sử dụng toán tử :: để truyền một hàm được đặt tên như một tham số cho một hàm khác.

```
fun enc2(input:String): String = input.reversed()
encodeMessage("abc", ::enc2)
```

Toán tử :: cho Kotlin biết rằng bạn đang truyền tham chiếu hàm như một tham số, và không cố gắng gọi hàm.

Cú pháp gọi tham số cuối cùng

Khi làm việc với các hàm bậc cao, Kotlin ưu tiên rằng bất kỳ tham số nào nhận một hàm là tham số cuối cùng. Kotlin có một cú pháp đặc biệt, được gọi là cú pháp gọi tham số cuối cùng, giúp mã trở nên ngắn gọn hơn.

```
encodeMsg("acronym", { input -> input.toUpperCase() })
```

Trong trường hợp này, ta có thể truyền một lambda cho tham số hàm, nhưng không cần phải đặt lambda bên trong dấu ngoặc đơn.

```
encodeMsg("acronym") { input -> input.toUpperCase() }
```

Sử dụng hàm bậc cao

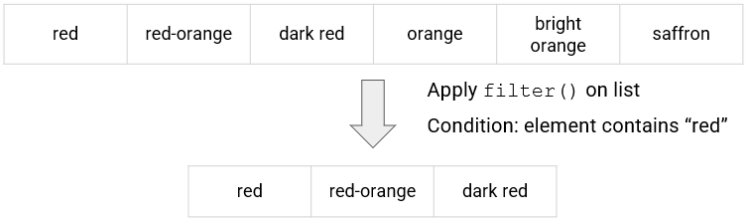
Nhiều hàm tích hợp của Kotlin được định nghĩa bằng cú pháp gọi tham số cuối cùng. Ví dụ sau mô tả cách gọi hàm **repeat()** đã được giới thiệu ở bài trước.

```
inline fun repeat(times: Int, action: (Int) -> Unit)
repeat(3) {
    println("Hello")
}
=> Hello
Hello
Hello
```

Chúng ta sẽ nói thêm về cách các hàm bậc cao có thể được sử dụng trong phần tiếp theo về "Bộ lọc Danh sách."

2.5. Bộ lọc danh sách

Bộ lọc là một cách nhanh chóng để lấy một phần của danh sách dựa trên một số điều kiện. Ở đây, chúng ta áp dụng một bộ lọc danh sách để chỉ chọn các màu có "red" trong tên của chúng.



Lặp qua danh sách

Nếu một hàm chữ chỉ có một tham số, bạn có thể bỏ qua khai báo của nó và dấu mũi tên "->". Tham số được khai báo ngầm dưới tên **it**.

```
val ints = listOf(1, 2, 3)
ints.filter { it > 0 }
```

Bộ lọc lặp qua một tập hợp, trong đó **it** là giá trị của phần tử trong quá trình lặp. Điều này tương đương với:

```
ints.filter { n: Int -> n > 0 }
hoặc
ints.filter { n -> n > 0 }
```

Hàm với đúng một tham số không yêu cầu bạn định nghĩa tham số một cách rõ ràng; bạn chỉ cần sử dụng **it**. Điều này làm cho nhiều cấu trúc ngôn ngữ như **filter** dễ sử dụng hơn vì bạn không phải chỉ định tên của tham số.

Tại mỗi bước, chúng ta có thể bỏ qua tính hình thức vì hệ thống kiểu có thể suy ra nội dung được mong đợi là gì. Với **it**, không có câu hỏi về kiểu sẽ là gì, và chỉ có một giá trị được cung cấp cho khối mã.

Điều kiện bộ lọc trong dấu ngoặc nhọn kiểm tra từng mục khi bộ lọc lặp qua. Nếu biểu thức trả về **true**, mục đó được bao gồm.

Hãy sử dụng một bộ lọc để in chỉ các phần bắt đầu bằng chữ cái "b". Đầu ra được bộ lọc ghi lại được hiển thị trong kết quả.

```
val books = listOf("nature", "biology", "birds")
println(books.filter { it[0] == 'b' })
=> [biology, birds]
```

Bộ lọc tích cực và lười biếng

Đánh giá các biểu thức trong danh sách:

- Tích cực: xảy ra bất kể kết quả có được sử dụng hay không
- Lười biếng: chỉ xảy ra nếu cần thiết tại thời điểm chạy

Đánh giá lười biếng của danh sách hữu ích nếu ta không cần toàn bộ kết quả, hoặc nếu danh sách đặc biệt lớn và nhiều bản sao không thể chứa trong bộ nhớ.

Các thao tác lười biếng tốn kém hơn riêng lẻ (do tăng các phân bổ nhỏ và nhánh) so với các thao tác tích cực, và chúng nên được giới hạn trong trường hợp có lợi ích rõ ràng. Sử dụng đánh giá lười biếng đặc biệt hữu ích khi làm việc với các tập hợp lớn, nơi bạn muốn tránh thực hiện các thao tác tốn kém khi bạn chỉ cần một phần của kết quả.

Bộ lọc là tích cực theo mặc định. Một danh sách mới được tạo mỗi khi bạn sử dụng bộ lọc.

```
val instruments = listOf("viola", "cello", "violin")
val eager = instruments.filter { it [0] == 'v' }
println("eager: " + eager)
```

=> eager: [viola, violin]

Trong Kotlin, danh sách kết quả có thể được tạo ngay lập tức, hoặc khi danh sách được truy cập. Nghĩa là, nó có thể là tích cực hoặc lười biếng tùy thuộc vào cách chúng ta cần. Theo mặc định, filter là tích cực, và mỗi khi bạn sử dụng filter, một danh sách được tạo ra.

Để làm cho bộ lọc lười biếng, ta có thể sử dụng một **Sequence**, là một tập hợp chỉ có thể nhìn vào một mục tại một thời điểm, bắt đầu từ đầu và đi đến cuối. Thuận tiện, đây chính là API mà một bộ lọc lười biếng cần.

Sequence là các cấu trúc dữ liệu sử dụng đánh giá lười biếng, và có thể được sử dụng với các bộ lọc để làm cho chúng lười biếng.

```
val instruments = listOf("viola", "cello", "violin")
val filtered = instruments.asSequence().filter { it[0] == 'v'}
println("filtered: " + filtered)
```

=> filtered: kotlin.sequences.FilteringSequence@386cc1c4

Sequence cho phép bạn xử lý các phần tử từng cái một cho đến khi một điều kiện lọc được thỏa mãn, giảm thiểu việc xử lý không cần thiết.

Ở đây, chúng ta đánh giá bộ lọc bằng một Sequence với asSequence(). Gán sequence cho một biến có tên là filtered, và in nó.

Khi trả về kết quả bộ lọc dưới dạng một sequence, biến filtered sẽ không chứa một danh sách mới, nó sẽ chứa một sequence của các phần tử danh sách và kiến thức về bộ lọc để áp dụng cho các phần tử đó. Bất cứ khi nào ta truy cập các phần tử của sequence, bộ lọc được áp dụng, và kết quả được trả về.

Sequence có thể được chuyển đổi lại thành danh sách bằng toList().

```
val filtered = instruments.asSequence().filter { it[0] == 'v'}
val newList = filtered.toList()
println("new list: " + newList)
```

=> new list: [viola, violin]

Các phép biến đổi danh sách khác

Hàm **map()** thực hiện cùng một phép biến đổi trên mọi mục và trả về danh sách mới với số lượng phần tử tương đương.

```
val numbers = setOf(1, 2, 3)
println(numbers.map { it * 3 })
```

=> [3, 6, 9]

Hàm **flatten()** trả về một danh sách duy nhất của tất cả các phần tử của các tập hợp lồng nhau, số lượng phần tử bằng tổng số lượng phần tử của các danh sách con.

```
val numberSets = listOf(setOf(1, 2, 3), setOf(4, 5), setOf(1, 2))
println(numberSets.flatten())
```

=> [1, 2, 3, 4, 5, 1, 2]

2.6. Bài tập

Bài tập chương 2: https://developer.android.com/courses/pathways/android-development-with-kotlin-2

Chương 3
Lớp và đối tượng

3.1. Lớp 39
3.2. Kế thừa 46
3.3. Các hàm mở rộng 49
3.4. Các lớp đặc biệt 50
3.5. Tổ chức mã nguồn 55
3.6. Bài tập 57

3.1. Lớp

Lớp là bản thiết kế cho các đối tượng. Mỗi lớp có thể chứa các thuộc tính và phương thức để thao tác trên đối tượng. Bạn có thể có các lớp khác nhau cho các loại đối tượng khác nhau.

Ví dụ, chúng ta có một lớp (hoặc bản thiết kế) cho một Ngôi nhà. Từ bản thiết kế, chúng ta có thể tạo các thực thể đối tượng Ngôi nhà thực tế. Mỗi thực thể đối tượng Ngôi nhà có tất cả các trường và phương thức được liệt kê trong định nghĩa của lớp.

Định nghĩa của một lớp được viết trong một tệp Kotlin.

Lớp Ngôi nhà

Dữ liệu

- Màu sắc ngôi nhà (String)
- Số lượng cửa sổ (Int)
- Có đang rao bán (Boolean)

Hành vi

- updateColor()
- putOnSale()

Lớp Ngôi nhà chứa bản thiết kế của những gì tạo nên một Ngôi nhà. Nó có các thuộc tính như màu sắc (là một String), số lượng cửa sổ (một Int), và liệu ngôi nhà có đang rao bán hay không (một Boolean).

Lớp Ngôi nhà cũng chứa các phương thức như cập nhật màu sắc ngôi nhà hoặc đặt ngôi nhà vào trạng thái rao bán.

Chúng ta sử dụng lớp để tạo các thực thể đối tượng của lớp. Ở bên phải là 3 thực thể đối tượng Ngôi nhà khác nhau có các thuộc tính khác nhau. Chúng có màu sắc khác nhau và một trong số đó thậm chí đang rao bán.

Định nghĩa và sử dụng lớp

Định nghĩa lớp

```
class House {
    val color: String = "white"
    val numberOfWindows: Int = 2
    val isForSale: Boolean = false
    fun updateColor(newColor: String){...}
        ...
    }
```

Tạo một thực thể đối tượng mới

```
val myHouse = House()
println(myHouse)
```

Hãy xem xét mã nguồn để định nghĩa và sử dụng một lớp. Để định nghĩa một lớp, sử dụng từ khóa **class** theo sau là tên của lớp, trong trường hợp này là House. Sau đó, sử dụng dấu ngoặc nhọn bao quanh phần thân của lớp. Bên trong định nghĩa lớp này, chúng ta thấy 3 thuộc tính và 1 hàm.

Sử dụng cú pháp ở bên phải để tạo một thực thể đối tượng mới của lớp House. Sử dụng tên lớp theo sau bởi dấu ngoặc đơn. Trong các ngôn ngữ khác, bạn có thể sử dụng từ khóa "new" để tạo các thực thể đối tượng mới, nhưng trong Kotlin không có từ khóa "new". Đây là một lý do khác khiến Kotlin là một ngôn ngữ ngắn gọn hơn!

Hàm khởi tạo Khi một hàm khởi tạo được định nghĩa trong khai báo lớp, nó có thể chứa:

- Không có tham số

```
class A
```

- Có tham số
  - Không được đánh dấu bằng var hoặc val → bản sao chỉ tồn tại trong phạm vi của hàm khởi tạo.

    ```
    class B(x: Int)
    ```

  - Được đánh dấu bằng var hoặc val → bản sao tồn tại trong tất cả các thực thể của lớp.

    ```
    class C(val y: Int)
    ```

Khai báo lớp bao gồm tên lớp, tiêu đề (chỉ định các tham số kiểu, hàm tạo chính, v.v.) và phần thân lớp, được bao quanh bởi dấu ngoặc nhọn. Cả tiêu đề và phần thân đều là tùy chọn; nếu lớp không có phần thân, dấu ngoặc nhọn có thể được bỏ qua, như trong ví dụ trên.

Một lớp trong Kotlin có thể có một hàm khởi tạo chính và một hoặc nhiều hàm khởi tạo phụ. Hàm khởi tạo chính là một phần của tiêu đề lớp: nó xuất hiện sau tên lớp (và các tham số kiểu tùy chọn).

Đây là các ví dụ khác nhau về cách bạn có thể định nghĩa các hàm khởi tạo với hoặc không có tham số.

Hàm khởi tạo cho lớp A không có tham số.

```
class A
val aa = A()
```

Hàm khởi tạo cho lớp B có 1 tham số đầu vào: x là một Int. Vì tham số không được đánh dấu là var hoặc val, biến x không tồn tại ngoài phạm vi của hàm khởi tạo. Do đó, nếu chúng ta tạo một thực thể đối tượng có tên là bb và cố gắng gọi thuộc tính x trên nó, chúng ta sẽ nhận được lỗi biên dịch. Thuộc tính x không tồn tại trên đối tượng.

```
class B(x: Int)
val bb = B(12)
println(bb.x)
```

=> lỗi biên dịch tham chiếu không xác định

Trong trường hợp thứ ba, chúng ta có một hàm khởi tạo cho lớp C với 1 tham số đầu vào: một val có tên là y. Nếu bạn tạo một thực thể đối tượng có tên là cc, bạn có thể truy cập thuộc tính y, trong trường hợp này có giá trị là 42.

```
class C(val y: Int)
val cc = C(42)
println(cc.y)
```

=> 42

Tóm lại, bạn có thể định nghĩa các thuộc tính trực tiếp trong hàm tạo, sử dụng var hoặc val (như trong ví dụ thứ ba).

Các thực thể lớp có thể có giá trị mặc định.

- Sử dụng giá trị mặc định để giảm số lượng hàm tạo cần thiết
- Các tham số mặc định có thể được kết hợp với các tham số bắt buộc
- Ngắn gọn hơn (không cần có nhiều phiên bản hàm tạo)

Tương tự như cách chúng ta đã học về các tham số mặc định trong hàm ở Bài 2, chúng ta có thể có các tham số mặc định cho các hàm tạo.

Điều đó có nghĩa là các tham số có thể có giá trị mặc định và không cần được chỉ định khi đối tượng được khởi tạo. Nếu không cung cấp giá trị mặc định, thì tham số là bắt buộc trong hàm tạo. Bạn có thể kết hợp các tham số mặc định và bắt buộc trong một hàm tạo.

```
class Box(val length: Int, val width:Int = 20, val height:Int = 40)
val box1 = Box(100, 20, 40)
val box2 = Box(length = 100)
val box3 = Box(length = 100, width = 20, height = 40)
```

Đối với khai báo lớp Box này, width và height có giá trị mặc định. Không có giá trị mặc định được cung cấp cho length, vì vậy việc chỉ định length là bắt buộc khi tạo một đối tượng Box mới. Vì ba tham số này được đánh dấu bằng val, chúng ta biết đây là các biến thành viên của lớp Box.

Dưới khai báo lớp, chúng ta có ba ví dụ về cách bạn sẽ tạo các đối tượng Box mới. Mặc dù chúng chỉ định các đối số hàm tạo theo các cách khác nhau, ba đối tượng Box này là tương đương.

Hàm khởi tạo chính Đối với tất cả các ví dụ chúng ta đã thấy cho đến nay, hàm khởi tạo nằm trong tiêu đề lớp. Đây được gọi là hàm khởi tạo chính.

Cú pháp này làm cho Kotlin ngắn gọn hơn khi định nghĩa các lớp.

```
class Circle(i: Int) {
    init {
        ...
    }
}

class Circle {
    constructor(i: Int) {
    ...
    }
}
```

Về mặt kỹ thuật, đoạn mã đầu tiên tương đương với đoạn thứ hai (dài dòng hơn). Đoạn mã thứ hai trông giống như cách bạn sẽ định nghĩa một lớp trong một ngôn ngữ khác, nơi hàm tạo được định nghĩa rõ ràng một mình.

Nhưng trong Kotlin, hãy viết theo đoạn mã đầu tiên. Bạn có để ý đến khối init? Hãy xem xét điều đó.

Trong Kotlin, bạn không thể đặt bất kỳ mã nào bên trong hàm khởi tạo chính, vì vậy hãy đặt đoạn mã khởi tạo trong một khối init - còn được gọi là khối khởi tạo. Bạn có thể có nhiều hơn 1 khối init trong định nghĩa lớp. Các khối này chỉ được thực thi theo thứ tự xuất hiện trong mã. Các khối init này thực chất trở thành phần thân của hàm tạo chính.

```
class Square(val side: Int) {
    init {
        println(side * 2)
    }
}

val s = Square(10)
```

=> 20

Trong ví dụ này, chúng ta có một lớp Square nơi hàm khởi tạo chính có 1 tham số đầu vào: độ dài cạnh là một số nguyên kiểu Int. Chúng ta muốn thực hiện một

số công việc trong hàm tạo của lớp Square, vì vậy chúng ta thiết lập một khối init. Bên trong nó, chúng ta có một câu lệnh println.

Sau khi định nghĩa lớp, chúng ta tạo một thực thể đối tượng Square mới với cạnh là 10. Ngay khi nó được khởi tạo, khối init được thực thi và câu lệnh println sẽ in giá trị 20 ra màn hình.

Điều gì xảy ra nếu chúng ta cần nhiều hàm khởi tạo? Ngoài hàm khởi tạo chính, một lớp có thể có một hoặc nhiều hàm khởi tạo phụ. Sử dụng từ khóa constructor để khai báo chúng.

Một hàm khởi tạo phụ phải gọi hàm khởi tạo chính bằng từ khóa **this**. Hoặc nó phải gọi một hàm khởi tạo phụ khác mà hàm đó có lời gọi hàm khởi tạo chính.

Cảnh báo: Việc sử dụng nhiều hàm tạo dẫn đến nhiều đường dẫn mã và các vấn đề với việc kiểm thử. Trước khi viết một hàm tạo phụ, hãy xem xét liệu một hàm khởi tạo khác có hoạt động thay thế hay không, để giữ định nghĩa lớp gọn gàng.

```
class Circle(val radius:Double) {
    constructor(name:String) : this(1.0)
    constructor(diameter:Int) : this(diameter / 2.0) {
        println("in diameter constructor")
    }
    init {
        println("Area: ${Math.PI * radius * radius}")
    }
}

val c = Circle(3)
```

Trong lớp Circle, chúng ta có một hàm khởi tạo chính (nhận giá trị bán kính kiểu Double làm đầu vào) và 2 hàm khởi tạo phụ.

Để định nghĩa một hàm khởi tạo phụ, bắt đầu với từ khóa constructor theo sau bởi các tham số, một dấu hai chấm, và một lời gọi đến hàm khởi tạo mặc định với this (nhận bán kính làm đầu vào). Đối với các lớp có nhiều hàm tạo như thế này, khối init chạy trước bất kỳ mã nào trong hàm tạo phụ. Do đó, câu lệnh println sẽ được thực thi trước bất kỳ mã nào trong các hàm tạo phụ.

Thuộc tính

Định nghĩa các thuộc tính trong một lớp bằng **val** hoặc **var**, như bạn đã học trước đó. Truy cập các thuộc tính này bằng cách sử dụng tên đối tượng theo sau bởi "." và tên thuộc tính. Nếu thuộc tính được khai báo là **var**, thì bạn cũng có thể

thiết lập thuộc tính bằng ký pháp chấm. Bạn không thể sửa đổi thuộc tính của một đối tượng nếu nó được khai báo là **val**. Bạn sẽ nhận được lỗi rằng **val** không thể được gán lại.

```
class Person(var name: String)
fun main() {
    val person = Person("Alex")
    println(person.name)   // Truy cập với .<tên thuộc tính>
    person.name = "Joey"   // Thiết lập với .<tên thuộc tính>
    println(person.name)
}
```

Trong ví dụ này, chúng ta tạo một lớp Person với một thuộc tính name. Trong hàm main, tạo một thực thể đối tượng Person. Sau đó truy cập và thiết lập thuộc tính name của nó bằng ký pháp chấm. Vì name là một var, chúng ta có thể thay đổi thuộc tính name bằng setter. Nếu nó được khai báo với val trong hàm tạo của lớp Person, chúng ta sẽ không thể thay đổi nó.

Nếu bạn không muốn hành vi get/set mặc định cho một thuộc tính, thì bạn có thể tùy chỉnh getter hoặc setter của riêng bạn cho một thuộc tính. Một lần nữa, bạn chỉ có thể ghi đè set() nếu thuộc tính được định nghĩa là var.

Đây là định dạng chung để khai báo một thuộc tính có thể thay đổi trong một lớp. Khai báo nó với var và sau đó là tên thuộc tính. Sử dụng dấu hai chấm theo sau bởi kiểu dữ liệu. Sau đó, đặt nó bằng một giá trị ban đầu (có thể là một biểu thức). Sau đó, bạn có thể tùy chọn ghi đè hàm get() hoặc set() cho thuộc tính.

```
class Person(val firstName: String, val lastName:String) {
    val fullName:String
    get() {
        return "$firstName $lastName"
    }
}
val person = Person("John", "Doe")
println(person.fullName)
```

=> John Doe

Hãy xem xét một ví dụ trong lớp Person. Định nghĩa một biến có tên là fullName, với một hàm getter tùy chỉnh kết hợp tên và họ thành một String duy nhất. Bạn có

thể truy cập fullName như một thuộc tính (sử dụng person.fullName trong ví dụ), và hàm get() được thực thi ngay lập tức.

```
var fullName:String = ""
get() = "$firstName $lastName"
set(value) {
    val components = value.split(" ")
    firstName = components[0]
    lastName = components[1]
    field = value
}
person.fullName = "Jane Smith"
```

Đối với thuộc tính đó, bạn cũng có thể định nghĩa một hàm setter tùy chỉnh nhận một chuỗi duy nhất và chia nó thành các thành phần tên và họ. Bạn có thể thiết lập fullName như một thuộc tính, sử dụng person.fullName = <tên mới>, là "Jane Smith" trong ví dụ. Hàm set() cơ bản sẽ được gọi.

Hàm thành viên

Ngoài các hàm tạo và thuộc tính, các lớp cũng có thể có các hàm thành viên. Khai báo các hàm theo các quy tắc chúng ta đã thảo luận trong "Các hàm" ở Bài 2.

- Sử dụng từ khóa fun
- Có thể có các tham số mặc định hoặc bắt buộc
- Chỉ định kiểu trả về (nếu không phải là Unit)

3.2. Kế thừa

Các lớp rất hữu ích, và bạn có thể nhận được nhiều lợi ích hơn từ chúng bằng cách kế thừa các thuộc tính và hành vi từ các lớp khác. Hãy nói về kế thừa trong Kotlin ở bài tiếp theo.

Kotlin có cấu trúc kế thừa chỉ từ một lớp cha duy nhất. Lớp con kế thừa tất cả các thuộc tính và phương thức của lớp cha bao gồm cả nhưng thuộc tính và phương thức mà lớp cha được kế thừa.

Nếu không muốn bị giới hạn bởi việc kế thừa từ một lớp, ta có thể định nghĩa thêm các giao diện (interface) khi mà có thể thực hiện bất kỳ giao diện nào nếu cần.

Các giao diện

Cung cấp các thuộc tính và phương thức mà các lớp thực hiện cần phải tuân theo. Giao diện chứa định nghĩa các phương thức và tên các thuộc tính, hoặc cũng có thể được kế thừa từ thuộc tính khác.

Cú pháp khai báo giao diện:

```
interface Tên_giao_diện {
    // Thân giao diện
}
```

Ví dụ về khai báo và thực hiện giao diện:

```
interface Shape {
    fun computeArea() : Double
}
class Circle(val radius:Double) : Shape {
    override fun computeArea() = Math.PI * radius * radius
}

val c = Circle(3.0)
println(c.computeArea())
```

=> 28.274333882308138

Trong ví dụ này, ta định nghĩa một giao diện tên là Shape với nội dung khai báo phương thức computeArea(), phương thức này không nhận tham số và trả về giá trị kiểu Double. Bất kỳ lớp nào thực hiện giao diện này đều phải viết chồng để định nghĩa nội dung cụ thể của phương thức computeArea() cho lớp đó. Lớp Circle thực hiện giao diện Shape và viết chồng phương thức computeArea() để tính diện tích hình tròn.

Bên dưới, một thực thể của lớp Circle được khởi tạo, gọi phương thức computeArea() để tính và in ra diện tích.

Mở rộng các lớp

Để mở rộng tính năng cho một lớp, có thể thực hiện các phương án sau:

- Tạo một lớp mới kế thừa từ lớp hiện tại.
- Thêm chức năng cho một lớp mà không cần tạo lớp mới (các hàm mở rộng).

Tạo lớp con mới

Các lớp trong Kotlin mặc định là lớp cuối, không được kế thừa. Để một lớp có thể được kế thừa, thì cần sử dụng từ khóa **open** khi khai báo lớp đó. Đối với các phương thức và thuộc tính cũng tương tự như vậy. Các phương thức và thuộc tính được viết chồng sử dụng từ khóa **override**.

```
// Định nghĩa lớp A
class A
// Định nghĩa lớp B kế thừa lớp A
class B: A()
```

=> Kotlin: This type is final, so it cannot be extended.

```
// Định nghĩa lớp C
open class C
// Định nghĩa lớp D kế thừa lớp C
class D: C()
```

Bởi vì lớp cha cần phải có một trong các hàm khởi tạo được sử dụng, ngay cả khi hàm khởi tạo không có tham số, do đó ta có thể dễ dàng phân biệt đâu là lớp cha và đâu là giao diện qua việc có hoặc không có cặp dấu đóng mở ngoặc.

Viết chồng

Các thuộc tính và phương thức cần viết chồng phải có từ khóa **open** khi khai báo. Từ khóa **override** được sử dụng để viết chồng một thuộc tính và phương thức ở lớp con. Một thuộc tính hoặc phương thức được khai báo bằng từ khóa **override** thì cũng có thể được viết chồng, trừ khi chúng được đánh dấu bởi từ khóa **final**.

Các lớp trừu tượng

Lớp trừu tượng được khai báo bởi từ khóa **abstract**, các lớp này không được sử dụng để khởi tạo các đối tượng mà được sử dụng làm lớp cơ sở cho các lớp con.

Lớp trừu tượng hoạt động tương tự như giao diện với việc thêm khả năng lưu trữ thông tin, nghĩa là ta có thể khởi tạo giá trị cho các thuộc tính hoặc định nghĩa nội dung của các phương thức.

Các thuộc tính và phương thức được khai báo bởi từ khóa **abstract** thì bắt buộc phải được viết chồng. Lớp trừu tượng cũng có thể bao gồm cả các thuộc tính và phương thức không có từ khóa abstract, không nhất thiết phải viết chồng ở các lớp con.

Ví dụ về lớp trừu tượng:

```
abstract class Food {
    abstract val kcal : Int
    abstract val name : String
    fun consume() = println("I'm eating ${name}")
}
class Pizza() : Food() {
    override val kcal = 600
    override val name = "Pizza"
}
fun main() {
    Pizza().consume()    // "I'm eating Pizza"
}
```

Lớp trừu tượng chia sẻ một vài điểm chung với giao diện khi chúng đều không được sử dụng trực tiếp và có các phương thức và thuộc tính cần phải viết chồng ở lớp con. Lớp trừu tượng khác với giao diện ở chỗ nó có thể có các thuộc tính hoặc phương thức được định nghĩa một cách đầy đủ và được kế thừa ở lớp con. Trong ví dụ trên, lớp Pizza kế thừa phương thức **consume()** từ lớp trừu tượng Food, nó không phải định nghĩa lại phương thức này nhưng phải viết chồng thuộc tính **kcal** và **name**.

3.3. Các hàm mở rộng

Hàm mở rộng trong Kotlin là một tính năng mạnh mẽ cho phép "mở rộng" các lớp hiện có bằng cách thêm các hàm mới mà không cần sửa đổi mã nguồn của lớp gốc. Điều này giúp code trở nên sạch sẽ, dễ đọc hơn và hỗ trợ lập trình hướng đối tượng mà không làm thay đổi cấu trúc lớp ban đầu.

Cú pháp định nghĩa hàm mở rộng:

```
fun ClassName.functionName(params) {
    // Thân hàm
}
```

- Khi sử dụng hàm mở rộng, cách gọi chúng giống hệt như gọi một phương thức thành viên của lớp gốc. Ví dụ, nếu bạn mở rộng lớp String với hàm fun String.isEmail(): Boolean ... , thì bạn có thể gọi myString.isEmail() – trông như thể lớp String (do Kotlin cung cấp) đã có sẵn hàm này từ đầu. Đây là thiết kế của Kotlin để làm cho code dễ đọc và tự nhiên hơn. Tuy nhiên, đây chỉ là

"ảo giác" cú pháp; hàm mở rộng thực chất là các hàm tĩnh (static functions) được biên dịch dưới dạng gọi tĩnh, không phải là phần của lớp gốc.

- Hàm mở rộng không thay đổi bất kỳ thứ gì trong lớp gốc. Chúng chỉ thêm chức năng mới ở mức biên dịch, và mã bytecode của lớp gốc vẫn giữ nguyên. Ví dụ, nếu bạn mở rộng lớp *List<Int>* với hàm fun *List<Int>.sumEven(): Int ...* , lớp *List* không bị chỉnh sửa; hàm này chỉ tồn tại trong scope của file hoặc module nơi bạn định nghĩa nó.
- Hàm mở rộng không phải là thành viên thực sự của lớp, nên chúng không thể truy cập các thành viên private hoặc protected của lớp đó. Ví dụ, nếu lớp *Person* có biến private *private val secret: String*, thì hàm mở rộng *fun Person.revealSecret() println(secret)* sẽ báo lỗi biên dịch vì không thấy *secret*.

Lý do sử dụng các hàm mở rộng:

- Trong Kotlin, nhiều lớp (đặc biệt là các lớp trong thư viện chuẩn hoặc thư viện bên thứ ba) được đánh dấu là final, nghĩa là chúng không thể được kế thừa để mở rộng chức năng. Hàm mở rộng cung cấp một cách để thêm các phương thức mới mà không cần kế thừa.
- Khi sử dụng các thư viện bên thứ ba hoặc các lớp từ Java/Kotlin Standard Library (như List, String, hoặc các lớp từ Android SDK), bạn không có quyền sửa đổi mã nguồn của chúng. Hàm mở rộng cho phép bạn thêm các phương thức tùy chỉnh mà không cần tạo lớp bao bọc (wrapper) hoặc kế thừa.
- Khi bạn sở hữu một lớp, bạn có thể muốn giữ API cốt lõi (các phương thức chính) sạch sẽ và tập trung vào chức năng chính của lớp, trong khi các phương thức hỗ trợ (helper methods) có thể được định nghĩa bên ngoài dưới dạng hàm mở rộng.

Ví dụ định nghĩa hàm mở rộng:

```
fun Int.isOdd(): Boolean { return this % 2 == 1 }

\\ Call isOdd() on an Int
3.isOdd()
```

3.4. Các lớp đặc biệt

Lớp dữ liệu (data class)

Data class trong Kotlin là một loại lớp đặc biệt được thiết kế để lưu trữ dữ liệu một cách đơn giản và hiệu quả. Chúng được tối ưu hóa để giảm bớt mã lệnh (boilerplate code) mà lập trình viên phải viết, nhờ vào các phương thức tự động sinh ra bởi Kotlin.

- Data class được tạo ra với mục đích chính là lưu trữ dữ liệu, tương tự như các lớp POJO (Plain Old Java Object) trong Java, nhưng với cú pháp ngắn gọn hơn và nhiều tính năng tích hợp. Chúng được sử dụng để đại diện cho các thực thể dữ liệu (như User, Product, v.v.) mà không cần chứa nhiều logic phức tạp.
- Để khai báo một data class, bạn cần thêm từ khóa data trước từ khóa class. Từ khóa này báo cho trình biên dịch Kotlin biết rằng lớp này là một data class, từ đó tự động sinh ra các phương thức đặc biệt (như toString(), equals(), hashCode(), v.v.).
- Khi bạn khai báo một data class, Kotlin tự động tạo các getter cho tất cả các thuộc tính (val hoặc var) và setter cho các thuộc tính có thể thay đổi (var) trong constructor chính. Điều này giúp truy cập và sửa đổi dữ liệu một cách dễ dàng.
- Data class tự động cung cấp các phương thức tiện ích sau:
  - toString(): Trả về một chuỗi đại diện cho đối tượng, hiển thị tất cả các thuộc tính trong constructor chính.
  - equals(): So sánh hai đối tượng dựa trên giá trị của các thuộc tính trong constructor chính.
  - hashCode(): Tạo mã băm (hash code) dựa trên các thuộc tính trong constructor chính, đảm bảo tính nhất quán với equals() để sử dụng trong các tập hợp như HashMap hoặc HashSet.
  - copy(): Tạo một bản sao của đối tượng với khả năng thay đổi một số thuộc tính.
  - Destructuring operators (Toán tử phân rã): Cho phép phân rã đối tượng thành các biến riêng lẻ.

Cú pháp định nghĩa lớp dữ liệu:

```
data class <Tên_Lớp>( // Danh sách thuộc tính )
```

Ví dụ định nghĩa lớp dữ liệu:

```
// Định nghĩa lớp
data class Player(val name: String, val score: Int)

// Sử dụng lớp
val firstPlayer = Player("Lauren", 10)
println(firstPlayer)
// => Player(name=Lauren, score=10)
// Phân rã thành các biến riêng lẻ
val (name, score) = firstPlayer
```

Lớp Pair và lớp Triple

Pair và Triple là các lớp đã được định nghĩa trước cho phép lưu 2 hoặc 3 thành phần dữ liệu trong một đối tượng. Để truy nhập vào các thành phần này, ta sử dụng thuộc tính .first, .second và .third. Tuy nhiên các lớp dữ liệu tự định nghĩa sẽ là lựa chọn tốt hơn khi nó chứa các tên có ý nghĩa hơn.

Ví dụ:

```
val bookAuthor = Pair("Harry Potter", "J.K. Rowling")
println(bookAuthor)
```

=> (Harry Potter, J.K. Rowling)

```
val bookAuthorYear = Triple("Harry Potter", "J.K. Rowling", 1997)
println(bookAuthorYear)
println(bookAuthorYear.third)
```

=> (Harry Potter, J.K. Rowling, 1997)
1997

Chú ý: Các thuộc tính first, second, và third có thể có các kiểu dữ liệu khác nhau, như trong ví dụ *bookAuthorYear*.

Lớp Pair có một hàm đặc biệt là hàm to cho phép ghép 2 đối tượng với nhau, đây là hàm infix, có thể lược bỏ dấu ngoặc và dấu chấm khi gọi hàm.

Ví dụ:

```
val bookAuth1 = "Harry Potter".to("J. K. Rowling")
val bookAuth2 = "Harry Potter" to "J. K. Rowling"
```

=> bookAuth1 and bookAuth2 are Pair (Harry Potter, J. K. Rowling)

Và cũng được sử dụng trong các dữ liệu tập hợp như Map và HashMap. Ví du:

```
val map = mapOf(1 to "x", 2 to "y", 3 to "zz")
```

=> map of Int to String 1=x, 2=y, 3=zz

Lớp enum

Lớp enum (enum class) trong Kotlin là một loại lớp đặc biệt được sử dụng để định nghĩa một tập hợp các giá trị hằng số (constants). Chúng rất hữu ích khi bạn muốn giới hạn một biến hoặc đối tượng chỉ nhận một trong số các giá trị được xác định trước.

- Lớp enum được dùng để biểu diễn một tập hợp hữu hạn các giá trị cố định, gọi là các hằng số enum (enum constants). Mỗi hằng số là một instance của lớp enum và được đảm bảo chỉ thuộc tập hợp các giá trị đã định nghĩa. Điều này rất hữu ích khi bạn muốn mô hình hóa các lựa chọn cố định, như trạng thái, ngày trong tuần, hoặc các loại cụ thể.
- Trong Kotlin, các hằng số của lớp enum (như MONDAY, TUESDAY trong ví dụ trên) không tự động ánh xạ trực tiếp đến một giá trị nguyên thủy cụ thể (như số nguyên hoặc chuỗi) mà bạn có thể thấy ngay lập tức. Thay vào đó, chúng là các đối tượng (instances) của lớp enum, và giá trị của chúng được xử lý nội bộ bởi Kotlin.
- Để khai báo một lớp enum trong Kotlin, bạn sử dụng từ khóa **enum** trước từ khóa **class**, theo sau là tên lớp và danh sách các hằng số. Cú pháp này báo cho trình biên dịch rằng đây là một lớp enum, từ đó cung cấp các tính năng đặc biệt như khả năng liệt kê các hằng số hoặc sử dụng chúng trong các biểu thức when.

Cú pháp định nghĩa lớp enum:

```
enum class EnumClass {NAME_1, NAME_2, ..., NAME_n}
```

Truy nhập theo cú pháp EnumClass.<ConstantName>

Ví dụ về lớp enum:

```
enum class Color(val r: Int, val g: Int, val b: Int) {
   RED(255, 0, 0), GREEN(0, 255, 0), BLUE(0, 0, 255)
}

println("" + Color.RED.r + " " + Color.RED.g + " " + Color.RED.b)
```

=> 255 0 0

Object/singleton

Trong Kotlin, object là một cách đặc biệt để định nghĩa một lớp với một instance duy nhất (singleton), đảm bảo rằng chỉ có một đối tượng của lớp đó tồn tại trong suốt vòng đời của ứng dụng. Đây là một cách tích hợp để triển khai mẫu thiết kế Singleton (Singleton Pattern) mà không cần viết mã phức tạp như trong Java.

- Trong một số trường hợp, bạn muốn đảm bảo rằng chỉ có một instance duy nhất của một lớp được tạo ra trong ứng dụng, ví dụ như để quản lý cấu hình toàn cục, kết nối cơ sở dữ liệu, hoặc các tài nguyên dùng chung. Object trong Kotlin tự động đảm bảo rằng chỉ có một instance được tạo và cung cấp điểm truy cập toàn cục cho instance này.
- Để khai báo một singleton trong Kotlin, bạn sử dụng từ khóa object thay vì class. Điều này báo cho trình biên dịch rằng đây là một đối tượng singleton, và Kotlin sẽ tự động tạo một instance duy nhất của đối tượng này khi nó được truy cập lần đầu tiên.
- Vì object là một singleton, bạn truy cập các thuộc tính và phương thức của nó trực tiếp thông qua tên của đối tượng, tương tự như cách truy cập các thành viên tĩnh (static) trong Java. Không cần tạo instance, vì instance duy nhất đã được Kotlin quản lý.

Ví dụ về object/singleton:

```
object Calculator {
    fun add(n1: Int, n2: Int): Int {
        return n1 + n2
    }
}

println(Calculator.add(2,4))
```

=> 6

Companion objects

Companion object trong Kotlin là một đối tượng đặc biệt được liên kết với một lớp, cho phép tất cả các instance của lớp đó chia sẻ một tập hợp các biến hoặc hàm duy nhất. Nó tương tự như các thành viên tĩnh (static) trong Java, nhưng được triển khai theo cách hướng đối tượng hơn.

- Companion object là một singleton được liên kết với một lớp, chứa các biến và hàm mà tất cả các instance của lớp đó có thể truy cập. Nó được sử dụng để lưu trữ các thuộc tính hoặc hành vi chung, không phụ thuộc vào từng instance cụ thể của lớp. Điều này rất hữu ích cho các trường hợp như hằng số, hàm tiện ích, hoặc factory methods.
- Để khai báo một companion object, bạn sử dụng từ khóa companion bên trong khối object trong một lớp. Từ khóa này báo cho Kotlin rằng đây là một đối tượng đặc biệt liên kết với lớp, và các thành viên của nó có thể được truy cập thông qua tên lớp mà không cần tạo instance.
- Các thành viên của companion object được truy cập trực tiếp thông qua tên lớp, tương tự như cách truy cập các thành viên tĩnh trong Java. Bạn không cần tạo instance của lớp để sử dụng các thuộc tính hoặc hàm trong companion object.

```
class PhysicsSystem {
    companion object WorldConstants {
        val gravity = 9.8
        val unit = "metric"
        fun computeForce(mass: Double, accel: Double): Double {
            return mass * accel
        }
    }
}
println(PhysicsSystem.WorldConstants.gravity)
println(PhysicsSystem.WorldConstants.computeForce(10.0, 10.0))
```

=>9.8
100.0

3.5. Tổ chức mã nguồn

Một file, nhiều thực thể

Trong nhiều ngôn ngữ lập trình như Java, có một quy ước phổ biến là mỗi tệp chỉ nên chứa một class hoặc interface công khai (public). Điều này xuất phát từ việc Java yêu cầu tên tệp phải khớp với tên của class/interface công khai duy nhất trong tệp đó. Tuy nhiên, Kotlin không áp đặt quy ước này, mang lại sự linh hoạt hơn trong cách tổ chức mã nguồn.

Kotlin khuyến khích lập trình viên nhóm các class, interface, hoặc các cấu trúc liên quan về mặt chức năng hoặc ngữ nghĩa vào cùng một tệp, thay vì tách chúng ra thành nhiều tệp riêng lẻ. Điều này giúp mã nguồn dễ đọc, dễ bảo trì và phản ánh tốt hơn mối quan hệ giữa các thành phần.

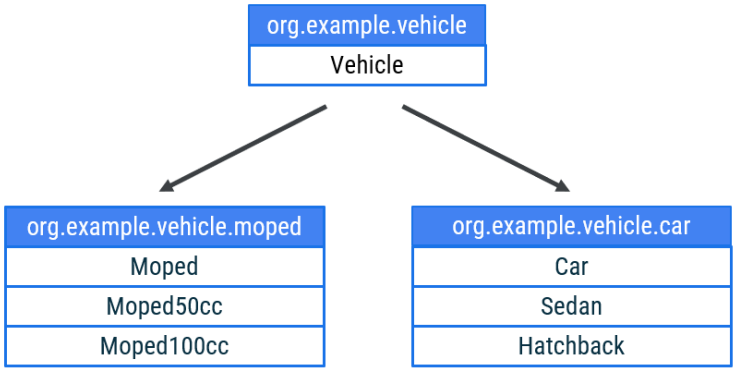
Mặc dù Kotlin cho phép đặt nhiều thực thể trong cùng một tệp, chúng ta cần cẩn thận để tránh làm cho tệp trở nên quá dài hoặc lộn xộn, vì điều này có thể gây khó khăn cho việc đọc và bảo trì mã.

Các gói

Trong Kotlin, việc tổ chức mã nguồn là rất quan trọng để đảm bảo dự án dễ đọc, dễ bảo trì và có cấu trúc rõ ràng, đặc biệt khi dự án lớn lên với nhiều tệp và module. Kotlin cung cấp một số phương tiện tổ chức mã nguồn, bao gồm: các gói (packages), tệp, hàm và biến cấp cao, object và companion object, modules.

Trong Kotlin (và Java), định danh (identifiers) của package thường được đặt tên theo quy ước sử dụng các từ in thường (lowercase), phân tách bằng dấu chấm (.). Điều này nhằm đảm bảo tính thống nhất và tránh nhầm lẫn khi làm việc với các package.

Trong Kotlin, nếu một tệp thuộc về một package cụ thể, chúng ta cần khai báo package bằng từ khóa package ở dòng mã không phải chú thích đầu tiên trong tệp. Điều này đảm bảo rằng trình biên dịch Kotlin biết được tệp thuộc không gian tên nào.



Hình trên là ví dụ về một cây phân cấp lớp. Trong ví dụ này, nếu lớp **Moped** và các lớp con của nó không quá dài thì có thể tổ chức vào một file, tương tự với lớp **Car**.

Kiểm soát phạm vi truy nhập

Trong Kotlin, visibility modifiers là các từ khóa được sử dụng để kiểm soát phạm vi truy cập (accessibility) của các thành phần như class, biến, hàm, hoặc thuộc tính.

Mục tiêu là áp dụng nguyên tắc đóng gói (encapsulation) trong lập trình hướng đối tượng, giúp:

- Ẩn các chi tiết triển khai không cần thiết.
- Bảo vệ dữ liệu khỏi việc bị truy cập hoặc sửa đổi không mong muốn.
- Tăng tính bảo trì và an toàn của mã nguồn.

Kotlin cung cấp các từ khóa điều chỉnh phạm vi:

- **public**: Có thể truy cập từ mọi nơi (mặc định).
- **private**: Chỉ truy cập được trong phạm vi class hoặc tệp chứa nó.
- **protected**: Chỉ truy cập được trong class và các lớp con (subclass).

3.6. Bài tập

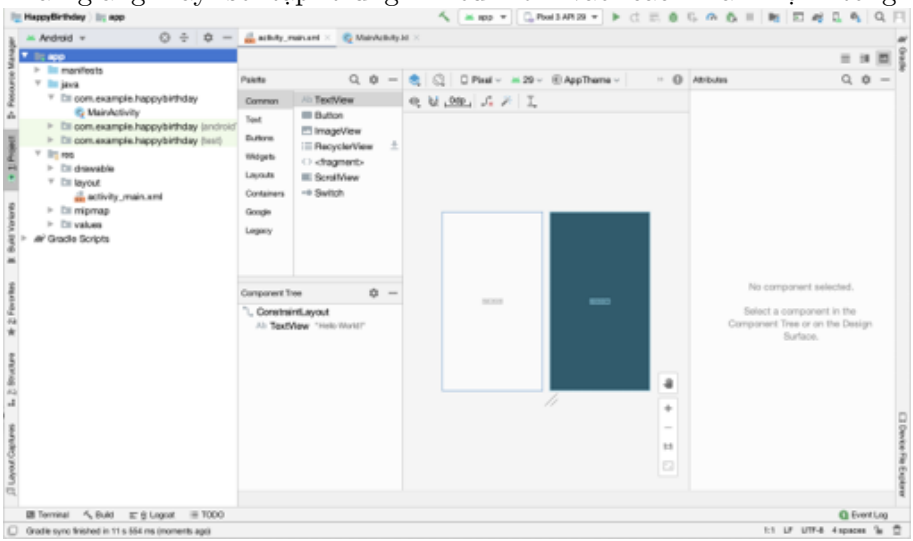
Bài tập chương 3: https://developer.android.com/courses/pathways/android-development-with-kotlin-3

Chương 4
Xây dựng ứng dụng Android đầu tiên

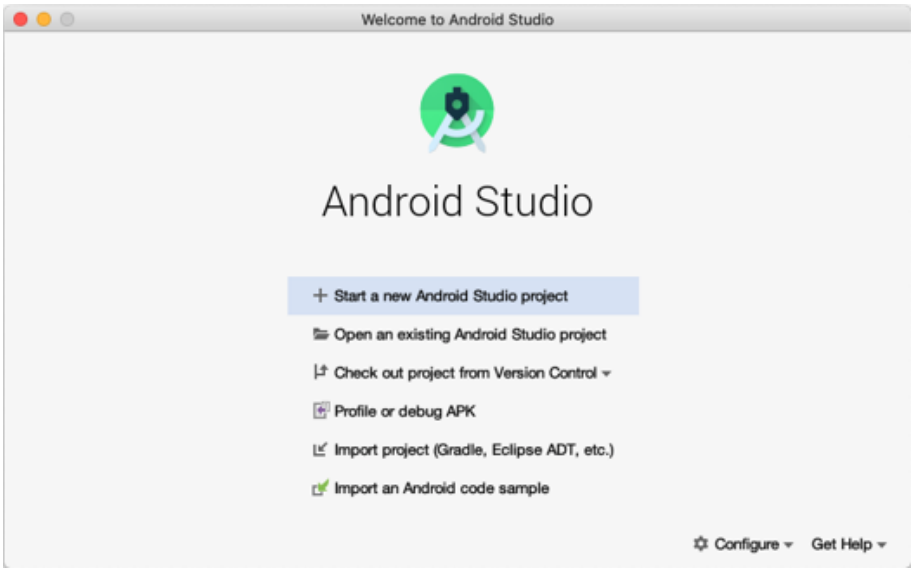
4.1. Ứng dụng đầu tiên của bạn 58
4.2. Cấu trúc của một ứng dụng Android 63
4.3. Layout và resources trong ứng dụng Android 64
4.4. Activities 68
4.5. Tạo tính tương tác cho ứng dụng 70
4.6. Gradle: Xây dựng ứng dụng Android 74
4.7. Tổng kết chương 75

4.1. Ứng dụng đầu tiên của bạn

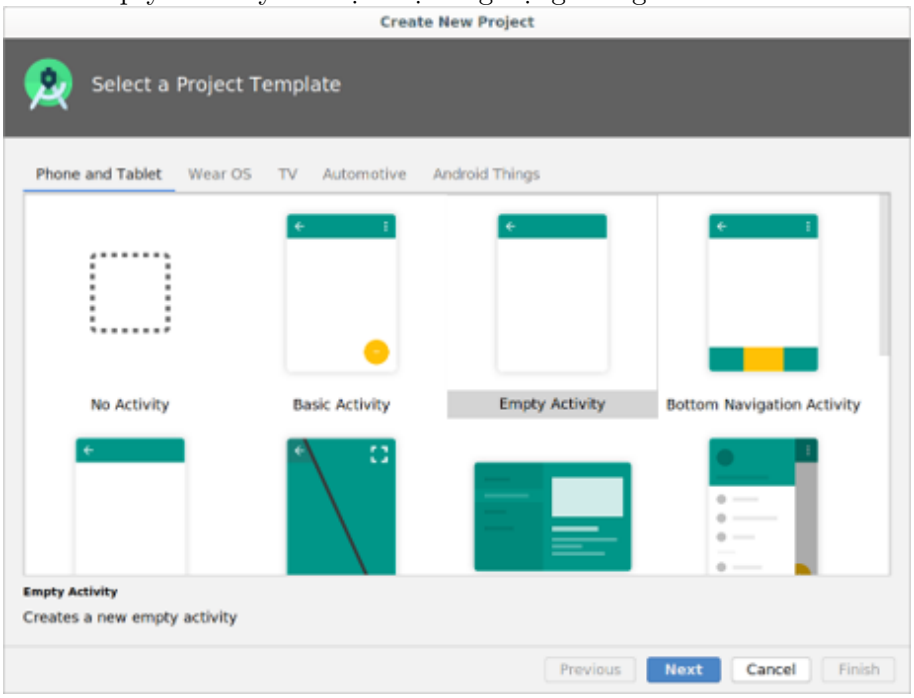
Đây là Android Studio - môi trường phát triển tích hợp (IDE) chính thức dành cho phát triển Android. Công cụ này được xây dựng trên IntelliJ IDEA và có nhiều tính năng giúp việc phát triển ứng dụng Android nhanh hơn và dễ dàng hơn. Hướng dẫn cài đặt chi tiết sẽ có trong bài thực hành. Bài giảng này sẽ tập trung nhiều hơn vào các khái niệm tổng quan.



Khi bạn mở Android Studio, hộp thoại "Welcome to Android Studio" sẽ hiển thị.



Tạo ứng dụng Android (Project Android) mới: có rất nhiều mẫu ở đây chứa các dự án khởi động giúp bạn xây dựng nhiều loại ứng dụng khác nhau. Chúng ta sẽ chọn mẫu "Empty Activity" để tạo một ứng dụng đơn giản.



Các thông tin Project:

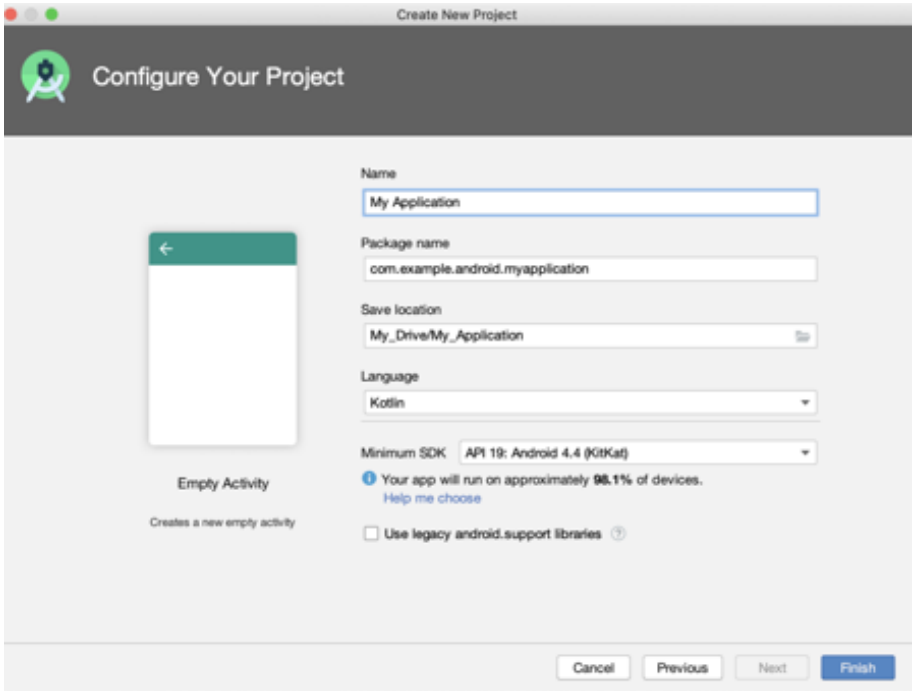
- Name - Tên ứng dụng.

- Package name - Package chứa ứng dụng, sử dụng khi đưa lên các kho ứng dụng như Google Play.

- Save location - vị trí lưu project trong máy tính của bạn.

- Language - Hiện tại cả ngôn ngữ Kotlin và Java đều được hỗ trợ. Trong khóa học này chúng ta sẽ dùng ngôn ngữ Kotlin.

- Minimum SDK - Cấu hình thiết bị tối thiểu để chạy ứng dụng. Cho Project đầu tiên này, chọn API 19: Android 4.4 (KitKat).



Platform Version	API Level	VERSION_CODE
Android 10.0	29	Q
Android 9	28	P
Android 8.1	27	O_MR1
Android 8.0	26	O
Android 7.1.1 Android 7.1	25	N_MR1
Android 7.0	24	N
Android 6.0	23	M
Android 5.1	22	LOLLIPOP_MR1
Android 5.0	21	LOLLIPOP

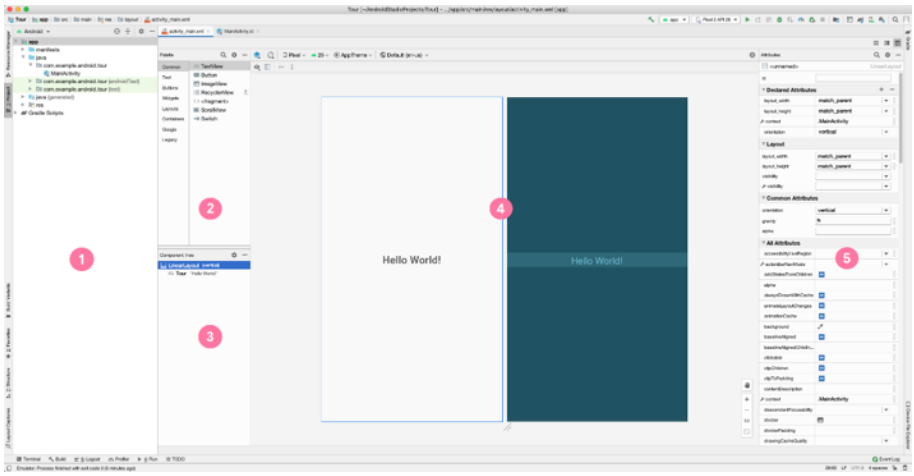
Khi bạn tạo một Project Android mới, bạn cần chọn 3 loại:

- Minimum SDK: Cấu hình thiết bị tối thiểu để chạy ứng dụng.
- Target SDK: Phiên bản cao nhất để thử nghiệm ứng dụng.
- Compile SDK: Phiên bản Android sử dụng để biên dịch ứng dụng.

minSdkVersion <= targetSdkVersion <= compileSdkVersion

Hướng dẫn sử dụng Android Studio: trong giao diện chính của Android Studio có hai thành phần cơ bản là Project window và Layout Editor. Layout Editor bao gồm Palette, Component Tree, a Design Editor, và cửa sổ Attributes.

- Project window: hiển thị cấu trúc file và thư mục của project.
- Palette: các components và layouts, bạn có thể kéo thả vào giao diện của ứng dụng như TextViews, ImageViews, và Buttons.
- Component Tree: cấu trúc phân cấp các thành phần trong giao diện của ứng dụng.
- Design Editor: hiển thị khung thiết kế giao diện của ứng dụng.
- Attributes window: danh sách các thuộc tính của từng thành phần giao diện.

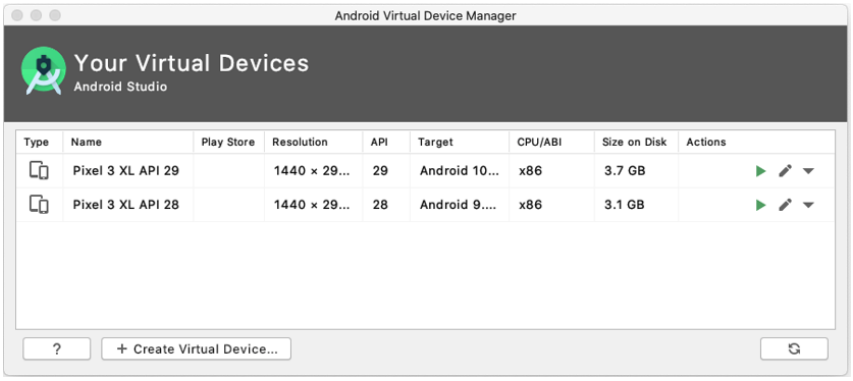


Có hai kỹ thuật thử nghiệm ứng dụng:

- Trên các thiết bị Android như smartphone, tablet.
- Trình mô phỏng trên máy tính.



Android Virtual Device (AVD) Manager: quản lý các thiết bị ảo.



4.2. Cấu trúc của một ứng dụng Android

Trong một Project Android, thành phần cơ bản bao gồm Activity, các tài nguyên (resources) và các file gradle.

- Activity: tạo giao diện hiển thị và tương tác với người dùng.
- Resources: các file tài nguyên bao gồm các file layour, ảnh, âm thanh, ...
- Các file Gradle: là các file cấu hình mô tả các thành phần cần cài đặt bổ sung vào project.

Khi bạn tạo một project Android mới thì cấu trúc project sẽ như sau:

```
MyApplication
├── app
│   ├── libs
│   └── src
│       ├── androidTest
│       ├── main
│       │   ├── java
│       │   ├── res
│       │   └── AndroidManifest.xml
│       └── test
├── build.gradle
└── gradlew
```

Các thư mục cơ bản trong project:

- app - lưu trữ mã nguồn và các file tài nguyên
- libs - lưu trữ các thư viện

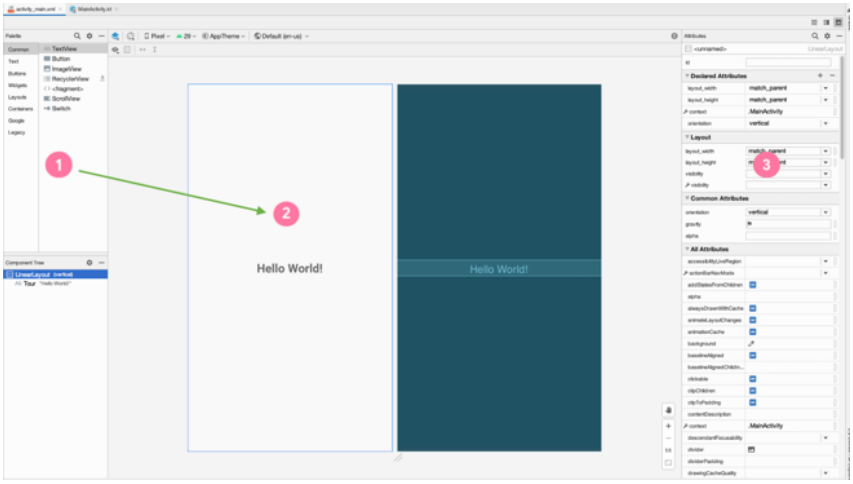
- androidTest - test code
- main - các file mã Java, Kotlin
- test - local unit tests
- AndroidManifest.xml - khai báo các thông tin cần thiết
- build.gradle - điều khiển quá trình biên dịch, thử nghiệm và triển khai ứng dụng
- gradlew - file thực thi gradle

4.3. Layout và resources trong ứng dụng Android

Bộ công cụ phát triển ứng dụng Android đã cung cấp rất nhiều các thành phần giao diện - view (thường gọi là widget) như TextView, ImageView, Button, ... Có các thuộc tính chung của các view như width, height. Ngoài ra mỗi widget còn có các thuộc tính riêng, ví dụ như với TextView có thêm các thuộc tính như font size, font family, text.

Mỗi view sẽ chiếm một vùng hình chữ nhật trên màn hình, có thể bắt sự kiện người dùng tương tác với ứng dụng (event handling). Ngoài ra có thể nhóm các view trong các nhóm biểu diễn bởi layout.

Bạn có thể bổ sung thêm view vào ứng dụng bằng cách kéo thả view từ Palette (1) vào Design view (2). Sau đó chỉnh sửa các thuộc tính ở Attributes window (3) phía bên phải màn hình.



Ngoài ra bạn thường thao tác trực tiếp các thuộc tính này trong file XML mô tả giao diện, các file này nằm trong thư mục layout.

Đây là đoạn mã XML biểu diễn một TextView trong layout. Có 3 thuộc tính của TextView: width, height, và text.

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"/>
```

Để xác định chiều rộng và chiều cao của một view, sử dụng hai thuộc tính $layout_width$ và $layout_height$.

- *wrap_content*: kích thước view vừa đủ với nội dung của view.

  ```
  android:layout_width="wrap_content"
  ```

- *match_parent*: lấy kích thước của parent chứa view. Ví dụ như nếu bạn muốn ImageView chiếm toàn màn hình thì thiết lập như sau:

  ```
  android:layout_width="match_parent"
  ```

- Muốn thiết lập kích thước cố định cho view thì dùng đơn vị dp (density-independent pixels).

  ```
  android:layout_width="48dp"
  ```

Để hiển thị nhiều view trên màn hình, sử dụng ViewGroup. ViewGroup còn hỗ trợ sắp đặt vị trí các view. Các loại ViewGroups phổ biến bao gồm:

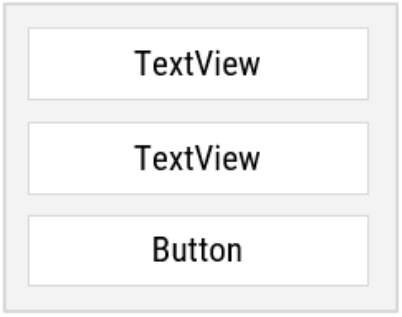
- FrameLayout: nếu chỉ có một view.
- LinearLayout: hiển thị các view theo hàng hoặc theo cột.
- ConstraintLayout: cho các layout phức tạp hơn.

Ví dụ với FrameLayout chỉ chứa một TextView. Nếu có nhiều view thì các view này sẽ bị chồng lên nhau, tuy nhiên đấy có thể là hiệu ứng mà ta mong muốn trong một số trường hợp cụ thể.

```
<FrameLayout
   android:layout_width="match_parent"
   android:layout_height="match_parent">
   <TextView
       android:layout_width="match_parent"
       android:layout_height="match_parent"
       android:text="Hello World!"/>
</FrameLayout>
```

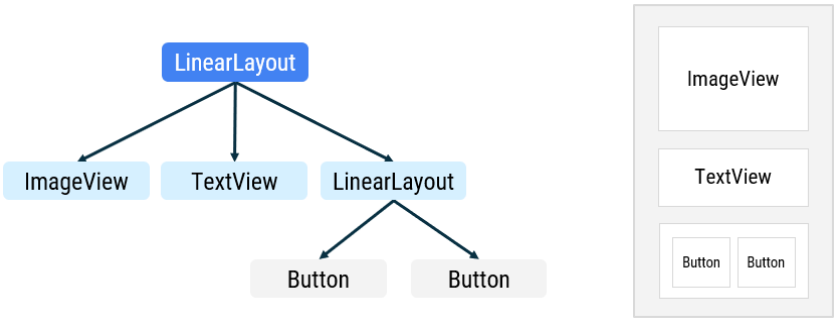
Đây là ví dụ sử dụng LinearLayout ViewGroup. LinearLayout này chứa 3 views: 2 TextViews và một Button.

LinearLayout đặt các view theo hàng hoặc theo cột, dựa vào thuộc tính orientation được thiết lập và "vertical" hay "horizontal".




```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <TextView ... />
    <TextView ... />
    <Button ... />
</LinearLayout>
```

Để thiết kế các giao diện phức tạp hơn, sử dụng View hierarchy (phân cấp view). Nghĩa là ViewGroup này lại có thể nằm trong ViewGroups khác.



App resources (các tài nguyên của ứng dụng): ngoài layout, ứng dụng còn cần các file tài nguyên khác như ảnh, âm thanh, icon, ... Android cung cấp sự hỗ trợ mạnh mẽ cho phép truy cấp các tài nguyên này từ mọi nơi trong ứng dụng. Các thành phần tài nguyên cơ bản mà ứng dụng thường sử dụng: Layout files, Images, Audio files, User interface strings, App icon. Cần đưa các file tài nguyên vào các thư mục cụ thể trong thư mục res.

```
main
├── java
└── res
    ├── drawable
    ├── layout
    ├── mipmap
    └── values
```

- Thư mục drawable: các tài nguyên ảnh.
- Thư mục layout: chứa các file layout XML files.
- Thư mục mipmap directory: chứa các icon với các độ phân giải màn hình khác nhau.
- Thư mục values: chứa các file mô tả các xâu ký tự, các mã mầu, styles, ...

Resource IDs:

- Mỗi tài nguyên sẽ được gắn một ID cụ thể. ID này sẽ được sử dụng trong mã nguồn để truy cập tới tài nguyên.

- Khi đặt tên các tài nguyên trong file XML thì về quy ước sử dụng các chữ cái thường và dấu gạch dưới giữa các từ.

- Android tự động tạo lớp có tên R.java chứa tham chiếu đến tất cả các tài nguyên trong project.

- Cách truy cập tài nguyên trong mã nguồn : R.<resource_type>.<resource_name>

- Ví dụ: R.drawable.ic_launcher (res/drawable/ic_launcher.xml)

  R.layout.activity_main (res/layout/activity_main.xml)

Ví dụ: với TextView có resource ID name "helloTextView" thì bạn có thể truy cập như sau: R.id.helloTextView. Khi đặt ID cho view, cần đảm bảo tính duy nhất cho từng view.

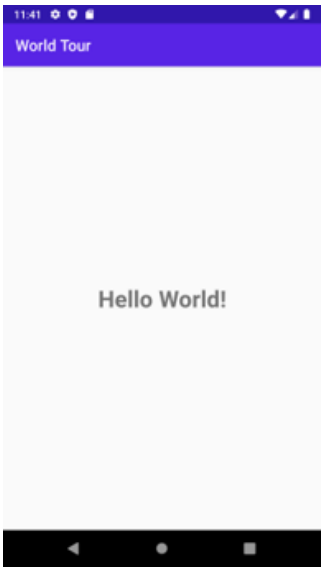
Để đặt ID cho view, sử dụng thuộc tính android:id attribute của View trong file XML với cú pháp @+id/name.

```
<TextView
 android:id="@+id/helloTextView"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:text="Hello World!"/>
```

Khi đó trong ứng dụng, bạn có thể truy cập đến TextView sử dụng R.id.helloTextView.

4.4. Activities

Activity là màn hình giao diện tương tác với người sử dụng. Một ứng dụng Android có thể có nhiều Activity.



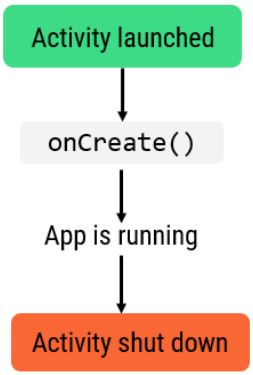
Ví dụ về Activity:

- Hiển thị danh sách các email.
- Hiển thị một danh bạ cụ thể.
- Chụp ảnh sử dụng camera.

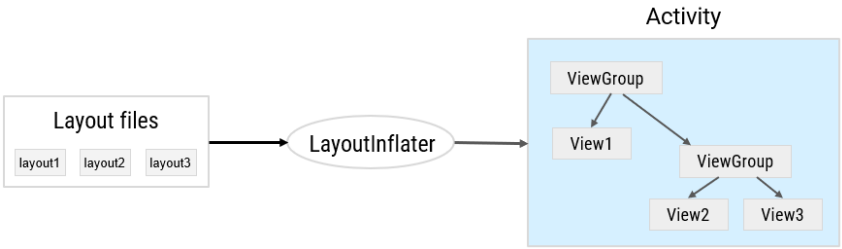
Mã nguồn của Activity được đặt trong các file Kotlin. Ví dụ file MainActivity.kt. File này sẽ được tạo ra tự động khi bạn chọn Empty Activity template khi tạo Project mới.

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Một Activity cụ thể được thừa kế từ lớp AppCompatActivity. AppCompat đảm bảo tương thích giữa các phiên bản Android. Trong lớp này có viết chồng hàm onCreate. Khi bắt đầu chạy chương trình thì Activity này được nạp lên, khi đó hàm onCreate này sẽ được gọi.

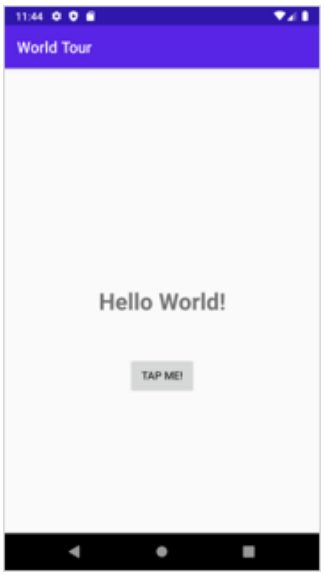


Trong hàm onCreate sẽ gọi đến hàm onCreate của lớp cha để cung cấp các thiết lập cơ bản cho Activity. Sau đó hàm setContentView() được gọi để nạp các thành phần giao diện trong file layout activity_main.xml để hiển thị lên Activity. Quá trình này được gọi là Layout inflation.



4.5. Tạo tính tương tác cho ứng dụng

Chúng ta vừa mô tả cách thức ứng dụng hiển thị giao diện thiết kế trong file layout. Bây giờ chúng ta sẽ tìm hiểu cách thức ứng dụng tương tác với người dùng.



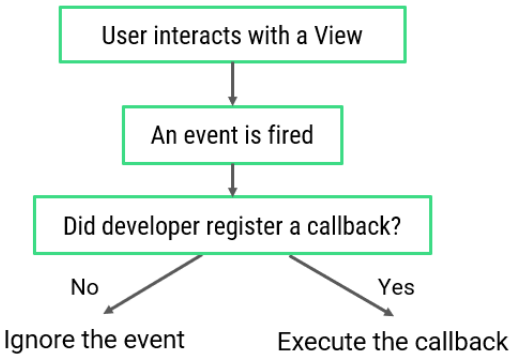
Ví dụ: khi ấn nút "TAP ME!", ứng dụng cập nhật nội dung của TextView. Đầu tiên, tìm tham chiếu đến TextView:

val resultTextView: TextView = findViewById(R.id.textView)

Thay đổi thuộc tính của TextView:

resultTextView.text = "Goodbye!"

Để bắt một sự kiện người dùng cụ thể như khi người dùng ấn nút Button, cần thiết lập đối tượng listeners để lắng nghe sự kiện. Mỗi đối tượng View có một tập các sự kiện người dùng, khi đó đối tượng listener sẽ khai báo các hàm callbacks. Các hàm này sẽ được gọi khi sự kiện xảy ra.



View.OnClickListener: bắt sự kiện OnClickListener trên một View. Trong ví dụ này ta sử dụng lớp MainActivity thực hiện giao diện OnClickListener, và viết chồng

hàm onClick() method. Sau đó thiết lập đối tượng listener để lắng nghe sự kiện trên button.

```
class MainActivity : AppCompatActivity(), View.OnClickListener {

   override fun onCreate(savedInstanceState: Bundle?) {
       ...
       val button: Button = findViewById(R.id.button)
       button.setOnClickListener(this)
   }

   override fun onClick(v: View?) {
       TODO("not implemented")
   }
}
```

Một cách khác để định nghĩa lớp hỗ trợ OnClickListener là tạo lớp thực hiện giao diện này. Vì interface View.OnClickListener chỉ có một hàm trừu tượng duy nhất, hàm onClick - Single Abstract Method (SAM), Kotlin có thể tự động chuyển đổi thành dạng thực hiện cho giao diện này:

Format: InterfaceName lambda body

```
 val runnable = Runnable { println("Hi there") }
```

tương đương với:

```
val runnable = (object: Runnable {
      override fun run() {
          println("Hi there")
      }
  })
```

Với việc ứng dụng SAM thì View.OnClickListener sẽ được khai báo như sau:

```
class MainActivity : AppCompatActivity() {

   override fun onCreate(savedInstanceState: Bundle?) {
       ...
```

```
        val button: Button = findViewById(R.id.button)
        button.setOnClickListener({ view -> /* do something*/ })
    }
}
```

Late initialization: Hãy cùng tìm hiểu thêm một khái niệm Kotlin mà chúng ta có thể tận dụng. Trong bài học trước, chúng ta đã biết rằng Kotlin rất chú trọng đến null safety. Một lựa chọn là thiết lập kiểu dữ liệu có thể chấp nhận giá trị null và đặt giá trị ban đầu thành null. Điều này có nghĩa là mỗi lần sử dụng sẽ cần phải kiểm tra null. Bạn có thể tránh phải làm điều này bằng cách sử dụng lateinit, hàm này chỉ báo cho trình biên dịch Kotlin biết rằng bạn chịu trách nhiệm khởi tạo. Nếu bạn cố gắng sử dụng trường hoặc thuộc tính lateinit trước khi khởi tạo, một ngoại lệ sẽ được đưa ra.

```
class Student(val id: String) {

    lateinit var records: HashSet<Any>

    init {
        // retrieve records given an id
    }
}
```

Ví dụ về sử dụng kỹ thuật lateinit trong Activity: trong lớp MainActivity, chúng ta không thể khởi tạo TextView result cho đến khi layout được inflate. Vì vậy, hãy đợi phương thức onCreate() lấy tham chiếu đến TextView trong layout để khởi tạo TextView result. Sau đó, bạn có thể sử dụng thuộc tính result và chắc chắn rằng nó sẽ không bị null. Nếu bạn sử dụng thuộc tính này trước khi được khởi tạo, một ngoại lệ sẽ được ném ra.

```
class MainActivity : AppCompatActivity() {

    lateinit var result: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        ...
```

```
        result = findViewById(R.id.result_text_view)
    }
}
```

4.6. Gradle: Xây dựng ứng dụng Android

Chúng ta hãy chuyển sang phần cuối cùng của chương này. Nếu bạn còn nhớ, cấu trúc cơ bản của một ứng dụng bao gồm Activity, các tài nguyên và layout, cùng các tệp Gradle. Hãy cùng thảo luận về Gradle.

Các phần của tệp build Gradle: Khai báo plugin, Định nghĩa thuộc tính Android, Xử lý các dependency, Kết nối với kho lưu trữ (repositories).

Tệp build.gradle thường bao gồm phần plugin trong đó phần build script sẽ khai báo plugin nào được áp dụng, khai báo các thư viện cần thiết bởi ứng dụng.

```
apply plugin: 'com.android.application'
apply plugin: 'kotlin-android'
apply plugin: 'kotlin-android-extensions'
```

Android configuration: trong khối này, bạn cần thiết lập các thông tin cơ bản về ứng dung như phiên bản SDK, build targets...

```
android {
   compileSdkVersion 30
   buildToolsVersion "30.0.2"

   defaultConfig {
       applicationId "com.example.sample"
       minSdkVersion 19
       targetSdkVersion 30
   }
}
```

Dependencies: mô tả các thư viện bạn cần sử dụng.

```
dependencies {
   implementation "org.jetbrains.kotlin:kotlin-stdlib-jdk7:$kotlin_version"
   implementation 'androidx.core:core-ktx:1.3.2'
   implementation 'androidx.appcompat:appcompat:1.2.0'
```

```
    implementation 'com.google.android.material:material:1.2.1'
    ...
}
```

Repositories: khai báo các kho chứa các thư viện.

```
repositories {
    google()
    mavenCentral()
}
```

4.7. Tổng kết chương

Trong chương 4, bạn đã học:

- Sử dụng Views và ViewGroups để xây dựng giao diện cho ứng dụng
- Truy cập đến các tài nguyên trong ứng dụng: R.<resource_type>.<resource_name>
- Xử lý sự kiện người dùng trong Activity (ví dụ, đăng ký OnClickListener với đối tượng giao diện)
- Sử dụng Gradle để biên dịch ứng dụng

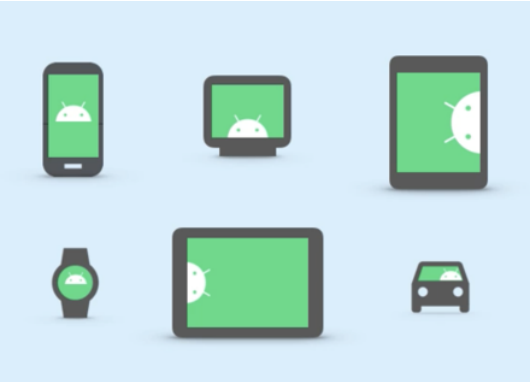
Bài thực hành: Build your first Android app.

Chương 5
Bố cục - Layout

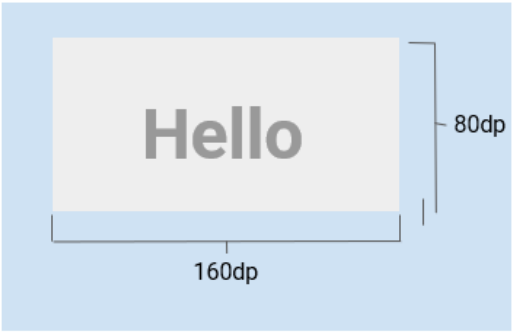
5.1. Layout trong Android 76
5.2. ConstraintLayout 78
5.3. Các chủ đề nâng cao với ConstraintLayout 84
5.4. Liên kết dữ liệu - Data binding 88
5.5. Hiển thị danh sách với RecyclerView 91
5.6. Tổng kết 94

5.1. Layout trong Android

Các thiết bị chạy Android rất đa dạng, từ điện thoại, máy tính bảng, đồng hồ, ti vi, ... và vì vậy có kích thước màn hình và độ phân giải cũng rất đa dạng. Do vậy chúng ta cần phải xem xét đến tính ổn định giao diện của ứng dụng khi chạy trên các thiết bị mục tiêu.



Density-independent pixels (dp): Không giống như pixel vật lý (px), density-independent pixels sử dụng mật độ màn hình (pixels per inch). Sử dụng dp và các layouts như ConstraintLayout (trình bày ở phần sau), sẽ làm cho giao diện hoạt động tốt trên các thiết bị khác nhau.

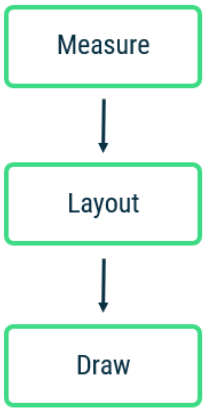


Màn hình được phân tách theo độ phân giải bao gồm: độ phân giải thấp (low density - 120 dpi), cho đến độ phân giải rất cao (extra-extra-extra-high density - 640dpi). Hầu hết các thiết bị đều có độ phân giải từ hdpi đến xxxhdpi.

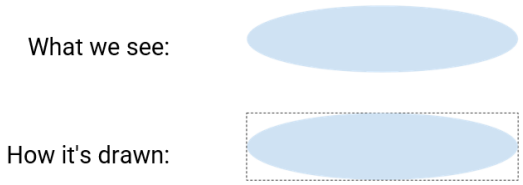
Density qualifier	Description	DPI estimate
ldpi (mostly unused)	Low density	~120dpi
mdpi (baseline density)	Medium density	~160dpi
hdpi	High density	~240dpi
xhdpi	Extra-high density	~320dpi
xxhdpi	Extra-extra-high density	~480dpi
xxxhdpi	Extra-extra-extra-high density	~640dpi

Các bước để Android hiển thị một thành phần giao diện (View) lên màn hình:

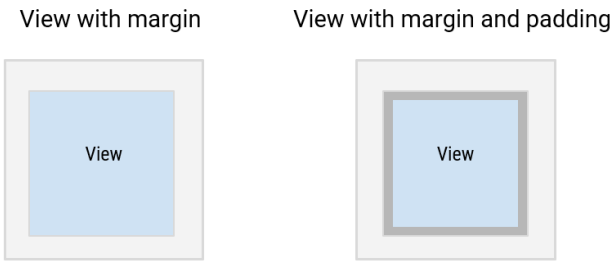
- Measure pass - xác định kích thước của View
- Layout pass - xác định vị trí của View dựa vào các luật của layout tương ứng.
- Draw - Hiển thị view dựa vào kết quả của hai bước trên.



Vùng vẽ của mọi View đều nằm trong một hình chữ nhật.



Có hai loại khoảng trống khi hiển thị View: margins và padding. Padding thêm khoảng trống bên trong hình chữ nhật, còn margin bổ sung khoảng giống giữa View và các view khác xung quanh.



5.2. ConstraintLayout

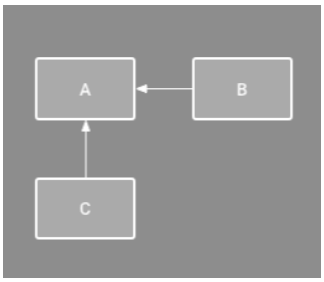
ConstraintLayout giống như các layout khác như LinearLayout, sẽ xác định vị trí của các view trên màn hình. Với các giao diện phức tạp, mặc dù với các layout khác hoàn toàn có thể áp dụng các layout lồng nhau, tuy nhiên điều này có thể dẫn đến giao diện được tạo ra sẽ bị chậm vì vị trí của các View cần phải được xác định trước khi vẽ View. Để tránh trường hợp này, ta sẽ sử dụng ConstraintLayout.

ConstraintLayout tích hợp các tính năng của cả LinearLayout và RelativeLayout, trong khi vẫn không cần sử dụng các layout lồng nhau.

Các đặc điểm của ConstraintLayout:

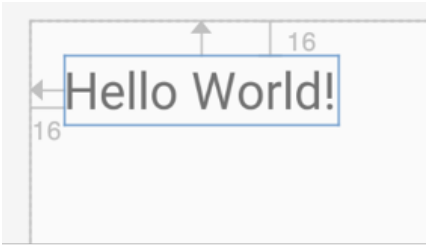
- Đây là layout ngầm định của Android (nghĩa là khi bạn sử dụng các mẫu project sẵn có thì Android sẽ sử dụng layout này).
- Giải quyết được vấn đề phức tạp của các layout lồng nhau, trong khi đó vẫn hỗ trợ các giao diện mạnh mẽ.
- Vị trí và kích thước của các view được xác định thông qua các ràng buộc (constraints).

Ràng buộc (constraints) sẽ hạn chế các thuộc tính của View mà layout sẽ cố gắng đáp ứng.



ConstraintLayout sẽ xem xét tất cả các ràng buộc của các View, sau đó sẽ xác định vị trí và kích thước của từng phần tử. Ví dụ trong hình này, B ràng buộc bên phải A và C ràng buộc phía dưới A.

Loại ràng buộc phổ biến nhất là ràng buộc tương đối về vị trí (giống như hai ràng buộc trong ví dụ trên). Hình dưới mô tả ràng buộc với parent (container của view, trong trường hợp này chính là ConstraintLayout). Ràng buộc thứ nhất: TextView liên kết với cạnh trên của màn hình, và ràng buộc thứ hai: TextView liên kết với cạnh trái của màn hình. TextView này sử dụng top và left margin đều là 16dp.




```
Cú pháp:
layout_constraint<SourceConstraint>_to<TargetConstraint>Of
Ví dụ với TextView trên:
app:layout_constraintTop_toTopOf="parent"
app:layout_constraintLeft_toLeftOf="parent"
```

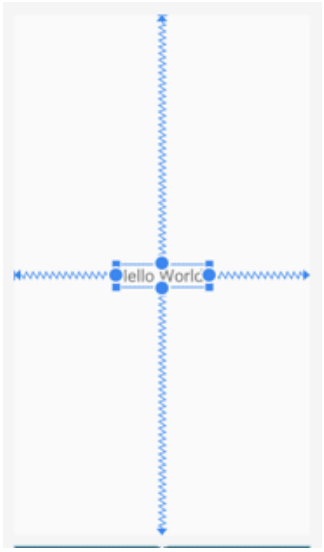
Hình sau mô tả các ràng buộc xét theo chiều dọc: top, bottom và text's baseline của các View chứa văn bản (đường kẻ chân chữ trong View).



Xét theo hàng ngang có các ràng buộc left (start) và right (end). Ta nên sử dụng start và end vì có những ngôn ngữ lại tuân thủ theo cách viết từ phải qua trái.



Ví dụ sau mô tả các ràng buộc trên màn hình thiết kế, ở đây ta có môt TextView với ràng buộc đến 4 cạnh top, bottom, start, và end.



```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

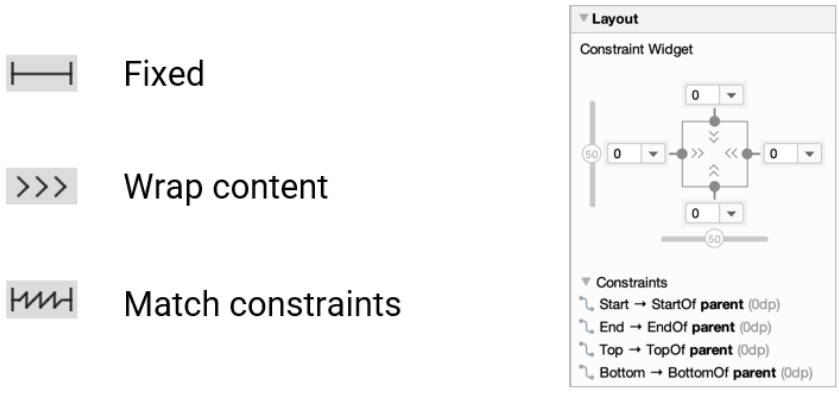
    <TextView
        ...
        app:layout_constraintBottom_toBottomOf="parent"
```

```
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

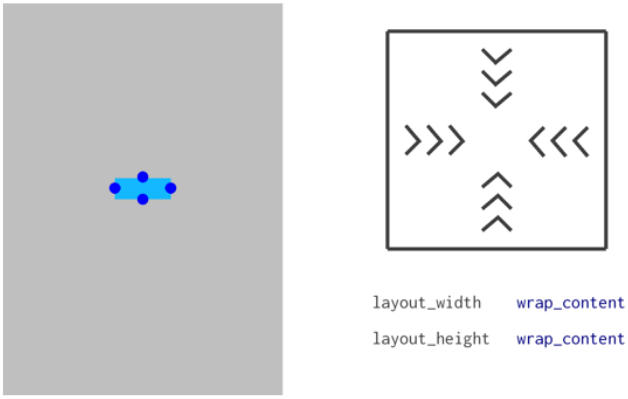
</androidx.constraintlayout.widget.ConstraintLayout>
```

Bạn có thể tạo ra các ràng buộc bằng cách soạn thảo trực tiếp trong file XML của layout hoặc sử dụng Android Studio's Layout Editor. Trong Layout Editor, tất cả các View đều hiển thị 4 vòng tròn nhỏ tương ứng với ràng buộc đến start, end, top, và bottom của view. Trong ví dụ trên, ta đã thiết lập layout_constraint Start với Start của parent, và End với End của parent. Ngầm định, hai ràng buộc này đều có vai trò như nhau với View (ta có thể thay đổi điều này bằng cách điều chỉnh thuộc tính constraint bias), do đó View sẽ nằm giữa màn hình.

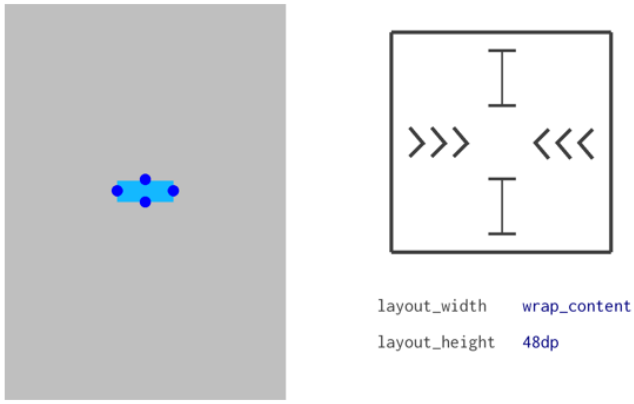
Có 3 loại ràng buộc về kích cỡ, bạn có thể điều chỉnh trên Constraint Widget trong Layout Editor (như hình dưới). Trong Layout Editor, Constraint Widget trong cửa sổ Attributes Window ở bên phải màn hình.



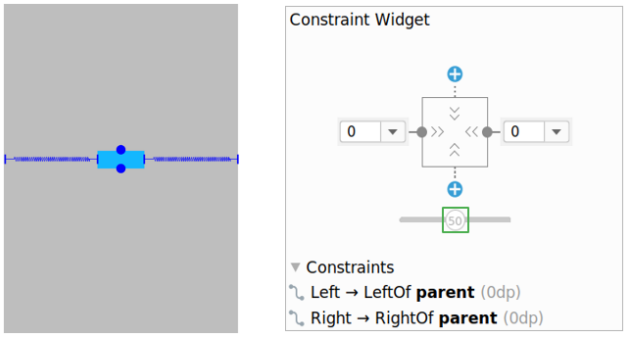
Wrap content: Hình vuông bên phải biểu diễn wrap_content cho chiều rộng và chiều cao của View.



Trong hình dưới, View có chiều cao cố định 48dp, trong đó chiều rộng vẫn là wrap_content.

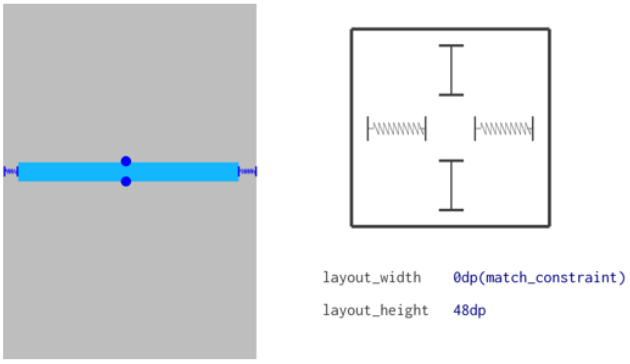


Căn giữa View theo hàng ngang: đặt ràng buộc start và end của View. Vì bias là 50 nên hai ràng buộc này ngang bằng. Vì vậy View đặt căn giữa.



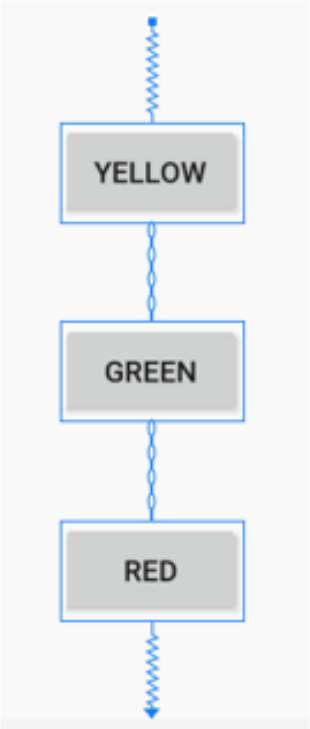
Sử dụng match_constraint: trong LinearLayout, chúng ta sử dụng match_parent để mô tả một view chiếm nhiều không gian nhất có thể. Với ConstraintLayout, chúng

ta không nên sử dụng match_parent cho view vì sẽ làm view chiếm hết không gian của các view khác, thay vào đó, chúng ta sử dụng match_constraint. Trong file XML, chế độ này sẽ được chọn bằng cách thiết lập giá trị width hoặc hight là 0dp (match_constraint).

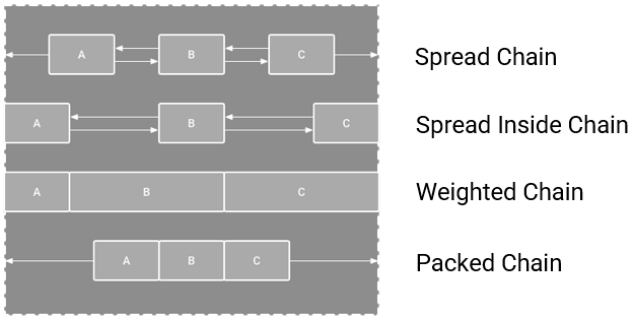


Chúng ta cũng có thể móc xích (Chains) các view với nhau: các view được liên kết với nhau bởi các ràng buộc hai chiều.

Thực hiện trong Layout Editor như sau: chọn các view cần móc xích, chọn Chains, chọn "Create a horizontal or vertical chain".



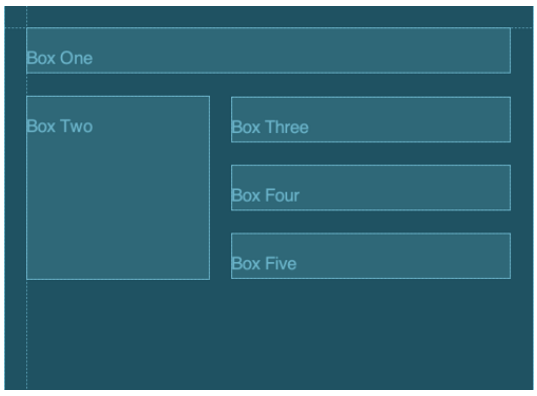
Các view có thể móc xích theo một trong 4 loại sau. Ngoài ra các khoảng trống cũng có thể thiết lập giữa các view.



5.3. Các chủ đề nâng cao với ConstraintLayout

Guideline: Bạn có thể liên kết nhiều view với một guideline (là đường thẳng ảo, dọc hoặc ngang). Guideline không hiểu thị trên thiết bị, chỉ nhằm mục đích thiết kế giao diện.

Hình sau mô tả Guideline trong Android Studio. Bạn có thể chỉnh sửa vị trí trong code XML, hoặc bằng cách chọn và kéo trên cửa sổ thiết kế. Các view đã ràng buộc với Guideline sẽ tự động điều chỉnh khi Guideline thay đổi.



Dưới đây là ví dụ TextView được ràng buộc với Guideline.

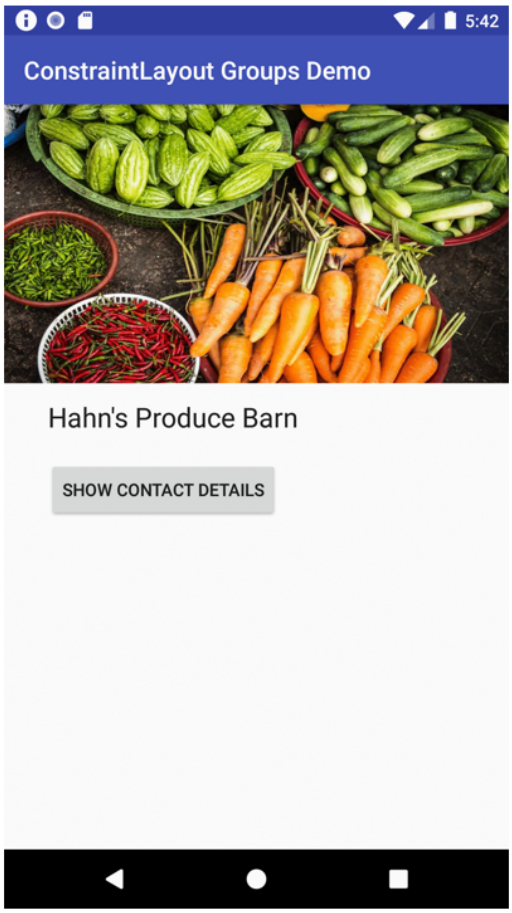
```
<ConstraintLayout>
   <androidx.constraintlayout.widget.Guideline
       android:id="@+id/start_guideline"
       android:layout_width="wrap_content"
       android:layout_height="wrap_content"
```

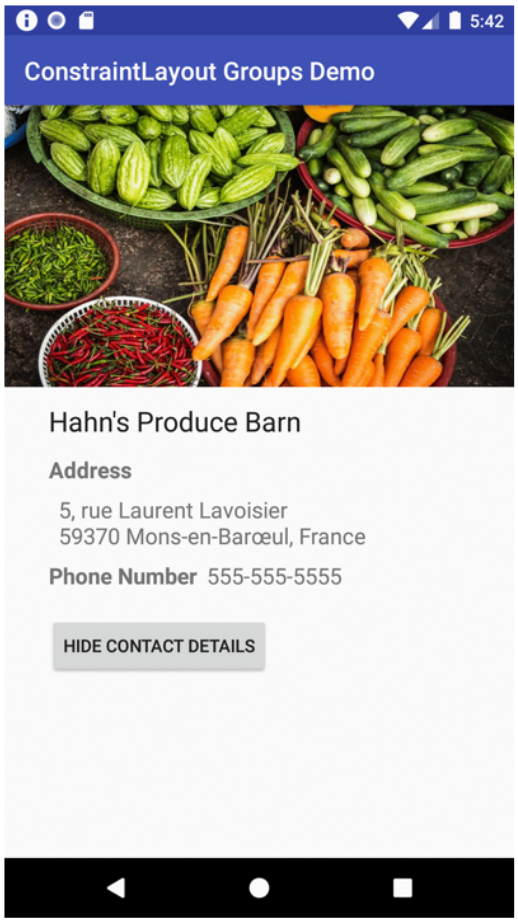
```
        android:orientation="vertical"
        app:layout_constraintGuide_begin="16dp" />
    <TextView ...
        app:layout_constraintStart_toEndOf="@id/start_guideline" />
</ConstraintLayout>
```

Vị trí của Guideline được xác định bằng đơn vị dp từ "begin" hoặc "end" của layout, hoặc bởi một số thực từ 0 đến 1 (ví dụ, 0.05).

```
layout_constraintGuide_begin
layout_constraintGuide_end
layout_constraintGuide_percent
```

Groups cho phép điều khiển hiển thị cả một nhóm widget thông qua mã nguồn.Trong hình sau, khi ấn nút "Show Contact Details" thì sẽ hiển thị cả các TextView Address và Phone Number, và ẩn cả cụm các View này khi ấn nút "Hide Contact Details".


5:42
ConstraintLayout Groups Demo
Hahn's Produce Barn
SHOW CONTACT DETAILS



Ví dụ sau khai báo Group trong file XML, các view trong Group được khai báo bởi thuộc tính constraint_referenced_ids.

```
<androidx.constraintlayout.widget.Group
    android:id="@+id/group"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:constraint_referenced_ids="locationLabel,locationDetails"/>
```

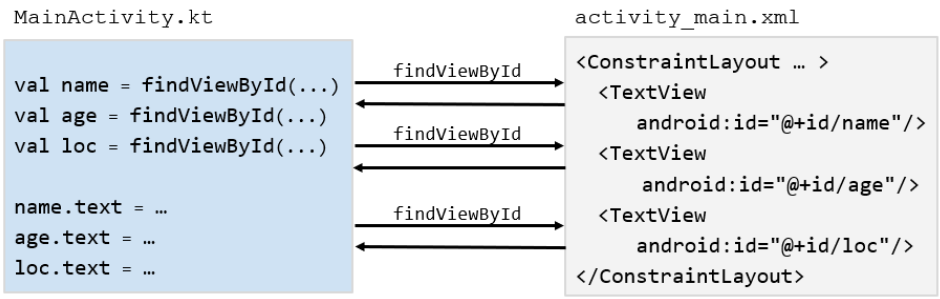
Mã nguồn để hiển thị hoặc ẩn một nhóm View trong một Group:

```
override fun onClick(v: View?) {
    if (group.visibility == View.GONE) {
        group.visibility = View.VISIBLE
```

```
        button.setText(R.string.hide_details)
    } else {
        group.visibility = View.GONE
        button.setText(R.string.show_details)
    }
}
```

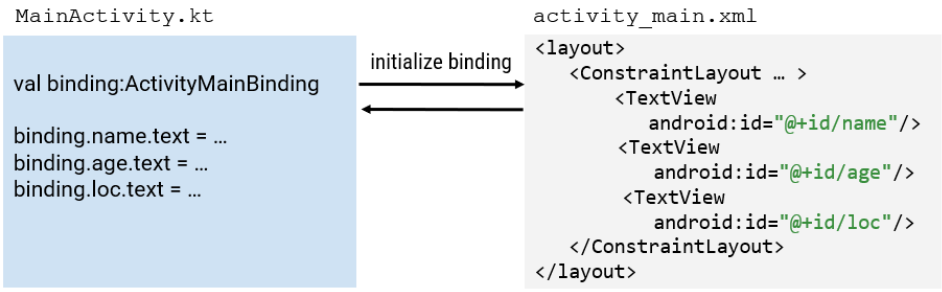
5.4. Liên kết dữ liệu - Data binding

Hiện tại trong mã nguồn của một Activity, để tìm một View thì ta sử dụng câu lệnh findViewById(). Khi đó câu lệnh này sẽ thất bại nếu ID không nằm trong Activity này.



Khi sử dụng data binding, ta sẽ đơn giản hóa mã nguồn và có thể bắt lỗi trên tại thời điểm biên dịch. Trong ví dụ này, chúng ta sẽ khởi tạo đối tượng binding và sử dụng đối tượng này để truy cập trực tiếp các View, thay vì phải sử dụng câu lệnh findViewById cho từng View.

Với đối tượng binding, ta có thể truy cập các TextView name, age, và location (ví dụ: binding.name), cập nhật text trong View (Ví dụ: binding.name.text). Điều này là khả thi vì name, age, và loc đều là các ID đã được khai báo cho các TextViews này bên trong file XML layout.



Bước 1. Sửa đổi file build.gradle để ứng dụng có thể sử dụng data binding. Khai báo này thông báo cho Gradle biết cần tạo ra một số lớp hỗ trợ cơ chế data binding.

```
android {
   ...
   buildFeatures {
       dataBinding true
   }
}
```

Tiếp theo, trong file XML layout, bổ sung thẻ layout bao bọc toàn bộ layout. Điều này nhấn mạnh rằng khi biên dịch sẽ tạo ra lớp binding chứa tham chiếu đến tất cả các View có ID được khai báo trong file layout này.

```
<layout>
    <androidx.constraintlayout.widget.ConstraintLayout>
        <TextView ... android:id="@+id/username" />
        <EditText ... android:id="@+id/password" />
    </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```

Trong mã nguồn Activity, thay vì gọi setContentView(), ta gọi DataBindingUtil.setContentView() và truyền vào Activity(this) và resource ID của layout. Biến binding trả về bởi hàm này sẽ cho phép ta truy cập đến tất cả các view. Lưu ý rằng ta không cần gọi findViewById trước khi truy cập đến các View.

```
Replace this
setContentView(R.layout.activity_main)
with this
val binding: ActivityMainBinding = DataBindingUtil.setContentView(
    this, R.layout.activity_main)

binding.username = "Melissa"
```

Ngoài ra, thay vì sửa đổi các View được binding trong code, ta có thể thiết lập trực tiếp trong layout. Ở phần đầu của file XML layout, ta thêm thẻ data để khai

biến thêm một biến (trong ví dụ này là name). Sau đó ta sử dụng biến này để hiển thị trong TextView.

Trong mã nguồn Activity, ta thiết lập nội dung của TextView bằng cách thiết lập giá trị của biến name này (trong đối tượng binding).

```
<layout>
   <data>
       <variable name="name" type="String"/>
   </data>
   <androidx.constraintlayout.widget.ConstraintLayout>
       <TextView
           android:id="@+id/textView"
           android:text="@{name}" />
   </androidx.constraintlayout.widget.ConstraintLayout>
</layout>

In MainActivity.kt:
binding.name = "John"
```

Cơ chế này cũng hỗ trợ các câu lệnh phức tạp hơn như các biến đổi đơn giản, hiện thị nội dung dựa trên phép so sánh hay truy cập đến nội dung của các View khác. Trong ví dụ này, ta gọi toUpperCase() trên biến name.

```
<layout>
   <data>
       <variable name="name" type="String"/>
   </data>

   <androidx.constraintlayout.widget.ConstraintLayout>
       <TextView
           android:id="@+id/textView"
           android:text="@{name.toUpperCase()}" />
   </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```

5.5. Hiển thị danh sách với RecyclerView

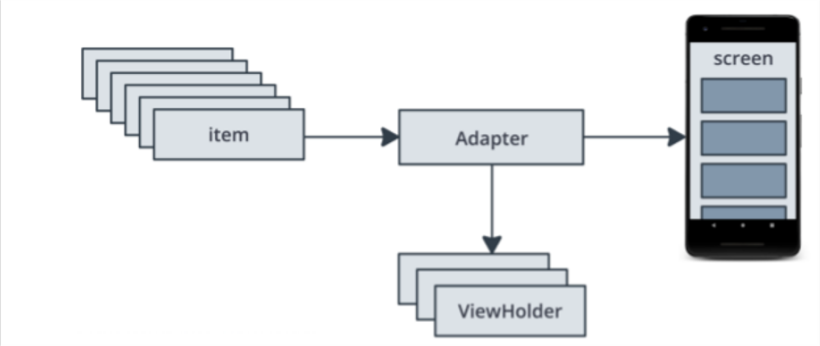
RecyclerView là:

- Widget để hiển thị danh sách dữ liệu.
- Tái sử dụng lại các view để làm thao tác "cuộn" các mục được hiệu quả hơn.
- Có thể chọn layout cho từng mục.



Các thành phần của RecyclerView's:

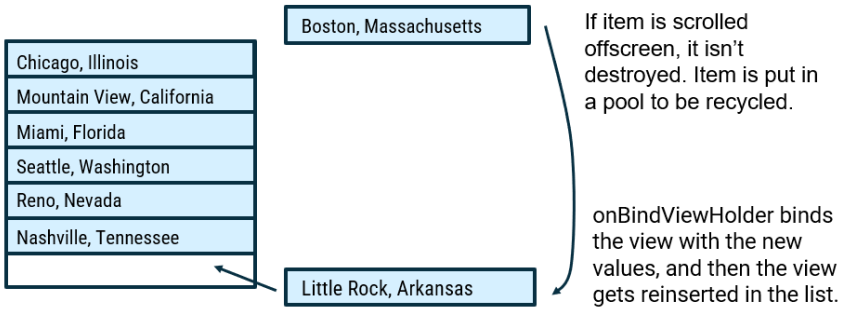
- item - Một mục dữ liệu để hiển thị. Tương ứng với một đối tượng dữ liệu mà ta đang quản lý.
- Adapter - Lấy dữ liệu, chuẩn bị view để hiển thị lên RecyclerView.
- ViewHolders - Một tập các View để RecyclerView tái sử dụng lại.
- RecyclerView - Views trên màn hình.



RecyclerView.Adapter là lớp chịu trách nhiệm chinh, cung cấp dữ liệu và layout để hiển thì trên RecyclerView. Ta cần viết một lớp override từ lớp này và bổ sung các hàm sau:

- getItemCount : trả về số mục dữ liệu
- onCreateViewHolder : hàm này được gọi khi RecyclerView cần tạo ra một dòng mới.
- onBindViewHolder : hàm này được gọi khi tái sử dụng lại view với dữ liệu của đối tượng mới.

Tại sao lại cần phải tái sử dụng lại view trong RecyclerView. Vì khi ta có một số lượng rất lớn các đối tượng dữ liệu (lớn hơn nhiều so với khả năng hiển thị trên màn hình), khi đó ta không thể tạo ra tất cả các View cho các đối tượng này. Hàm onCreateViewHolder() được gọi để tạo ra view holders cho các mục có thể hiện thị đồng thời trên màn hình. Sau đó khi người dùng cuộn màn hình, hệ thống sẽ loại bỏ các view đó và gọi onBindViewHolder() để tái sử dụng lại các View khác với việc cập nhật các đối tượng mới tương ứng hiển thị trên màn hình.



Để thực hiện kỹ thuật này, đầu tiên thêm phần tử RecyclerView vào XML layout.

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/rv"
    android:scrollbars="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

Tiếp theo, tạo layout để hiển thị từng mục trên RecyclerView. Trong trường hợp này chỉ sử dụng FrameLayout chứa một TextView.

```
res/layout/item_view.xml
<FrameLayout
   android:layout_width="match_parent"
   android:layout_height="wrap_content">
   <TextView
       android:id="@+id/number"
       android:layout_width="match_parent"
       android:layout_height="wrap_content" />
</FrameLayout>
```

Sau đó tạo list adapter. Lớp này chịu trách nhiệm tạo từng mục cho RecyclerView.

- Tạo lớp Adapter mới (MyAdapter) thừa kế (extends) từ lớp RecyclerView.Adapter.
- Định nghĩa lớp ViewHolder để chứa các view trong layout của một mục.
- Override 3 hàm từ RecyclerView.Adapter class: onCreateViewHolder, onBindViewHolder, getItemCount.

```
class MyAdapter(val data: List<Int>) :
    RecyclerView.Adapter<MyAdapter.MyViewHolder>() {

   class MyViewHolder(val row: View) :    RecyclerView.ViewHolder(row) {
       val textView = row.findViewById<TextView>(R.id.number)
   }
```

```
        override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
             MyViewHolder {
        val layout = LayoutInflater.from(parent.context).
        inflate(R.layout.item_view,parent, false)
        return MyViewHolder(layout)
    }
    override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
        holder.textView.text = data.get(position).toString()
    }
    override fun getItemCount(): Int = data.size
```

Cuối cùng, thiết lập adapter với RecyclerView trong file mã nguồn của MainActivity.

- Tìm tham chiếu đến RecyclerView trong layout.
- Thiết lập Layout Manager cho RecyclerView. LinearLayoutManager hoặc GridLayoutManager thường được sử dụng.
- Khởi tạo đối tượng adapter của lớp adapter ở trên, thiết lập adapter này làm adapter của RecyclerView.

```
In MainActivity.kt:

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val rv: RecyclerView = findViewById(R.id.rv)
    rv.layoutManager = LinearLayoutManager(this)

    rv.adapter = MyAdapter(IntRange(0, 100).toList())
}
```

5.6. Tổng kết

Trong chương 5, bạn đã học:

- Xác định kích thước của các View theo đơn vị dp.
- Làm việc với các mật độ màn hình khác nhau của các thiết bị Android khác nhau.
- Tích hợp layout vào Activity.
- Thêm các view vào ConstraintLayout sử dụng các ràng buộc (constraints).
- Đơn giản hóa việc truy cập đến các thành phần của Activity thông qua cơ chế data binding.
- Hiển thị một danh sách các đối tượng text sử dụng RecyclerView và lớp adapter tùy biến.

Chương 6
Điều hướng các màn hình trong Android

6.1. Ứng dụng với nhiều activities

Trong các bài học trước chúng ta mới phát triển các ứng dụng Android chỉ có một màn hình (một Activity). Khi bạn thêm nhiều chức năng vào ứng dụng, khi đó sẽ cần có thêm các màn hình khác, ví dụ:

- Thông tin chi tiết của một mục như thông tin chi tiết của một sản phẩm trong các ứng dụng bán hàng.
- Tạo ra mục mới, ví dụ như tạo email mới.
- Hiển thị các setting của ứng dụng.
- Truy nhập đến các services của các ứng dụng khác, ví dụ như photo gallery hay tìm kiếm tài liệu.

Kỹ thuật thông thường đó là thêm các Activity mới bằng cách sử dụng Intent. Android sử dụng cấu trúc Intent cho phép yêu cầu một hành động từ một component khác. Ví dụ như Intent có thể sử dụng để yêu cầu ứng dụng mở một Activity khác. Intent cũng có thể chứa dữ liệu cần gửi đến Activity đích như thông tin chi tiết của mục dữ liệu cần hiển thị. Dữ liệu cũng có thể gửi trả ngược trở về Activity ban đầu.

Một Intent thường có hai thông tin sau:

- Hành động cần thực hiện (ví dụ, ACTION_VIEW, ACTION_EDIT)
- Dữ liệu cần thao tác (ví dụ bản ghi dữ liệu của một người trong danh bạ điện thoại).

Có hai loại Intent: explicit và implicit Intents. Với explicit intent thì cần biết tên lớp của Activity. Ta cũng có thể sử dụng để tận dụng các tính năng của app bên thứ ba.

Dưới đây là 2 ví dụ explicit Intent. Trong ví dụ đầu tiên, Activity đích nằm trong ứng dụng, còn với ví dụ thứ hai thì sẽ di chuyển tới ứng dụng khác, khi đó ta cần kiểm tra xem có ứng dụng tương ứng đó hay không.

```
Di chuyển giữa các acvitity trong cùng ứng dụng:
fun viewNoteDetail() {
   val intent = Intent(this, NoteDetailActivity::class.java)
   intent.putExtra(NOTE_ID, note.id)
   startActivity(intent)
}

Di chuyển tới ứng dụng bên ngoài:
fun openExternalApp() {
   val intent = Intent("com.example.workapp.FILE_OPEN")
   if (intent.resolveActivity(packageManager) != null) {
       startActivity(intent)
   }
}
```

Với implicit intent thì lại không cần xác định chính xác Activity đích, thay vào đó ta chỉ cần cung cấp đủ thông tin cho hệ thống. Với các Activity của các ứng dụng khác, đây là dạng khuyến khích sử dụng. Nếu có nhiều ứng dụng bên ngoài thỏa mãn điều kiện thì hệ thống sẽ cho phép người dùng chọn lựa ứng dụng cần chạy.

Như vậy, implicit intent:

- Cung cấp hành động chung mà ứng dụng có thể thực hiện.
- Đã giải quyết bằng cách ánh xạ kiểu dữ liệu và hành động vào các thành phần đã biết.
- Cho phép bất kỳ ứng dụng nào đáp ứng tiêu chí xử lý yêu cầu.

Dưới đây là ví dụ sử dụng implicit intent để gửi email. Ta không cần xác định cụ thể ứng dụng nào sẽ xử lý email.

```
fun sendEmail() {
    val intent = Intent(Intent.ACTION_SEND)
    intent.type = "text/plain"
    intent.putExtra(Intent.EXTRA_EMAIL, emailAddresses)
    intent.putExtra(Intent.EXTRA_TEXT, "How are you?")

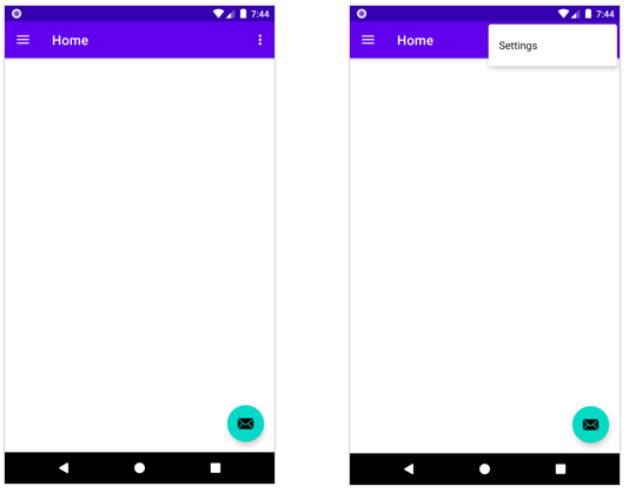
    if (intent.resolveActivity(packageManager) != null) {
```

```
        startActivity(intent)
    }
}
```

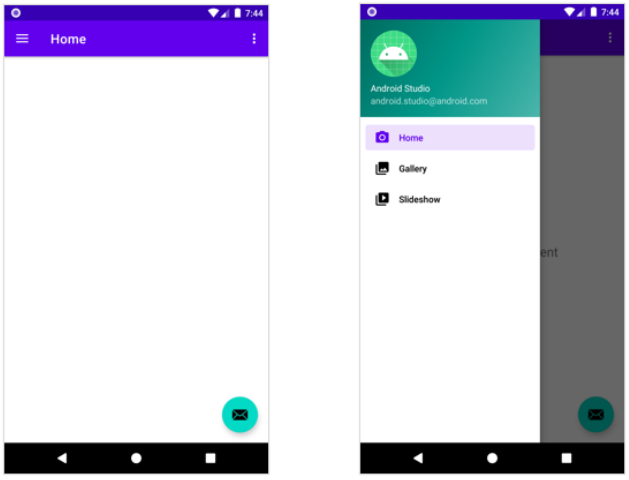
6.2. App bar, navigation drawer, và menus

Trong phần này ta sẽ tìm hiểu các thành phần UI thông thường sử dụng để di chuyển trong ứng dụng.

Phía trên màn hình là App Bar, hiển thị tên của Activity hoặc tên ứng dụng. Ngoài ra App Bar cũng cung cấp khả năng truy cập tới các thành phần khác trong ứng dụng bằng menu. Một mục menu đặc biệt là Overflow Menu (hiển thị bằng 3 dấu chấm ở phía trên bên phải của App Bar), khi người dùng ấn vào thì sẽ hiển thị menu bổ sung.



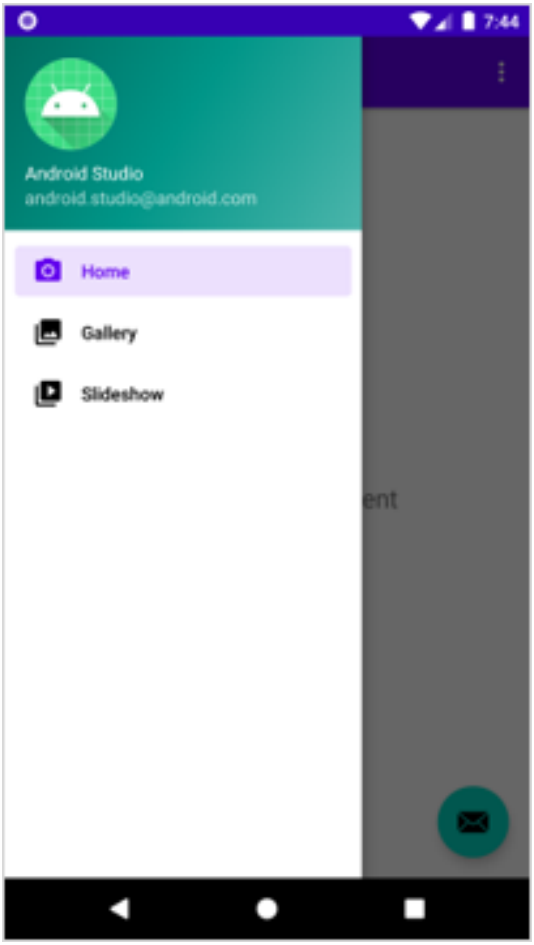
Navigation drawer là giao diện thường gặp. Để mở menu loại này bằng cách ấn vào nút ba gạch ngang (phía trên bên trái App Bar) hoặc là vuốt ở cạnh trái màn hình. Option menu và navigation drawer cũng sử dụng kỹ thuật menu.



Có một số loại menu: options menu, context menu, và popup menu. Menu được khai báo trong file định dạng XML ở trong thư mục res/menu với thẻ menu. Mỗi mục menu được khai báo với thẻ item bao gồm các thuộc tính: title, id, và một số thuộc tính tùy chọn khác như thứ tự xuất hiện hay icon. Dưới đây là ví dụ:

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto">
   <item
       android:id="@+id/action_settings"
       android:orderInCategory="100"
       android:title="@string/action_settings"
       app:showAsAction="never" />
</menu>
```

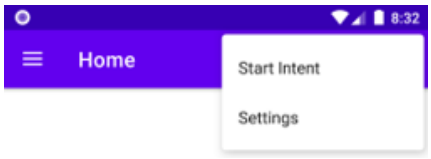
Dưới đây là file XML cho một navigation drawer, mỗi menu ngoài id còn có hai thuộc tính icon và title.




```
<menu>
   <group android:checkableBehavior="single">
       <item
           android:id="@+id/nav_home"
           android:icon="@drawable/ic_menu_camera"
           android:title="@string/menu_home" />
       <item
           android:id="@+id/nav_gallery"
           android:icon="@drawable/ic_menu_gallery"
           android:title="@string/menu_gallery" />
       <item
           android:id="@+id/nav_slideshow"
           android:icon="@drawable/ic_menu_slideshow"
           android:title="@string/menu_slideshow" />
```

```
    </group>
</menu>
```

Và dưới đây là ví dụ về option menu: thêm một menu mới với id và title.




```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <item android:id="@+id/action_intent"
        android:title="@string/action_intent" />

    <item
        android:id="@+id/action_settings"
        android:orderInCategory="100"
        android:title="@string/action_settings"
        app:showAsAction="never" />
</menu>
```

Để hiển thị được menu, ta cần viết chồng hàm onCreateOptionsMenu() trong Activity với thông tin file XML menu vừa tạo ra ở trên.

```
override fun onCreateOptionsMenu(menu: Menu): Boolean {
    menuInflater.inflate(R.menu.main, menu)
    return true
}
```

Để bắt sự kiện người dùng chọn một mục menu, viết chồng hàm onOptionsItemSelected() trong Activity. Như trong ví dụ dưới đây, khi chọn menu "action_intent" thì ứng dụng sẽ chuyển đến ứng dụng Web tìm kiếm pizza, với các menu khác thì sẽ hiển thị message.

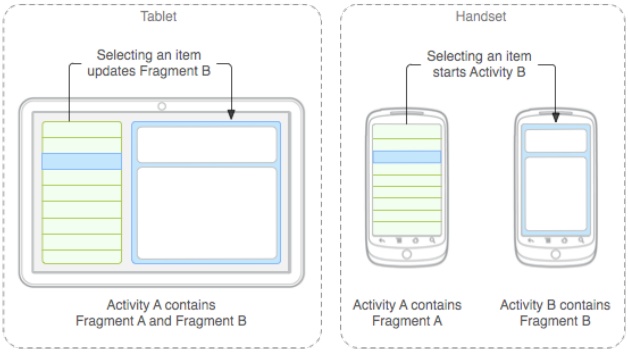
```
override fun onOptionsItemSelected(item: MenuItem): Boolean {

    when (item.itemId) {
        R.id.action_intent -> {
            val intent = Intent(Intent.ACTION_WEB_SEARCH)
            intent.putExtra(SearchManager.QUERY, "pizza")
            if (intent.resolveActivity(packageManager) != null) {
                startActivity(intent)
            }
        }
        else -> Toast.makeText(this, item.title, Toast.LENGTH_LONG).show()
    ...
```

6.3. Fragment

Để di chuyển đến phần giao diện khác của ứng dụng chúng ta có thể sử dụng Activity hoặc Fragment.

Fragment sẽ hiệu quả khi ứng dụng cần hỗ trợ chạy trên cả điện thoại và máy tính bảng. Vì máy tính bảng thường có kích thước màn hình lớn hơn điện thoại nên có thể hiển thị được nhiều thông tin hơn, vì vậy trên điện thoại có thể cần 2 Activity để hiển thị hai thông tin (ví dụ một thông tin là danh sách các mục và một thông tin là chi tiết của một mục) nhưng trên máy tính bảng thì chỉ cần một Activity là có thể hiển thị cả hai thông tin này. Để có thể tận dụng lại thiết kế cho từng thông tin thì nên sử dụng Fragment.



Về mặt chính thức, một fragment biểu diễn giao diện và hành vi của "một phần" giao diện trong một Activity. Ta có thể hiển thị nhiều frament trên một Activity,

và có thể sử dụng lại Fragment ở nhiều Activity khác nhau. Mỗi Fragment cũng có vòng đời (lifecycle) riêng, nhận sự kiện người dùng riêng và ta cũng có thể thêm hoặc loại bỏ Fragment khỏi Activity trong quá trình chạy ứng dụng.

Các đặc điểm cơ bản của Fragment:

- Phải nằm trong một Activity. Must be hosted in an activity.
- Lifecycle của Fragment được gắn với lifecycle của Activity chủ.
- Có thể thêm vào hoặc loại bỏ khi đang chạy ứng dụng.

Lưu ý: cần sử dụng lớp Fragment trong gói androidx. (androidx.fragment.app.Fragment). Không sử dụng lớp Fragment trong gói android (android.app.Fragment).

6.4. Điều hướng (Navigation) trong ứng dụng Android

Khi ta có nhiều Fragment trong ứng dụng, ta cần biết phương pháp di chuyển giữa các Fragment này. Điều này được quản lý bởi Navigation component.

Navigation component là một tập hợp các thư viện, công cụ tích hợp trong IDE cho phép tạo ra navigation paths của ứng dụng. Kiến trúc này hoạt động tốt với cơ chế "một Activity, nhiều Fragment". Gồm có 3 thành phần cơ bản, với cơ chế giống như khi chúng ta xem các chương trình truyền hình trên ti vi.

- Navigation Graph: bao gồm một tập các Fragment và Activity trong ứng dụng. Điều này tương tự danh mục các chương trình truyền hình.
- Navigation Host: là container hiển thị các Fragment được mô tả trong navigation graph, tương tự như ti vi.
- Navigation Controller: là phương tiện để di chuyển giữa các Fragment, tương tự như thiết bị điều khiển từ xa của ti vi.

Để làm việc với Navigation component Đầu tiên cần bổ sung dependencies trong file build.gradle. Hãy tìm phiên bản Navigation library mới nhất và cập nhật vào nav_version.

```
In build.gradle, under dependencies:
implementation "androidx.navigation:navigation-fragment-ktx:$nav_version"
implementation "androidx.navigation:navigation-ui-ktx:$nav_version"
```

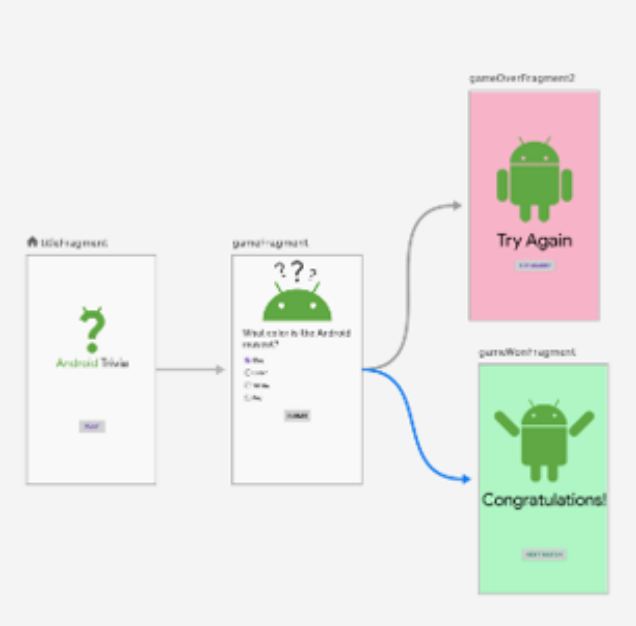
Navigation host (NavHost) là nơi chứa các Fragment. Một navigation host cần thực hiện NavHost interface. Nên sử dụng NavHostFragment là lớp đã hỗ trợ di chuyển giữa các Fragment.

Sử dụng thẻ fragment để khai báo NavHostFragment trong file XML layout của Activity. Một số thuộc tính quan trọng:

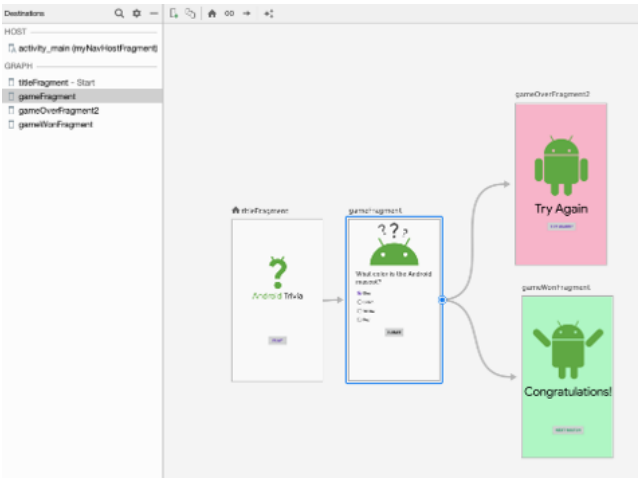
- android:name là tên đầy đủ của NavHostFragment.
- app:defaultNavHost được thiết lập "true" để đảm bảo đồng bộ với nút Back của hệ thống.
- app:navGraph trỏ đến file navigation graph.

```
<fragment
    android:id="@+id/nav_host"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:defaultNavHost="true"
    app:navGraph="@navigation/nav_graph_name"/>
```

Navigation graph là file ở định dạng XML chứa tất cả các Fragment cũng như các hành động di chuyển giữa chúng. Ví dụ sau mô tả giao diện thiết kế một Navigation graph cụ thể.



Để thiết kế Navigation Graph, ta sử dụng Navigation Editor trong Android Studio. Ta có thể kéo thả liên kết từ Fragment này sang Fragment kia.



Để tạo ra một fragment thì lớp này cần thừa kế từ lớp Fragment. Để cung cấp giao diện cho Fragment thì ta cần viết chồng hàm onCreateView() như sau:

```
class DetailFragment : Fragment() {

    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?,
            savedInstanceState: Bundle?): View? {
        return inflater.inflate(R.layout.detail_fragment, container, false)
    }
}
```

Để xác định Fragment đích thì ta có thể sử dụng Navigation Editor bằng cách kéo từ fragment nguồn và thả đến fragment đích. Khi đó action này sẽ được cung cấp tự động một ID có dạng action_<sourceFragment>_to_<destinationFragment>. Ta cũng có thể đặt tên lại ID này.

Đoạn code sau mô tả ví dụ khai báo fragment trong file Navigation Graph.

```
<fragment
    android:id="@+id/welcomeFragment"
    android:name="com.example.android.navigation.WelcomeFragment"
    android:label="fragment_welcome"
    tools:layout="@layout/fragment_welcome" >
```

```
    <action
        android:id="@+id/action_welcomeFragment_to_detailFragment"
        app:destination="@id/detailFragment" />

</fragment>
```

Để có thể di chuyển đến Fragment mới, ta sử dụng Navigation Controller (NavController). Trong Activity, NavController được ràng buộc chặt chẽ với navigation host. Phương thức navigate được gọi để di chuyển đến Fragment mới.

```
class MainActivity : AppCompatActivity() {

   override fun onCreate(savedInstanceState: Bundle?) {
       ...
       val navController = findNavController(R.id.myNavHostFragment)
   }
   fun navigateToDetail() {
       navController.navigate(R.id.action_welcomeFragment_to_detailFragment)
   }
}
```

6.5. Truyền dữ liệu

Dữ liệu có thể được Fragment nguồn gửi tới Fragment đích. Trong Navigation component có Safe Args plugin cho phép tạo đối tượng dữ liệu để truyền. Các đặc điểm của Safe Args:

- Đảm bảo các tham số có kiểu dữ liệu chính xác.
- Cung cấp giá trị ngầm định cho tham số.
- Tạo lớp <SourceDestination>Directions với phương thức di chuyển cho mỗi action
- Tạo lớp để thiết lập các tham số cho tất cả các action
- Tạo lớp <TargetDestination>Args để cho phép truy nhập đến các tham số của Fragment đích.

Để cài đặt thư viện Safe Args thì cần bổ sung vào hai file Gradle. Đầu tiên với file build.grade của project thì thêm classpath dependency cho Safe Args plugin.

```
buildscript {
   repositories {
       google()
   }
   dependencies {
       classpath "androidx.navigation:navigation-safe-args-gradle-plugin:$nav_version"
   }
}
```

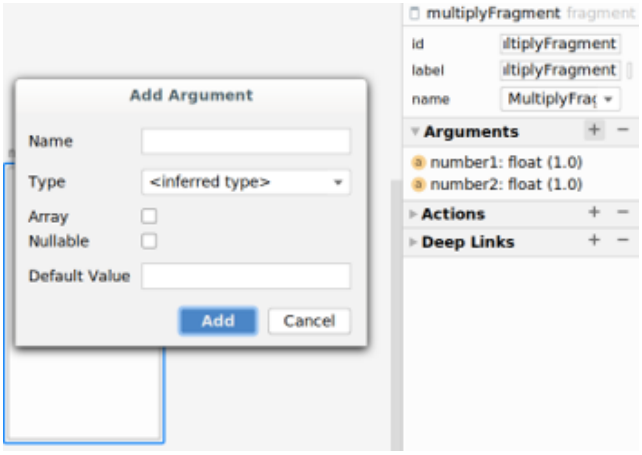
Tiếp theo trong file build.gradle của ứng dụng thì bổ sung plugin này.

apply plugin: "androidx.navigation.safeargs.kotlin"

Các bước truyền tham số từ Fragment nguồn đến Fragment đích:

- Khai báo tham số mà Fragment đích cần nhận.
- Tạo action để liên kết từ Fragment nguồn đến Fragment đích.
- Thiết lập tham số cần gửi đi trong phương thức action của lớp <Source>FragmentDirections.
- Di chuyển đến Fragment mới với action trên sử dụng Navigation Controller.
- Tìm lại tham số trong Fragment đích.

Để định nghĩa các tham số trong Fragment đích, ta có thể sử dụng Navigation Editor. Khi đó đoạn code XML trong file Navigation Graph sẽ được bổ sung các tham số tương ứng. Một tham số sẽ cần ba thuộc tính: name (tên tham số), argType (kiểu dữ liệu của tham số) và defaultValue (giá trị ngầm định của tham số). Trong ví dụ này ta thiết lập hai tham số number1 và number2 đều thuộc kiểu float và có giá trị ngầm định là 1.0.

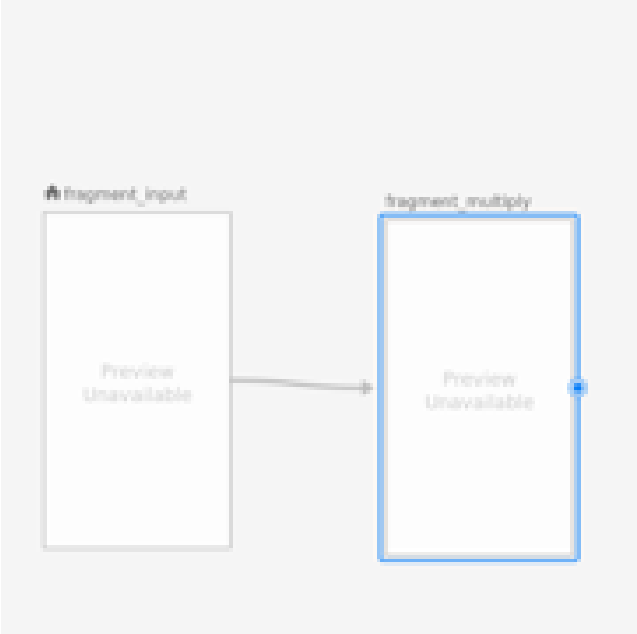



```
<fragment
    android:id="@+id/multiplyFragment"
    android:name="com.example.arithmetic.MultiplyFragment"
    android:label="MultiplyFragment" >
    <argument
        android:name="number1"
        app:argType="float"
        android:defaultValue="1.0" />
    <argument
        android:name="number2"
        app:argType="float"
        android:defaultValue="1.0" />
 </fragment>
```

Bảng sau mô tả các kiểu dữ liệu cho các tham số được hỗ trợ trong Android bao gồm các kiểu dữ liệu cơ bản như số, kiểu logic boolean, kiểu xâu ký tự string và các kiểu dữ liệu phức tạp hơn như kiểu mảng dữ liệu hay kiểu enum.

Type	Type Syntax app:argType=<type>	Supports Default Values	Supports Null Values
Integer	"integer"	Yes	No
Float	"float"	Yes	No
Long	"long"	Yes	No
Boolean	"boolean"	Yes ("true" or "false")	No
String	"string"	Yes	Yes
Array	above type + "[]" (for example, "string[]" "long[]")	Yes (only "@null")	Yes
Enum	Fully qualified name of the enum	Yes	No
Resource reference	"reference"	Yes	No

Việc tạo ra action để liên kết từ Fragment nguồn đến Fragment đích đã được mô tả ở trên. Dưới đây là một ví dụ cụ thể với action từ InputFragment tới MultiplyFragment.



File nav_graph.xml:

```
<fragment
    android:id="@+id/fragment_input"
    android:name="com.example.arithmetic.InputFragment">

    <action
        android:id="@+id/action_to_multiplyFragment"
```

```
        app:destination="@id/multiplyFragment" />

</fragment>
```

Để di chuyển sang Fragment mới, ta gọi hàm action trên lớp Direction (lớp này được tự động tạo ra, ví dụ InputFragmentDirections). Vì Fragment đích (MultiplyFragment) yêu cầu 2 tham số, vì vậy ta cũng cần truyền hai tham số vào hàm action. Sau đó gọi navigate() trên đối tượng NavController.

Trong ví dụ này, ta đang sử dụng cơ chế data binding trong fragment.

```
File InputFragment.kt:
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
   super.onViewCreated(view, savedInstanceState)
   binding.button.setOnClickListener {
       val n1 = binding.number1.text.toString().toFloatOrNull() ?: 0.0
       val n2 = binding.number2.text.toString().toFloatOrNull() ?: 0.0

       val action = InputFragmentDirections.actionToMultiplyFragment(n1, n2)
       view.findNavController().navigate(action)
   }
}
```

Cuối cùng ta cần tìm lại các tham số trong Fragment đích. MultiplyFragmentArgs cũng là lớp được tạo ra tự động để nhận các tham số này. navArgs đến từ androidx.navigation.fragment.navArgs. Sau khi giao diện Fragment được tạo ra thì ta đã có thể truy cập và sử dụng các tham số này. Trong ví dụ này, ta cập nhật TextView bằng tích của hai số nhận được.

```
class MultiplyFragment : Fragment() {
   val args: MultiplyFragmentArgs by navArgs()
   lateinit var binding: FragmentMultiplyBinding
   override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
       super.onViewCreated(view, savedInstanceState)
       val number1 = args.number1
       val number2 = args.number2
       val result = number1 * number2
```

```
        binding.output.text = "${number1} * ${number2} = ${result}"
    }
}
```

6.6. Back Stack

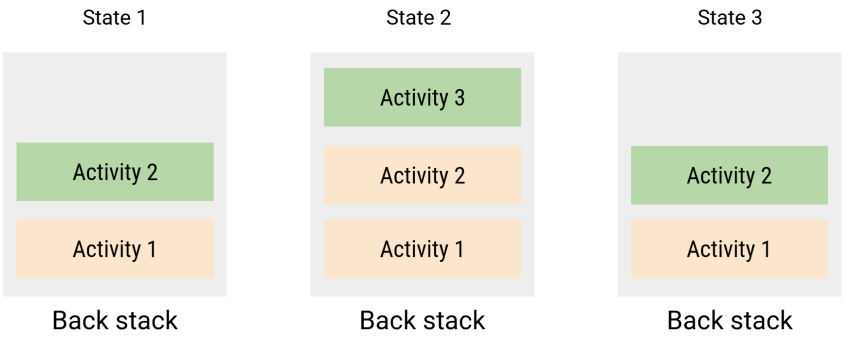
Hãy cùng tìm hiểu về một khái niệm khác mà bạn sẽ gặp phải khi xử lý điều hướng ứng dụng: ngăn xếp back stack. Trong ứng dụng của bạn, Android theo dõi các hoạt động bạn đã mở dưới dạng một tập hợp các hoạt động trong một ngăn xếp, được gọi là ngăn xếp back stack.

Khi ứng dụng của bạn chạy lần đầu, Activity đầu tiên sẽ được thêm vào ngăn xếp backstack. Khi bạn duyệt qua các phần trong ứng dụng, Android có phương pháp "vào sau, ra trước" ("last in, first out") để theo dõi các Activity đã truy cập. Android sẽ thêm Activity được truy cập gần đây nhất lên đầu ngăn xếp.

Trạng thái 1: Một Activity mới (Activity 2) được bắt đầu và được thêm vào đầu ngăn xếp. Activity 2 hiện đã lấy focus và người dùng có thể tương tác.

Trạng thái 2: Khi bạn bắt đầu một Activity khác (Activity 3), Activity này sẽ được thêm vào đầu ngăn xếp. Activity 2 bị dừng và Activity 3 bây giờ có focus.

Trạng thái 3: Khi bạn nhấn nút Back, Android sẽ đẩy Activity 3 ra khỏi ngăn xếp và kết thúc. Activity 2 hiện đã ở trên cùng của ngăn xếp và được tiếp tục.



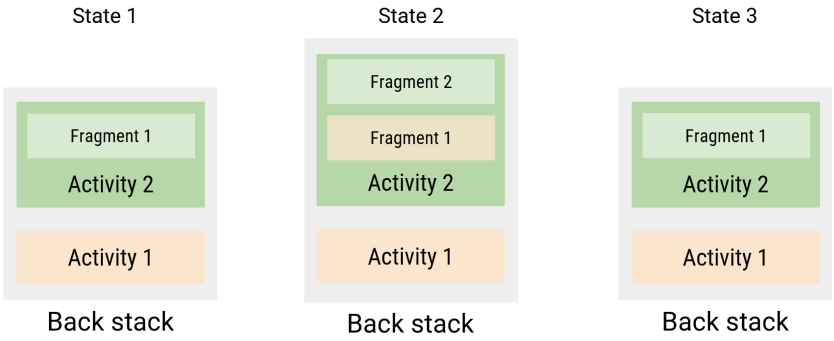
Hơn nữa, khi sử dụng fragment với thành phần Navigation, cũng có một back stack các fragment được duy trì khi chúng ta duyệt qua các fragment khác nhau trong ứng dụng. Trong ví dụ này:

Trạng thái 1: Chúng ta bắt đầu với Activity 2 hiển thị Fragment 1 (Activity 1 hiện đang dừng và ở chế độ nền - background).

Trạng thái 2: Sau đó, trong Activity 2, chúng ta điều hướng đến một Fragment khác (Fragment 2), Fragment này sẽ được thêm vào đầu ngăn xếp trong Activity của nó.

Trạng thái 3: Khi bạn chạm vào nút Back, Fragment 2 sẽ được đẩy ra khỏi Back Stack và Fragment 1 sẽ trở thành Fragment đang hoạt động một lần nữa.

Lưu ý rằng mặc dù Fragment có vòng đời riêng trong Activity chủ, nhưng vòng đời của chúng lại gắn liền trực tiếp với vòng đời của Activity.



6.7. Tổng kết

Trong chương 6, bạn đã tìm hiểu :

- Sử dụng explicit và implicit intent để điều hướng giữa các Activity.
- Cấu trúc lại giao diện ứng dụng sử dụng fragment.
- Điều hướng các Fragment với NavGraph, NavHost, và NavController.
- Sử dụng Safe Args để truyền dữ liệu giữa các Fragment.
- Android lưu trữ một ngăn xếp Back Stack chứa tất cả các đích đến mà bạn đã truy cập, với mỗi đích đến mới sẽ được đẩy vào ngăn xếp.

Chương 7
Vòng đời hoạt động của Activity và Fragment

7.1. Vòng đời Activity 113
7.2. Ghi log (Logging) 117
7.3. Vòng đời Fragment 117
7.4. Các thành phần nhận biết vòng đời (Lifecycle-aware components) 119
7.5. Nhiệm vụ và ngăn xếp (Tasks & Back stack) 120
7.6. Tóm tắt 123

7.1. Vòng đời Activity

Việc hiểu rõ các trạng thái trong vòng đời Activity là rất quan trọng để bạn có thể triển khai hành vi ứng dụng phù hợp với kỳ vọng của người dùng. Ví dụ, chúng ta cần lưu giữ dữ liệu và trạng thái của người dùng khi họ tạm thời rời ứng dụng và quay lại, khi có cuộc gọi điện thoại làm gián đoạn, hoặc khi xoay màn hình thiết bị.

Đây là trách nhiệm của lập trình viên ứng dụng để xử lý những thay đổi trạng thái này một cách hợp lý, tránh tình trạng ứng dụng bị crash hoặc tiêu tốn tài nguyên hệ thống. Android framework cung cấp các callback methods giúp bạn biết khi nào Activity đang chuyển vào mỗi trạng thái trong Activity lifecycle.

Trong các bài giảng trước, chúng ta đã nhắc sơ qua về Activity lifecycle. Khi một Activity được khởi chạy, phương thức onCreate() sẽ được gọi, và một số thao tác khởi tạo sẽ diễn ra để Activity có thể chạy thành công.

Trong bài học này, chúng ta sẽ đi sâu hơn vào những yếu tố biến sơ đồ đơn giản ở slide trước thành luồng vòng đời (full lifecycle flow) đầy đủ.

Hãy cùng nói về các trạng thái cốt lõi của một Activity: Created, Started, Resumed, Paused, Stopped, và Destroyed.

Trong sơ đồ, ta có thể thấy cách một Activity chuyển đổi giữa các trạng thái khác nhau. Quá trình này chủ yếu diễn ra tuyến tính, tiến dần qua từng trạng thái.

Khi ứng dụng ở foreground, Activity sẽ ở trạng thái Resumed và xử lý tương tác của người dùng. Ngay khi Activity bị che một phần, hoặc khi người dùng rời khỏi Activity, ứng dụng sẽ chuyển qua các trạng thái Paused và Stopped (và đôi khi là Destroyed).

Một số trạng thái có thể chuyển ngược lại. Ví dụ:

Một Activity ở trạng thái Paused có thể quay lại Resumed.

Một Activity ở trạng thái Stopped có thể được Started lại.

Một Activity ở trạng thái Destroyed có thể được khởi tạo lại.

Lớp Activity cung cấp các callback cho những thay đổi trạng thái này. Ngoài ra còn có một số callback khác không được thể hiện trong sơ đồ, nhưng trong bài này chúng ta sẽ tập trung vào các callback cho những trạng thái chính.

onCreate()

Trong các ví dụ trước, chúng ta đã ghi đè (override) phương thức callback này để thực hiện việc thiết lập Activity và inflate layout. Phương thức onCreate() của Activity được gọi khi hệ thống lần đầu tiên tạo ra Activity. Đây là nơi bạn thực hiện logic khởi động cho ứng dụng, chẳng hạn như inflate giao diện người dùng (UI) của Activity và khởi tạo các biến hoặc thành phần của ứng dụng.

Tuy nhiên, Activity instance không dừng lại ở trạng thái Created. Sau khi mã trong onCreate() được thực thi, Activity sẽ chuyển sang trạng thái Started và hệ thống sẽ gọi phương thức onStart().

onStart

Phương thức onStart() được gọi khi Activity được khởi động và hiển thị với người dùng. Nó có thể được gọi khi Activity được khởi chạy lần đầu tiên (sau onCreate()), hoặc khi Activity được khởi động lại (sau onRestart()).

onResume()

Khi Activity bước vào trạng thái Resumed, phương thức `onResume()` sẽ được gọi. Lúc này, Activity đã hoạt động và sẵn sàng nhận tương tác từ người dùng.

Activity sẽ duy trì ở trạng thái này cho đến khi người dùng (hoặc hệ thống) thực hiện một hành động khiến Activity bị tạm dừng. Khi đó, phương thức `onPause()` sẽ được gọi.

onPause()

Trạng thái Paused được kích hoạt khi Activity mất tiêu điểm, nhưng vẫn còn hiển thị trên màn hình. Activity có thể đang trong quá trình đóng lại. Đây cũng là thời điểm đầu tiên bạn nên cân nhắc giải phóng tài nguyên. Tuy nhiên, bạn nên tránh khởi chạy các tác vụ đồng bộ kéo dài vì thời gian ở trạng thái này thường rất ngắn.

Nếu một Activity khác được khởi chạy đè lên trên, bạn sẽ nhận được lời gọi đến `onStop()` sau `onPause()`. Tuy nhiên, nếu Activity này chỉ bị che một phần và sau đó quay trở lại foreground, thì phương thức `onResume()` có thể sẽ được gọi tiếp theo.

onStop()

Một Activity khi ở trạng thái Stopped nên giải phóng nhiều tài nguyên vì lúc này Activity không còn hiển thị với người dùng. Đây là thời điểm thích hợp để dừng việc làm mới giao diện, chạy hoạt ảnh, hoặc các thay đổi trực quan khác.

Tuy nhiên, các Activity trong trạng thái Stopped chưa biến mất hoàn toàn. Nếu người dùng điều hướng quay lại Activity, thì `onRestart()` sẽ được gọi tiếp theo (sau đó là `onStart()` và `onResume()`). Nếu Activity đang kết thúc hoặc bị hệ thống hủy, thì `onDestroy()` sẽ được gọi tiếp theo.

onDestroy()

Activity có thể bước vào trạng thái Destroyed trong các trường hợp sau: người dùng chủ động tắt ứng dụng, Activity đã hoàn tất (ví dụ được khởi chạy để thực hiện một hành động và trả về giá trị), hoặc có sự thay đổi cấu hình (xoay màn hình hoặc thay đổi chế độ — từ chế độ một cửa sổ sang đa cửa sổ).

Phương thức `onDestroy()` sẽ xử lý việc dọn dẹp tài nguyên cuối cùng.

Tóm tắt các trạng thái của Activity

State	Callbacks	Description
Created	`onCreate()`	Activity được tạo lần đầu, khởi tạo UI và biến, nhưng chưa hiển thị với người dùng.
Started	`onStart()`	Activity trở nên hiển thị với người dùng nhưng chưa có focus để tương tác.
Resumed	`onResume()`	Activity ở foreground, hoạt động và sẵn sàng nhận input từ người dùng.
Paused	`onPause()`	Activity mất focus nhưng vẫn hiển thị; nên giải phóng tài nguyên tạm thời, tránh tác vụ dài.
Stopped	`onStop()`	Activity không còn hiển thị; nên giải phóng nhiều tài nguyên, dừng refresh UI và animations.
Restarted	`onRestart()`	Được gọi khi Activity từ trạng thái Stopped quay lại, trước khi `onStart()` và `onResume()`.
Destroyed	`onDestroy()`	Activity bị hủy (do người dùng tắt app, Activity kết thúc, hoặc thay đổi cấu hình). Dọn dẹp tài nguyên cuối cùng.

Lưu trạng thái

Trong slide về `onDestroy()`, chúng ta đã nói đến các thay đổi cấu hình, chẳng hạn như việc xoay thiết bị từ chế độ ngang sang dọc, hoặc thay đổi từ chế độ một cửa sổ sang đa cửa sổ. Những thay đổi cấu hình này sẽ khiến một Activity bị hủy và tạo lại.

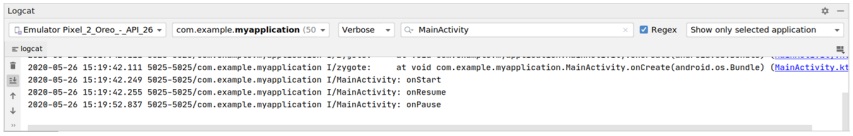
Framework cho phép chúng ta lưu một lượng dữ liệu nhỏ trong một `Bundle` để tái tạo lại bố cục. Không nên lưu trữ dữ liệu lớn trong `saved instance state bundles`. Chỉ nên lưu một lượng dữ liệu tối thiểu, chẳng hạn như một ID hoặc văn bản trong các ô nhập liệu, để có thể khôi phục lại trạng thái UI đã hiển thị cho người dùng trước đó. `Bundle` này sẽ được truyền lại cho Activity như một tham số đầu vào của các phương thức callback như `onCreate()` để bạn có thể thiết lập lại UI.

Một số phương thức callback của Activity và Fragment cũng cung cấp tham số cho phép khởi tạo lại trạng thái UI bằng `Bundle`. Trong các bài học sau, chúng ta sẽ tìm hiểu thêm những cách khác để lưu trạng thái.

7.2. Ghi log (Logging)

Logging có thể là một cách hữu ích để hiểu rõ hơn về các chuyển trạng thái trong vòng đời của Activity và Fragment. Cửa sổ `Logcat` trong Android Studio hiển thị cả các thông báo hệ thống, cũng như các thông báo log mà bạn đã thêm vào ứng dụng. Hãy cung cấp một chuỗi `String tag` duy nhất cho các thông điệp log của bạn để có thể tìm kiếm chúng dễ dàng hơn trong log. Quy ước phổ biến là sử dụng tên lớp đã kích hoạt thông điệp log làm log tag. Trong `Logcat`, bạn cũng có thể lọc thông điệp log dựa trên mức độ ưu tiên hoặc theo ứng dụng.

Nếu ứng dụng của bạn bị crash, `stack trace` sẽ hiển thị trong `Logcat`, và đây là công cụ rất hữu ích cho việc gỡ lỗi.



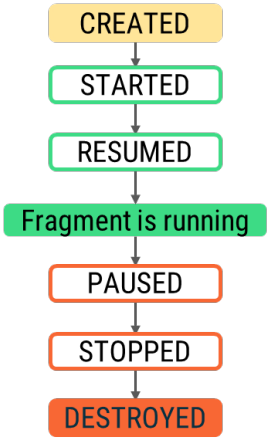
Bạn có thể gửi thông điệp tới `Logcat` với các mức độ ưu tiên khác nhau để chỉ ra tầm quan trọng của thông điệp, từ verbose (mức thấp nhất) đến error (mức cao nhất). Ghi log lỗi bằng `Log.e()`, và ghi log cảnh báo bằng `Log.w()`. Các log ở mức verbose, debug, và info thường được dùng cho mục đích thông tin.

Không nên biên dịch các log verbose vào ứng dụng, trừ khi đang trong giai đoạn phát triển. Log debug được biên dịch nhưng sẽ bị loại bỏ khi chạy thực tế, trong khi log error, warning và info thì luôn được giữ lại.

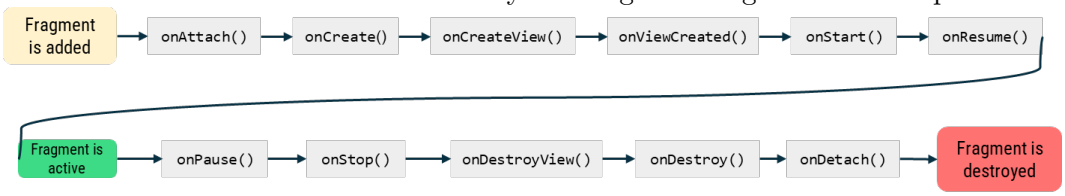
Priority level	**Log method**
Verbose	`Log.v()`
Debug	`Log.d()`
Info	`Log.i()`
Warning	`Log.w()`
Error	`Log.e()`

7.3. Vòng đời Fragment

Khi bạn tạo các fragment riêng và kế thừa từ lớp `Fragment`, sẽ có rất nhiều phương thức callback tương tự như trong lớp `Activity`. Tuy nhiên, cũng có một số callback mới mà bạn sẽ thấy ở slide tiếp theo.



Hình này minh họa một số callback mới của `Fragment`, chẳng hạn như `onAttach()`, `onCreateView()`, `onViewCreated()`, `onDestroyView()`, và `onDetach()`. Chúng ta sẽ tìm hiểu chi tiết hơn về các callback này trong những slide tiếp theo.



`onAttach()` được gọi ngay trước phương thức `onCreate()` của một Fragment. `onAttach()` được gọi khi Fragment chưa có quyền truy cập vào layout của nó, vì vậy bạn sẽ không thường xuyên ghi đè phương thức này.

Phương thức `onCreateView()` được gọi để Fragment khởi tạo giao diện UI của nó. Phương thức này được gọi giữa `onCreate()` và `onViewCreated()`. Chúng ta nên chỉ thực hiện việc inflate layout trong phương thức này, và chuyển phần logic dùng để thay đổi `View` đã trả về sang `onViewCreated()`.

Hoàn tất việc thiết lập UI trong callback `onViewCreated()`, bởi vì tại thời điểm này chúng ta có thể chắc chắn rằng các view đã sẵn sàng.

`onDestroyView()` được gọi khi `View` (trước đó đã được tạo ra từ `onCreateView()`) bị tách ra khỏi Fragment. `onDetach()` được gọi khi Fragment không còn gắn với Activity chứa nó nữa.

State	Callbacks	Description
Attached	`onAttach()`	Fragment gắn với Activity. Được gọi trước `onCreate()`, thường ít khi override.
Created	`onCreate()`	Fragment được tạo lần đầu, khởi tạo dữ liệu nhưng chưa có UI.
Create View	`onCreateView()`	Fragment khởi tạo UI view, thường chỉ inflate layout.
View Created	`onViewCreated()`	Fragment đã có view, có thể thiết lập UI hoặc bind dữ liệu.
Started/Resumed	`onStart()`, `onResume()`	Fragment hiển thị và sẵn sàng nhận tương tác từ người dùng.
Destroy View	`onDestroyView()`	View bị gỡ khỏi Fragment, giải phóng tài nguyên liên quan đến view.
Detached	`onDetach()`	Fragment không còn gắn với Activity. Đây là bước cuối trước khi Fragment bị huỷ.

Lưu trạng thái Fragment khi thay đổi cấu hình: Tương tự như trong Activity, bạn có thể giữ lại trạng thái UI của một Fragment khi có thay đổi cấu hình hoặc khi ứng dụng bị hệ điều hành tạm dừng/ngắt bằng cách sử dụng `onSaveInstanceState()` và đưa dữ liệu của bạn vào trong `Bundle`. Cùng một `Bundle` này sẽ được truyền lại thông qua các phương thức callback khi một instance mới của Fragment được tạo lại.

Chỉ nên lưu một lượng dữ liệu tối thiểu trong `Bundle`. Các dữ liệu người dùng khác nên được lưu trữ bằng các giải pháp lưu trữ bền vững khác, chẳng hạn như cơ sở dữ liệu.

7.4. Các thành phần nhận biết vòng đời (Lifecycle-aware components)

Để điều chỉnh hành vi của chúng dựa trên vòng đời của Activity hoặc Fragment.

- Sử dụng thư viện `androidx.lifecycle`.
- `Lifecycle` theo dõi trạng thái vòng đời của một Activity hoặc Fragment.
  - Giữ trạng thái vòng đời hiện tại.
  - Gửi (dispatch) các sự kiện vòng đời khi có thay đổi trạng thái.

Một lớp triển khai giao diện `LifecycleOwner` sẽ có một vòng đời (lifecycle) và cần triển khai phương thức `getLifecycle()`. Giao diện này trừu tượng hóa việc sở hữu một `Lifecycle` khỏi từng lớp cụ thể, chẳng hạn như `Fragment` và

`AppCompatActivity`. Điều này cũng cho phép các lớp khác có thể triển khai giao diện `LifecycleOwner`.

Với bất kỳ lớp nào muốn lắng nghe các sự kiện vòng đời, hãy triển khai giao diện `LifecycleObserver`. Lưu ý rằng `LifecycleObserver` không có phương thức trừu tượng nào, mà thay vào đó sử dụng các phương thức được đánh dấu bằng annotation `@OnLifecycleEvent`, như trong ví dụ. Sau đó, đăng ký observer này với `Lifecycle` để nhận thông báo về các sự kiện vòng đời.

```
class MyObserver : LifecycleObserver {
@OnLifecycleEvent(Lifecycle.Event.ON_RESUME)
fun connectListener() {
    ...
}
```

Thêm observer vào vòng đời (lifecycle):

```
myLifecycleOwner.getLifecycle().addObserver(MyObserver())
```

Việc sử dụng các thành phần nhận biết vòng đời (lifecycle-aware components) sẽ giúp bạn viết code gọn gàng, có tổ chức hơn và dễ bảo trì hơn.

7.5. Nhiệm vụ và ngăn xếp (Tasks & Back stack)

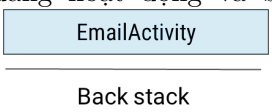
Bây giờ khi chúng ta đã tìm hiểu nhiều hơn về vòng đời của `Fragment` và `Activity`, hãy xem việc điều hướng giữa các Activity hoặc Fragment ảnh hưởng như thế nào đến `back stack` của ứng dụng.

Ôn lại kiến thức bạn đã học trong bài trước về `task`: một `task` là tập hợp các Activity mà người dùng tương tác khi thực hiện một công việc nào đó. Các Activity được sắp xếp trong một ngăn xếp — gọi là `back stack` — theo thứ tự chúng được mở ra. Nó được coi là một ngăn xếp vì có cấu trúc "last in, first out": Activity vào sau cùng sẽ là Activity được loại bỏ đầu tiên.

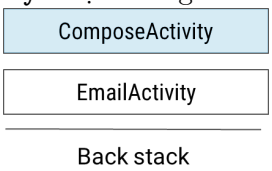
Khi người dùng mở một ứng dụng, `task` của ứng dụng đó sẽ xuất hiện ở foreground. Nếu chưa có `task` nào tồn tại cho ứng dụng (ví dụ ứng dụng chưa được sử dụng gần đây), thì một `task` mới sẽ được tạo và Activity "main" của ứng dụng sẽ mở ra như là root activity trong ngăn xếp. Hãy cùng xem một ví dụ.

Giả sử bạn mở ứng dụng email trên điện thoại, và nếu chưa có `task` nào cho ứng dụng này, một `task` mới sẽ được tạo. Activity chính của ứng dụng email (giả sử gọi là `EmailActivity`) sẽ được thêm vào `back stack`. Nó nằm trên

đỉnh `back stack`, do đó nó đang hoạt động và bạn có thể tương tác với nó.

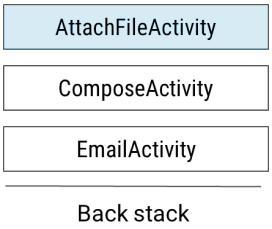


Sau đó, trong ứng dụng email, bạn quyết định viết một email mới. Việc này được thực hiện bằng cách mở một Activity thứ hai mà chúng ta gọi là `ComposeActivity`. Lúc này, Activity đầu tiên (`EmailActivity`) đã chuyển sang trạng thái `Paused` và `Stopped` theo vòng đời. Còn `ComposeActivity` thì đã được tạo (`Created`), khởi động (`Started`) và tiếp tục hoạt động (`Resumed`) vì bạn đang tương tác với nó. Bạn vẫn đang ở trong cùng một `task`, và `ComposeActivity` hiện đang nằm trên đỉnh của `back stack`.

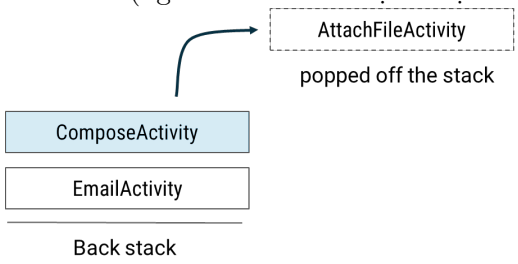


Tiếp theo, bạn quyết định đính kèm một bức ảnh vào email này. Bạn nhấn vào menu "Attach File", và một Activity mới mở ra để duyệt các tệp trên thiết bị của bạn.

Bây giờ `AttachFileActivity` đã được mở. Lúc này `ComposeActivity` không còn hiển thị, nên Activity đó đã chuyển sang trạng thái `Stopped`. `AttachFileActivity` hiện đang hoạt động và nằm trên đỉnh của `back stack`.

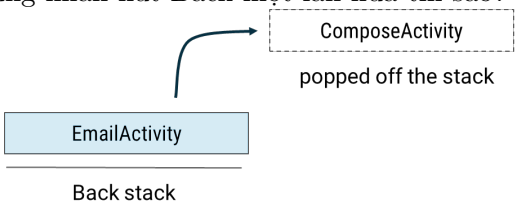


Nhờ việc duy trì một ngăn xếp các Activity, nếu người dùng nhấn nút Back, họ có thể quay lại Activity trước đó. Trong `AttachFileActivity`, khi người dùng nhấn Back, thì `AttachFileActivity` sẽ bị `popped` khỏi stack (nghĩa là nó được loại bỏ khỏi ngăn xếp).



Vậy bây giờ Activity nào đang ở trên đỉnh của `back stack`? Đó chính là `ComposeActivity`. Do đó, `ComposeActivity` sẽ được `Started` và `Resumed` trở lại, và trở thành Activity mà người dùng đang tương tác.

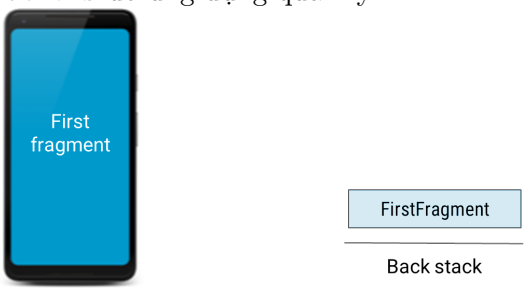
Vậy nếu người dùng nhấn nút Back một lần nữa thì sao?



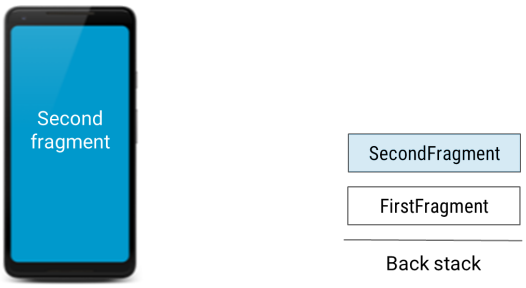
Khi `ComposeActivity` bị `popped` ra khỏi `back stack`, `EmailActivity` sẽ còn lại trên đỉnh và trở nên hiển thị cũng như hoạt động trở lại. Điều này cho thấy vì sao việc có một `back stack` các Activity lại hữu ích.

Bạn cũng có thể duy trì một `back stack` của các Fragment. Khi sử dụng thư viện `Navigation`, một `back stack` của các `destination` được giữ lại, nhờ vậy nút Back vẫn hoạt động đúng chức năng.

Hãy cùng xem điều này hoạt động như thế nào. Giả sử chúng ta có một ứng dụng đang chạy với một Activity duy nhất có `navigation host`. `FirstFragment` được gắn, khởi tạo và sẵn sàng để người dùng tương tác. Fragment này được thêm vào `stack of destinations` do ứng dụng quản lý.

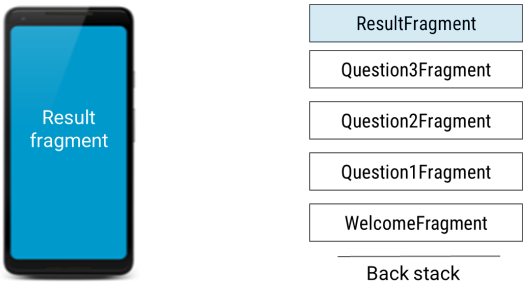


Sau đó, người dùng nhấn một nút để điều hướng đến một `destination` mới là `SecondFragment`. Trong khi fragment mới này đang gọi `onCreate()`, `onCreateView()`, v.v., thì `FirstFragment` không còn hiển thị và chuyển sang trạng thái `Paused` rồi `Stopped`. Lúc này, `SecondFragment` nằm trên đỉnh của `stack of destinations`.

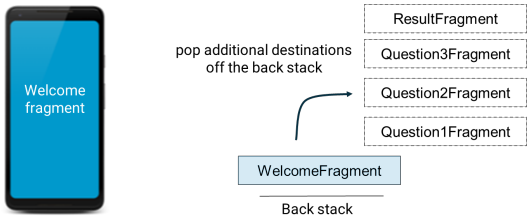


Trong nhiều trường hợp, hành vi mặc định của `back stack` là phù hợp: tức là, một lần nhấn nút Back sẽ đưa bạn lùi lại một bước. Tuy nhiên, trong một số tình huống, bạn có thể muốn quay lại một `destination` sớm hơn trong `back stack`.

Ví dụ, giả sử bạn có một ứng dụng quiz. Sau khi nhận kết quả và nhấn nút Back, bạn mong đợi điều gì sẽ xảy ra? Quay lại câu hỏi cuối cùng của bài quiz, hay quay lại `WelcomeFragment` ban đầu để làm lại quiz? Phần lớn người dùng sẽ muốn bắt đầu lại quiz từ đầu.



Trong trường hợp này, bạn có thể ghi đè hành vi mặc định để `pop` nhiều `destination` khỏi stack, quay trở lại `WelcomeFragment` chỉ với một lần nhấn nút Back.



7.6. Tóm tắt

Trong bài học 7, bạn đã học cách:

- Hiểu cách một instance của `Activity` chuyển qua các trạng thái vòng đời khác nhau khi người dùng tương tác hoặc rời ứng dụng.
- Giữ lại trạng thái UI khi thay đổi cấu hình bằng `Bundle`.

- Các phương thức callback vòng đời của `Fragment` tương tự như `Activity`, nhưng có thêm một số phương thức khác.
- Sử dụng các thành phần nhận biết vòng đời (`Lifecycle-aware components`) để tổ chức code gọn gàng hơn.
- Sử dụng hành vi mặc định hoặc tuỳ chỉnh của `back stack`.
- Sử dụng `logging` để hỗ trợ debug và theo dõi trạng thái của ứng dụng.

Chương 8
Kiến trúc ứng dụng (Tầng giao diện)

8.1. Kiến trúc ứng dụng Android 125
8.2. ViewModel 127
8.3. Data Binding 131
8.4. LiveData 133
8.5. Transform LiveData 137
8.6. Tóm tắt chương 137

8.1. Kiến trúc ứng dụng Android

Khi bạn đưa các giải pháp vá lỗi ngắn hạn vào ứng dụng (thay vì làm việc theo kế hoạch và áp dụng best practices), ứng dụng của bạn có thể bắt đầu tích lũy `technical debt`. `Technical debt` là chi phí cho việc phải làm lại trong tương lai, do đã bỏ qua những công việc lẽ ra nên được thực hiện ngay từ đầu.

`Technical debt` có thể phát sinh từ các yếu tố bên ngoài như hạn chót cho một tính năng, một bài tập ở trường, hoặc một lỗi khẩn cấp cần được sửa gấp. Một chuỗi các bản vá nhanh có thể trông như một giải pháp dễ dàng, và trong một số trường hợp hiếm hoi thì việc chọn cách sửa tạm có thể chấp nhận được. Tuy nhiên, với những dự án có nhiều mốc (milestone), việc không cân nhắc kỹ các quyết định thiết kế khi phạm vi dự án mở rộng có thể gây hại về lâu dài.

Mặc dù một số `technical debt` là không thể tránh khỏi, chiến lược tốt nhất là nhận thức được nó và cố gắng giảm thiểu bằng cách tuân thủ best practices.

Các ví dụ về quyết định ngắn hạn trong Android có thể dẫn đến `technical debt`, và làm cho việc mở rộng ứng dụng trở nên khó khăn, thiếu linh hoạt và kém khả năng mở rộng:

- Tuỳ biến ứng dụng cho một thiết bị cụ thể thay vì tính đến nhiều thiết bị khác nhau → giới hạn phạm vi người dùng mà ứng dụng có thể tiếp cận.

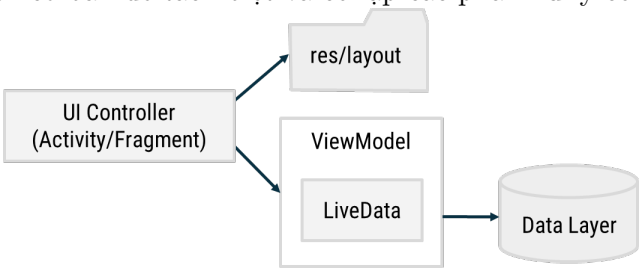
- Sao chép và dán code của bên thứ ba vào file mà không thực sự hiểu nó → dễ gây xung đột và lỗi không mong muốn.
- Đặt toàn bộ business logic vào file `Activity` → khiến file trở nên quá lớn và khó quản lý.
- Hardcode các chuỗi hiển thị với người dùng trong code → gây khó khăn cho việc `localization` sang các ngôn ngữ khác.

Trong các nội dung tiếp theo, chúng ta sẽ tập trung vào `app architecture`. Một `app architecture` tốt sẽ đảm bảo rằng ứng dụng của bạn có thể mở rộng, đáng tin cậy và dễ quản lý. Khi có một cấu trúc rõ ràng quy định nơi mà `business logic` thuộc về trong ứng dụng, code sẽ dễ hiểu hơn và các lập trình viên có thể dễ dàng cộng tác cũng như duy trì tính nhất quán giữa các module.

Để giúp bạn xây dựng ứng dụng với `app architecture` tốt, chúng ta sẽ sử dụng `Android Jetpack`. Thực tế, bạn đã từng dùng các thư viện Jetpack khi làm việc với `ConstraintLayout`, `RecyclerView`, và các thư viện trong gói `androidx.*`.

`Android Jetpack` là một tập hợp các thư viện, công cụ, và tài liệu hướng dẫn giúp bạn viết các ứng dụng chất lượng cao một cách dễ dàng hơn. Các thành phần này giúp bạn tuân thủ best practices, giảm bớt việc viết `boilerplate code`, và đơn giản hóa các tác vụ phức tạp. Hướng dẫn về kiến trúc app trong Jetpack (Jetpack guide) được liên kết bên dưới là một tài liệu rất hữu ích.

Hướng dẫn "`Guide to app architecture`" trong Jetpack nói về các nguyên tắc kiến trúc phổ biến cần tuân thủ. Một trong số đó là phân tách mối quan tâm. Hình minh họa này không chỉ ra một mẫu thiết kế cụ thể nào cần tuân theo, nhưng nó cho thấy một cách cơ bản để tách biệt và cô lập các phần xử lý công việc.



`Activity` hoặc `Fragment` chịu trách nhiệm hiển thị dữ liệu, ghi nhận input từ người dùng và các sự kiện từ hệ thống Android. Nó đóng vai trò như một `controller` cho UI.

`ViewModel` chứa toàn bộ dữ liệu cần thiết để vẽ UI, cũng như các hàm để giữ cho UI luôn được cập nhật. Sau này trong bài học, chúng ta sẽ nói kỹ hơn về cách

`LiveData` có thể giúp bạn giữ cho UI luôn cập nhật.

Trong các bài giảng tiếp theo, chúng ta sẽ thảo luận về những thành phần cụ thể kiểm soát cách chúng ta cung cấp dữ liệu cho `ViewModel`.

Có thể bạn đã từng nghe đến các mẫu thiết kế kiến trúc như `MVVM` (Model–View–ViewModel), `MVI` (Model–View–Interactor), và nhiều mẫu khác. Những mẫu thiết kế này đưa ra định hướng tổng quát về cách ứng dụng của bạn nên được tổ chức.

Các thành phần kiến trúc trong thư viện Jetpack cung cấp cho bạn những công cụ cụ thể để triển khai các mẫu thiết kế đó. Trong bài học này, chúng ta sẽ tập trung vào các thành phần kiến trúc đó.

8.2. ViewModel

Trong sơ đồ trước, bạn có thể đã chú ý đến lớp `ViewModel`, vốn là một phần của các thành phần kiến trúc Android. Hãy cùng thảo luận tại sao chúng ta cần lớp này.

Framework Android quản lý vòng đời của các UI controller như `Activity` và `Fragment`. Framework có thể quyết định huỷ hoặc tạo lại một UI controller để phản ứng với các hành động của người dùng hoặc các sự kiện hệ thống nằm ngoài tầm kiểm soát của bạn. Nếu hệ thống huỷ hoặc tạo lại UI controller, mọi dữ liệu tạm thời lưu trong đó sẽ bị mất.

Bởi vì các UI controller chủ yếu chịu trách nhiệm hiển thị thông tin cho người dùng và xử lý các sự kiện nhập liệu, chúng ta không muốn chất thêm trách nhiệm quản lý dữ liệu cho chúng. Do đó, chúng ta cần một thực thể riêng để quản lý dữ liệu, và đó chính là `ViewModel`.

Thêm dependency sau vào `build.gradle (Module: app)`:

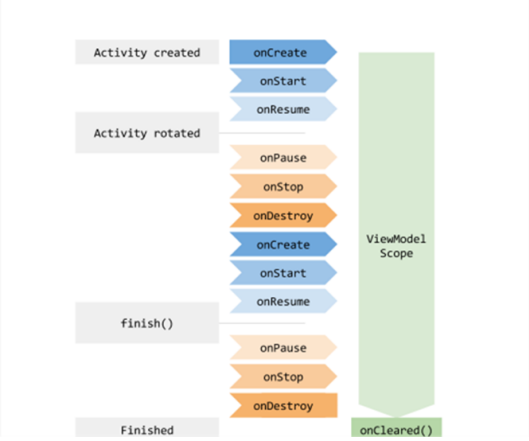
```
dependencies {
    def lifecycle_version = "2.8.0"
    implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:$lifecycle_version"
    implementation "androidx.lifecycle:lifecycle-livedata-ktx:$lifecycle_version"
    implementation "androidx.lifecycle:lifecycle-runtime-ktx:$lifecycle_version"
}
```

`ViewModel` là lớp chịu trách nhiệm chuẩn bị và quản lý dữ liệu cho UI, còn `Activity` hoặc `Fragment` sẽ hiển thị dữ liệu từ `ViewModel`. Tuy nhiên, `ViewModel` không nên giữ tham chiếu trực tiếp tới các view trong cây view, hoặc tham chiếu

tới chính `Activity` hay `Fragment`. Điều này giúp tách biệt việc quản lý dữ liệu ra khỏi logic của UI controller.

Các đối tượng `ViewModel` được gắn với một `Lifecycle` (mỗi `Activity` và `Fragment` đều có `Lifecycle`). Các `ViewModel` sẽ tự động được giữ lại khi có thay đổi cấu hình, nhờ vậy dữ liệu mà chúng lưu giữ vẫn sẵn sàng cho instance tiếp theo của `Activity` hoặc `Fragment`. `ViewModel` sẽ ở trong bộ nhớ cho đến khi `Lifecycle` mà nó gắn với bị huỷ hoàn toàn.

Sơ đồ này minh họa vòng đời của một `ViewModel` song song với `Activity` liên kết của nó, khi `Activity` trải qua các thay đổi vòng đời khác nhau. `ViewModel` vẫn được giữ lại ngay cả khi có thay đổi cấu hình (khi `Activity` bị huỷ và tạo lại).

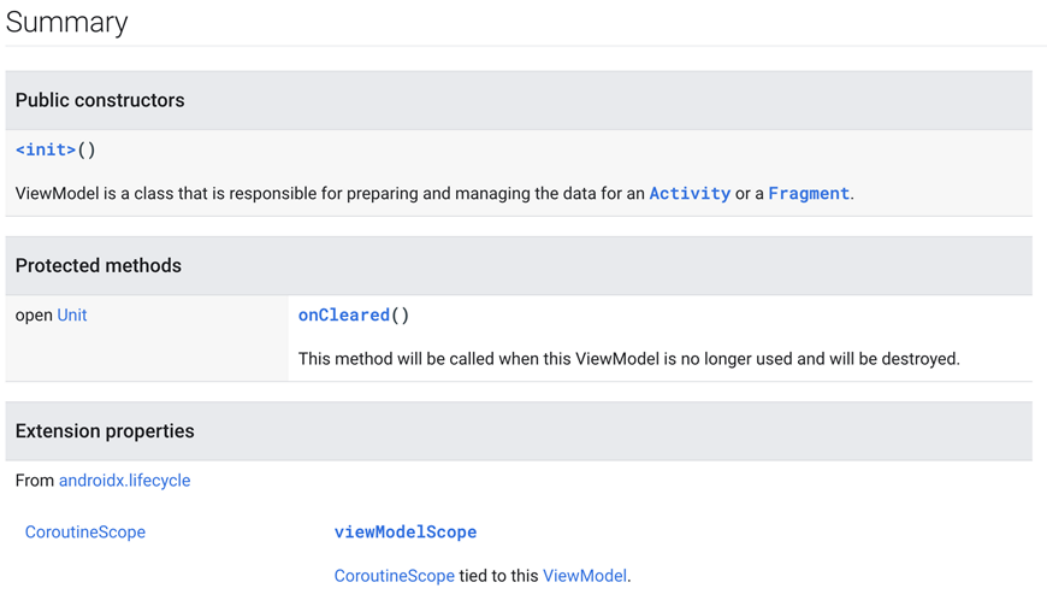


Lưu ý rằng chúng ta không còn cần phải thủ công lưu và khôi phục trạng thái từ `Bundle` nữa, bởi vì ta có thể lấy lại cùng một instance của `ViewModel` sau khi thay đổi cấu hình và thiết lập lại UI. `ViewModel` sẽ tồn tại trong bộ nhớ cho đến khi `Activity` kết thúc, và sau đó phương thức `onCleared()` của `ViewModel` sẽ được gọi để dọn dẹp tài nguyên.

Để thấy cách `ViewModel` được sử dụng trong một ứng dụng, hãy cùng xem một số bước cần thiết để tạo một ứng dụng ghi điểm cho môn Kabaddi — một môn thể thao đối kháng phổ biến ở Nam Á. Trong Kabaddi, có hai đội, mỗi đội gồm bảy người chơi, và điểm số được cộng theo bội số +1 hoặc +2 cho một trong hai đội.



Trước khi chúng ta xem code để tạo một `ViewModel`, cần lưu ý rằng `ViewModel` là một `abstract class` trong Kotlin. Nếu bạn còn nhớ từ các bài học Kotlin trước, một `abstract class` thì không thể được khởi tạo trực tiếp, mà phải được kế thừa. Nó có phương thức `protected onCleared()`, được gọi khi `ViewModel` không còn được sử dụng và sẽ bị huỷ.



Summary

Public constructors

<init>()

ViewModel is a class that is responsible for preparing and managing the data for an Activity or a Fragment.

Protected methods

open Unit — onCleared()

This method will be called when this ViewModel is no longer used and will be destroyed.

Extension properties

From androidx.lifecycle

CoroutineScope — viewModelScope

CoroutineScope tied to this ViewModel.

Để tạo một `ViewModel` cho ứng dụng KabaddiKounter, trước tiên hãy tạo lớp `ScoreViewModel` kế thừa từ `ViewModel` trừu tượng. `ScoreViewModel` sẽ lưu trữ hai điểm số kiểu `Int`, một cho đội A và một cho đội B.

```
    class ScoreViewModel : ViewModel() {
    var scoreA : Int = 0
    var scoreB : Int = 0
    fun incrementScore(isTeamA: Boolean) {
        if (isTeamA) {
            scoreA++
        }
        else {
            scoreB++
        }
    }
}
```

Trong `MainActivity`, hãy sử dụng Kotlin property delegate `by viewModels()` (từ thư viện `activity-ktx`) để lấy `ScoreViewModel`.

```
    class MainActivity : AppCompatActivity() {
    // Delegate provided by androidx.activity.viewModels
```

```
val viewModel: ScoreViewModel by viewModels()

override fun onCreate(savedInstanceState: Bundle?) {
    ...
    val scoreViewA: TextView = findViewById(R.id.scoreA)
    scoreViewA.text = viewModel.scoreA.toString()
}
```

Chúng ta thêm một `click listener` cho nút +1 của đội A, thao tác này sẽ tăng điểm bằng cách gọi đến `ViewModel`. Sau đó, ta cập nhật `TextView` với điểm số mới. Chúng ta sẽ cải tiến sau để không phải tự cập nhật `TextView` thủ công nữa.

```
val scoreViewA: TextView = findViewById(R.id.scoreA)
val plusOneButtonA: Button = findViewById(R.id.plusOne_teamA)

plusOneButtonA.setOnClickListener {
    viewModel.incrementScore(true)
    scoreViewA.text = viewModel.scoreA.toString()
}
```

8.3. Data Binding

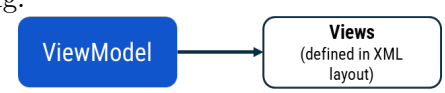
Các đối tượng `ViewModel` lưu giữ dữ liệu trong ứng dụng của bạn. Trong layout, sẽ đơn giản hơn nếu instance của `ViewModel` có thể giao tiếp trực tiếp với các `View`, mà không cần phụ thuộc vào UI controller như một tầng trung gian.

Bằng cách truyền các đối tượng `ViewModel` vào `data binding`, bạn có thể tự động hoá một phần việc giao tiếp giữa các `View` và `ViewModel`.

Kiến trúc không sử dụng data binding:



Sử dụng data binding:



Chúng ta có thể sử dụng một `ViewModel` trong layout XML giống như bất kỳ đối tượng nào khác. Lưu ý rằng việc khai báo này không tự động bind một `ViewModel`. Nó chỉ khai báo rằng sẽ có một `ScoreViewModel` được expose cho binding này.

```
<layout>
   <data>
       <variable>
           name="viewModel"
           type="com.example.kabaddikounter.ScoreViewModel" />
   </data>
   <ConstraintLayout ../>
</layout>
```

Đây là nơi chúng ta thực sự gắn một `ViewModel` vào `binding`.

```
class MainActivity : AppCompatActivity() {
    val viewModel: ScoreViewModel by viewModels()
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val binding: ActivityMainBinding = DataBindingUtil.setContentView(this,
            R.layout.activity_main)
        binding.viewModel = viewModel
        ...
```

Sau khi hoàn tất bước binding, chúng ta có thể sử dụng `ViewModel` đó trong một biểu thức `data binding` trong layout XML. Khi view được inflate lần đầu, `TextView` này sẽ phản ánh đúng điểm số của Team A.

Trong file `activity_main.xml`:

```
<TextView
    android:id="@+id/scoreViewA"
    android:text="@{viewModel.scoreA.toString()}" />
        ...
```

Khi nút được bấm, chúng ta vẫn cần cập nhật TextView vì thuộc tính android:text của TextView chỉ được thiết lập khi view được khởi tạo. Nó không tự cập nhật khi ViewModel thay đổi. Chúng ta sẽ xử lý vấn đề đó ở phần tiếp theo.

```
override fun onCreate(savedInstanceState: Bundle?) {
    ...
   val binding: ActivityMainBinding = DataBindingUtil.setContentView(this,
```

```
        R.layout.activity_main)

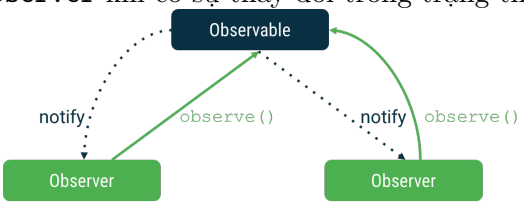
    binding.plusOneButtonA.setOnClickListener {
        viewModel.incrementScore(true)
        binding.scoreViewA.text = viewModel.scoreA.toString()
    }
}
```

8.4. LiveData

Mẫu thiết kế observer mô tả việc một đối tượng (subject hoặc observable) duy trì danh sách các đối tượng phụ thuộc (observers) để thông báo khi trạng thái của subject thay đổi.

`Data binding` đã giúp giảm bớt một số phép gán và lời gọi hàm trong UI của chúng ta. Tuy vậy, chúng ta vẫn phải chủ động cập nhật các `View` khi giá trị trong `ViewModel` thay đổi. Có một cách tốt hơn.

Mẫu thiết kế observer quy định cách giao tiếp giữa một đối tượng `observable` (hay còn gọi là subject của việc quan sát) và các `observer`. Một `observable` sẽ thông báo cho các `observer` khi có sự thay đổi trong trạng thái của nó.



Trong ứng dụng của chúng ta, chúng ta sẽ sử dụng `LiveData`, một data holder có khả năng quan sát (observable) và nhận biết vòng đời (lifecycle-aware). Về cơ bản, `LiveData` là một lớp bao bọc quanh bất kỳ kiểu dữ liệu nào. Ví dụ: `LiveData<Int>` có thể chứa một giá trị kiểu `Int`. Vì dữ liệu đã được bao bọc bởi `LiveData`, chúng ta có thể quan sát các thay đổi của dữ liệu đó.

Theo mẫu thiết kế `observer` mà chúng ta vừa nhắc đến, `LiveData` đóng vai trò là `observable`, và ta có thể thêm các `observer` để lắng nghe sự thay đổi dữ liệu. Các `observer` này sẽ là các UI Controller như `Activity` hoặc `Fragment`, những thành phần muốn biết khi dữ liệu thay đổi để cập nhật UI. Các lớp `LiveData` đóng vai trò quan trọng trong việc truyền dữ liệu từ `ViewModel` tới `Fragment` hoặc `Activity`.

Khi nhìn vào phương thức, hãy lưu ý rằng một `Observer` được thêm cùng với một `LifecycleOwner`, và `Observer` chỉ nhận cập nhật khi `LifecycleOwner` đang ở

trạng thái `active`.

`LiveData` không có các phương thức public để cập nhật dữ liệu bên trong, bạn có thể xem nó như dạng "chỉ đọc". Ngược lại, lớp `MutableLiveData` cung cấp các phương thức public là `setValue(T)` và `postValue(T)`.

Trong thực tế, `MutableLiveData` thường được khai báo như một biến `private` trong `ViewModel`, và sau đó `ViewModel` chỉ expose một đối tượng `LiveData` bất biến ra ngoài cho các `observer`. Điều này đảm bảo nguyên tắc `separation of concerns`: `ViewModel` chịu trách nhiệm triển khai logic và cập nhật `MutableLiveData`, trong khi vai trò của UI chỉ đơn thuần là phản ứng lại với những thay đổi giá trị trong `LiveData`.

LiveData vs MutableLiveData

LiveData<T>	**MutableLiveData<T>**
- Dùng để quan sát dữ liệu. - Chỉ có thể *đọc* dữ liệu, không thể thay đổi giá trị bên trong. - Thường được `expose` ra ngoài từ ViewModel để UI quan sát.	- Kế thừa từ `LiveData<T>`. - Có thể *ghi* dữ liệu thông qua `setValue(T)` hoặc `postValue(T)`. - Thường được khai báo `private` trong ViewModel để giữ quyền cập nhật dữ liệu.

Ở đây, chúng ta đã cập nhật code của `ViewModel` để sử dụng `LiveData`. Chúng ta không muốn bất kỳ đối tượng hoặc `observer` bên ngoài nào có thể thay đổi điểm số, vì vậy ta dùng một `MutableLiveData` dạng `private` và một `LiveData` dạng `public` (không thể bị gán lại). Lưu ý rằng biến `_scoreA` được khởi tạo thành 1 `MutableLiveData` chứa giá trị `Int` bằng 0. Khi cần tăng điểm, ta chỉ thay đổi `_scoreA`. Nếu bất kỳ bên ngoài nào truy cập thuộc tính `scoreA` công khai, lớp sẽ trả về `_scoreA` dưới dạng một `LiveData` bất biến.

```
class ScoreViewModel : ViewModel() {

    private val _scoreA = MutableLiveData<Int>(0)
    val scoreA: LiveData<Int>
        get() = _scoreA

    fun incrementScore(isTeamA: Boolean) {
        if (isTeamA) {
            _scoreA.value = _scoreA.value!! + 1
        }
```

```
        ...\
```

Bây giờ khi đã thiết lập xong, chúng ta có thể tạo một `observer` để theo dõi `scoreA` và cập nhật `TextView` tương ứng trên màn hình mỗi khi người dùng nhấn nút +1. Lưu ý rằng chúng ta có thể sử dụng phiên bản `SAM` của interface `Observer` (`onChanged(t: T)` được chuyển đổi thành `SAM`).

Thiết lập sự kiện click để tăng điểm trong `ViewModel`:

```
binding.plusOneButtonA.setOnClickListener {
    viewModel.incrementScore(true)
}
```

Tạo `observer` để cập nhật điểm số của đội A trên màn hình:

```
val scoreA_Observer = Observer<Int> { newValue ->
    binding.scoreViewA.text = newValue.toString()
}
```

Thêm `observer` vào `scoreA LiveData` trong `ViewModel`:

```
viewModel.scoreA.observe(this, scoreA_Observer)
```

Trong các mục tiếp theo, chúng ta sẽ xem trạng thái cuối cùng của code khi đã tích hợp `LiveData`. Với cách này, chúng ta đã giảm đáng kể lượng code cần viết để giữ cho UI luôn đồng bộ với dữ liệu.

Chúng ta gán thuộc tính `text` của `TextView` trực tiếp bằng biến `ViewModel`. Cách này sẽ loại bỏ nhu cầu phải viết `observer` cho `LiveData`.

```
<layout>
   <data>
       <variable>
           name="viewModel"
           type="com.example.kabaddikounter.ScoreViewModel" />
   </data>
   <ConstraintLayout ..>
       <TextView ...
           android:id="@+id/scoreViewA"
```

```
            android:text="@{viewModel.scoreA.toString()}" />
        ...
    </ConstraintLayout>
</layout>
```

Chúng ta gán `viewModel` cho đối tượng `binding`, đồng thời chỉ định `activity` hiện tại làm `lifecycleOwner`.

Lưu ý rằng lúc này chúng ta không còn cần đối tượng `observer`. Chúng ta cũng không cần cập nhật `TextView` điểm số thủ công khi nút +1 được nhấn.

```
class MainActivity : AppCompatActivity() {
    val viewModel: ScoreViewModel by viewModels()
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val binding: ActivityMainBinding = DataBindingUtil
        .setContentView(this, R.layout.activity_main)
        binding.viewModel = viewModel
        binding.lifecycleOwner = this
        binding.plusOneButtonA.setOnClickListener {
            viewModel.incrementScore(true)
        }
```

Giờ đây, mọi thay đổi đối với điểm số của đội A hoặc đội B sẽ tự động cập nhật các `TextView` trong layout. Mỗi khi phương thức `incrementScore()` được gọi, thay đổi sẽ được phản ánh ngay lập tức trên UI.

```
class ScoreViewModel : ViewModel() {
    private val _scoreA = MutableLiveData<Int>(0)
    val scoreA : LiveData<Int>
        get() = _scoreA
    private val _scoreB = MutableLiveData<Int>(0)
    val scoreB : LiveData<Int>
        get() = _scoreB
    fun incrementScore(isTeamA: Boolean) {
        if (isTeamA) {
            _scoreA.value = _scoreA.value!! + 1
```

```
        } else {
            _scoreB.value = _scoreB.value!! + 1
        }
    }
}
```

8.5. Transform LiveData

Chúng ta có thể sử dụng các phương thức `Transformation` để thao tác dữ liệu trên `LiveData` gốc và trả về một đối tượng `LiveData`. Những phép biến đổi này sẽ không được tính toán trừ khi có một `observer` đang quan sát `LiveData` được trả về.

Các phương thức này nhận vào hai tham số:

`source LiveData` (nguồn dữ liệu gốc).

Một `function` dùng để thao tác dữ liệu từ `source LiveData`.

Trong đoạn code này, giả sử một trận đấu kết thúc khi một đội đạt 10 điểm. Hàm `map` sẽ quan sát `scoreA` trong `ViewModel` và tạo ra một `LiveData` mới chứa một chuỗi `String`. Đối tượng `LiveData` này có thể được quan sát để kích hoạt thay đổi trong UI.

```
val result: LiveData<String> = Transformations.map(viewModel.scoreA) {
    x -> if (x > 10) "A Wins" else ""
}
```

8.6. Tóm tắt chương

Trong bài học 8, bạn đã học cách:

- Tuân theo thiết kế kiến trúc ứng dụng tốt và nguyên tắc `separation of concerns` để giúp ứng dụng dễ bảo trì hơn và giảm `technical debt`.
- Tạo một `ViewModel` để lưu dữ liệu tách biệt khỏi UI controller.
- Sử dụng `ViewModel` cùng với `data binding` để tạo UI phản hồi nhanh hơn với ít code hơn.
- Sử dụng `observer` để tự động nhận cập nhật từ `LiveData`.

Chương 9
Kiến trúc ứng dụng (Lưu trữ bền vững)

9.1. Lưu trữ dữ liệu 138
9.2. Lập trình bất đồng bộ 148
9.3. Kiểm thử với CSDL 154
9.4. Tóm tắt chương 155

9.1. Lưu trữ dữ liệu

Cho đến nay, các ứng dụng mà chúng ta đã tạo chủ yếu chỉ hiển thị dữ liệu được lưu trong bộ nhớ của ứng dụng. Tuy nhiên, chúng ta có thể xây dựng một ứng dụng hữu ích hơn nhiều nếu biết cách lưu trữ dữ liệu bền vững (persist data) vào bộ nhớ cục bộ của thiết bị theo cách có thể mở rộng và linh hoạt.

Android sử dụng hệ thống file tương tự như các hệ thống file dựa trên đĩa ở các nền tảng khác. Hệ thống cung cấp nhiều lựa chọn để bạn lưu trữ dữ liệu ứng dụng:

- App-specific files: Lưu các file chỉ dành riêng cho ứng dụng của bạn. Ví dụ: file dữ liệu có cấu trúc (JSON), file văn bản thuần, file media.
- Shared files: Lưu các file mà ứng dụng của bạn dự định chia sẻ với ứng dụng khác, chẳng hạn như file media hoặc tài liệu.
- Preferences: Lưu dữ liệu riêng tư, đơn giản dưới dạng cặp key-value. Ví dụ: `SharedPreferences` dùng để lưu dữ liệu key-value.
- Databases: Lưu dữ liệu có cấu trúc trong một cơ sở dữ liệu, chỉ riêng ứng dụng của bạn có quyền truy cập.

Trong bài học này, chúng ta sẽ tập trung vào lựa chọn cuối cùng: lưu trữ và truy xuất dữ liệu cục bộ bằng cơ sở dữ liệu.

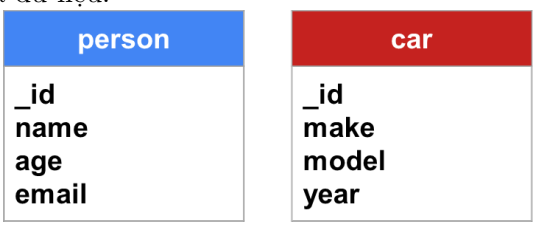
9.1.1. Cơ sở dữ liệu

Một cơ sở dữ liệu (database) là tập hợp dữ liệu có cấu trúc, có thể dễ dàng được truy cập, tìm kiếm và tổ chức. Dữ liệu được lưu trong các bảng (table), và mỗi bảng chứa các trường (field) có liên quan.

Ví dụ: chúng ta có một bảng `person` chứa thông tin về con người. Chúng ta có thể có thêm một bảng `car` chứa thông tin về xe trong ứng dụng. Mỗi bảng có thể có nhiều hàng (row), và mỗi hàng đại diện cho một đối tượng cụ thể. Các cột (column) mô tả loại dữ liệu mà mỗi hàng lưu trữ. Một hàng trong bảng `person` có thể đại diện cho một người, với các thông tin như tên, tuổi và email. Một hàng trong bảng `car` có thể đại diện cho một chiếc xe, với các thông tin như hãng sản xuất, mẫu xe và năm sản xuất.

Mỗi bảng cũng có một cột `_id`, cung cấp một định danh duy nhất cho mỗi hàng trong bảng (gọi là `primary key`).

Đây được gọi là cơ sở dữ liệu quan hệ (relational database), bởi vì cấu trúc của cơ sở dữ liệu (bao gồm bảng, hàng và cột) cho phép chúng ta thiết lập các mối quan hệ và ràng buộc giữa dữ liệu.



Để tương tác với cơ sở dữ liệu quan hệ, chúng ta sử dụng `SQL` (Structured Query Language), đây là một ngôn ngữ chuyên biệt dành cho cơ sở dữ liệu.

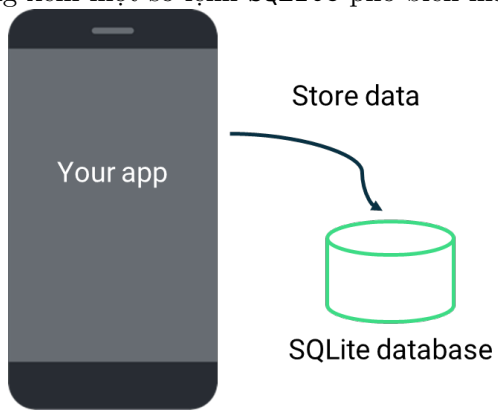
`SQL` có rất nhiều khả năng, nhưng trong phạm vi này, chúng ta sẽ tập trung vào một số chức năng cốt lõi, chẳng hạn như:

- Tạo bảng trong cơ sở dữ liệu.
- Truy vấn (query) dữ liệu.
- Thêm (insert) và cập nhật (update) dữ liệu.
- Xoá (delete) dữ liệu khỏi cơ sở dữ liệu.

Do thiết bị di động bị giới hạn về phần cứng và khả năng tính toán, Android sử dụng `SQLite`, vốn dựa trên chuẩn `SQL`. `SQLite` nhẹ, mã nguồn mở, lý tưởng cho các thiết bị nhúng, và cũng chính là công nghệ bạn sẽ dùng để lưu dữ liệu trong cơ sở dữ liệu của ứng dụng Android.

`SQLite` hỗ trợ hầu hết các tính năng của `SQL`. (Để biết chi tiết những gì không được hỗ trợ, bạn có thể tham khảo đường link tài liệu bên dưới.)

Bây giờ, hãy cùng xem một số lệnh `SQLite` phổ biến mà bạn cần nắm.



Dưới đây là các lệnh ví dụ để thao tác trên bảng cơ sở dữ liệu có tên `colors`, bảng này lưu tên màu và mã màu thập lục phân (hexadecimal color codes).

Chúng ta có thể:

- Chèn (insert) các hàng mới vào bảng `colors`,
- Đọc (read) dữ liệu hiện có từ bảng,
- Cập nhật (update) các hàng,
- Và xoá (delete) các hàng khớp với điều kiện cho trước.

Các thao tác `Create - Read - Update - Delete` này còn được gọi là CRUD operations. Như đã đề cập, đây là những tương tác phổ biến nhất mà bạn sẽ làm việc với cơ sở dữ liệu.

Lưu ý rằng các từ viết hoa trong câu lệnh là các từ khóa (keywords) của `SQLite` (ví dụ: `INSERT`, `UPDATE`, `DELETE`, `VALUES`, `WHERE`, ...), do đó bạn không thể sử dụng chúng làm tên bảng hoặc tên cột. Xem danh sách đầy đủ các từ khóa trong tài liệu đi kèm.

Android cung cấp các API để giúp bạn tạo và quản lý cơ sở dữ liệu `SQLite` trực tiếp trong ứng dụng. Việc tương tác trực tiếp với cơ sở dữ liệu rất mạnh mẽ, nhưng đòi hỏi nhiều thời gian và sự cẩn thận để sử dụng đúng cách và tránh sai sót. Ví dụ, bạn sẽ không có khả năng kiểm tra lỗi truy vấn SQL ngay tại thời điểm biên dịch (compile-time verification). Bạn cũng phải viết rất nhiều `boilerplate code` để chuyển đổi giữa truy vấn SQL và các đối tượng dữ liệu.

May mắn thay, Android Jetpack đã giới thiệu thư viện Room persistence, cung cấp một lớp trừu tượng (abstraction layer) cho phép bạn dễ dàng hơn khi làm việc với cơ sở dữ liệu trong ứng dụng.

9.1.2. Thư viện lưu trữ Room

Thư viện Room cung cấp một lớp trừu tượng trên `SQLite`, cho phép truy cập cơ sở dữ liệu một cách mạnh mẽ và đáng tin cậy hơn, đồng thời vẫn tận dụng đầy đủ sức mạnh của `SQLite`.

Để sử dụng thư viện Room, bạn cần thêm các dependency sau vào file `build.gradle` của ứng dụng.

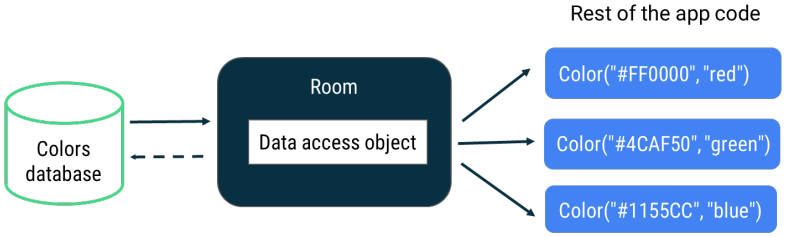
Phiên bản ổn định hiện tại tại thời điểm viết là 2.2.5. Bạn sẽ định nghĩa giá trị này (hoặc một phiên bản ổn định mới hơn) trong biến `room_version`. Lưu ý: Đối với các ứng dụng viết bằng Kotlin, hãy sử dụng `kapt` thay cho `annotationProcessor`, và thêm plugin `kotlin-kapt`:

```
dependencies {
  implementation "androidx.room:room-runtime:$room_version"
  kapt "androidx.room:room-compiler:$room_version"

  // Kotlin Extensions and Coroutines support for Room
  implementation "androidx.room:room-ktx:$room_version"

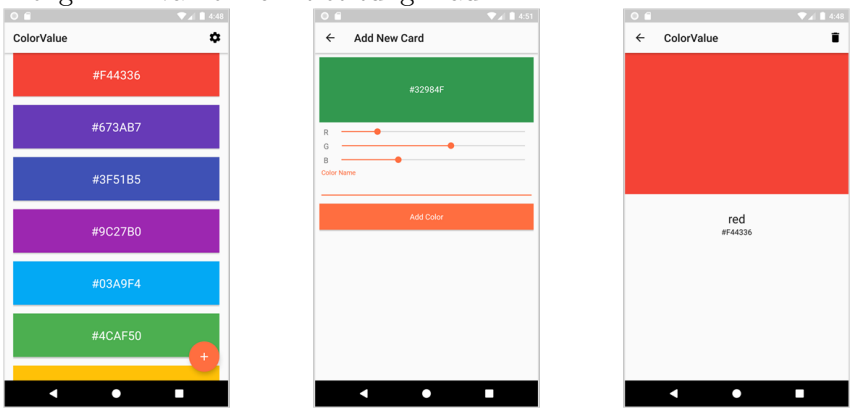
  // Test helpers
  testImplementation "androidx.room:room-testing:$room_version"
}
```

`Room` là một thư viện Object Relational Mapping (ORM), cho phép chuyển đổi dữ liệu từ biểu diễn trong cơ sở dữ liệu thành các đối tượng mà chúng ta có thể thao tác trực tiếp trong code của ứng dụng. Ngược lại, `Room` cũng có thể thực hiện quá trình đảo ngược, tức là đẩy dữ liệu từ các đối tượng trở lại cơ sở dữ liệu.



Đây là ứng dụng mẫu ColorValue mà chúng ta sẽ theo dõi trong phần còn lại

của bài học. Ứng dụng này hiển thị một danh sách các màu, cho phép bạn thêm màu của riêng mình và xem chi tiết từng màu.



Có ba thành phần chính trong `Room`:

- **Entity**: Đại diện cho một bảng trong cơ sở dữ liệu.
- **DAO (Data Access Object)**: Chứa các phương thức dùng để truy cập cơ sở dữ liệu.
- **Database**: Chứa `database holder`, là điểm truy cập chính đến dữ liệu quan hệ được lưu trữ trong ứng dụng.

Không có giới hạn về số lượng lớp `Entity` hoặc `DAO`, nhưng chúng phải duy nhất trong cơ sở dữ liệu.

Trong ứng dụng ColorValue, chúng ta sẽ khai báo ba lớp:

- `Color`
- `ColorDao`
- `ColorDatabase`

Trước tiên, định nghĩa một lớp `Color` để lưu dữ liệu cho một màu cụ thể, bao gồm tên màu và giá trị mã màu hex. Chúng ta có thể khai báo nó như một `data class`, nhưng điều này không bắt buộc.

Tuy nhiên, để các lớp Kotlin có ý nghĩa với cơ sở dữ liệu `Room`, bạn cần phải sử dụng `annotation`.

```
data class Color {
    val hex: String,
```

```
    val name: String
}
```

Nói chung, `annotation` là các ghi chú dùng để giải thích hoặc làm nổi bật những từ hoặc đoạn văn trong một văn bản. Tương tự, trong các ngôn ngữ lập trình, `annotation` gắn metadata vào code của bạn và có thể cung cấp thêm thông tin cho trình biên dịch. (Ví dụ: trong Java, bạn có thể quen thuộc với `@Override` khi ghi đè phương thức của lớp cha trong lớp con.)

`Room` sử dụng thông tin này để tự động sinh code cho bạn.

Bạn có thể tìm thấy danh sách đầy đủ các `annotation` trong phần `Room package summary reference`.

Hãy cùng xem các `annotation` được dùng với `entity`, tức là các lớp ánh xạ tới bảng `SQLite`. `Entity` thường được cài đặt dưới dạng `data class`.

Thêm `@Entity` vào lớp. Dùng `@PrimaryKey` để đánh dấu một cột là `primary key` của bảng (trường trong bản ghi cơ sở dữ liệu xác định tính duy nhất). Trong các cơ sở dữ liệu đơn giản, thường sử dụng số nguyên `auto-increment`.

Theo mặc định, `Room` sử dụng tên thuộc tính để tạo cột trong cơ sở dữ liệu. Bạn có thể thay đổi điều này bằng `@ColumnInfo`.

Hãy thêm `@Entity` vào lớp `Color`. Theo mặc định, `Room` sử dụng tên lớp làm tên bảng trong cơ sở dữ liệu. Nếu bạn muốn bảng có tên khác, hãy thiết lập thuộc tính `tableName` của `@Entity`. Trong ví dụ này, tên bảng là `"colors"`.

Chúng ta cũng thêm một trường `_id` vì mỗi `entity` phải có ít nhất một trường được định nghĩa là `primary key`. Đánh dấu trường này với `@PrimaryKey`. Nếu bạn muốn `Room` tự động gán ID cho các `entity`, hãy đặt thuộc tính `autoGenerate` của `@PrimaryKey` thành `true`.

Chúng ta muốn cột được gọi là `"hex_color"`, vì vậy ta thêm `@ColumnInfo` vào trường `hex` và truyền vào tên cột mong muốn.

```
@Entity(tableName = "colors")
data class Color {
    @PrimaryKey(autoGenerate = true) val _id: Int,
    @ColumnInfo(name = "hex_color") val hex: String,
    val name: String
}
```

Để truy cập dữ liệu của ứng dụng bằng thư viện `Room`, bạn sẽ làm việc với các `data access objects` (viết tắt là `DAO`). Hãy định nghĩa các tương tác với cơ sở dữ liệu mà bạn muốn có ngay trong lớp `DAO`, thay vì dùng `query builder` hoặc truy vấn trực tiếp.

`Room` cũng kiểm tra (xác minh) tất cả các truy vấn trong các lớp `Dao` khi ứng dụng được biên dịch, vì vậy nếu có vấn đề trong một truy vấn, bạn sẽ được thông báo ngay lập tức. Bên cạnh đó, `DAO` giúp bạn dễ dàng `mock` việc truy cập cơ sở dữ liệu khi kiểm thử ứng dụng.

`@Dao` đánh dấu interface của chúng ta là một `DAO`. Lưu ý rằng nó chứa các phương thức được đánh dấu với `@Query`, `@Insert`, `@Update`, và `@Delete`, chính là các thao tác mà chúng ta muốn thực hiện trên cơ sở dữ liệu `colors`.

```
@Dao
interface ColorDao {
    @Query("SELECT * FROM colors")
    fun getAll(): Array<Color>
    @Insert
    fun insert(vararg color: Color)
    @Update
    fun update(color: Color)
    @Delete
    fun delete(color: Color)
```

Với mỗi hàm trong `DAO` thực hiện truy vấn cơ sở dữ liệu, chúng ta cần một `@Query` để cho cơ sở dữ liệu biết cần lấy gì. Tên bảng (`colors`) và tên cột (`name`, `hex_color`) phải tương ứng với những gì đã khai báo trong lớp `Color`.

Câu truy vấn sử dụng một `domain-specific-language (DSL)` rất gần với lệnh `SQL`. Một điểm khác biệt quan trọng là cách bạn chỉ định tham số trong truy vấn. Bất kỳ tham số nào hàm cần truyền vào truy vấn phải tham chiếu đúng tên tham số của hàm. Ví dụ: trong `getColorByHex()`, nếu chúng ta truyền tham số `hex` vào truy vấn, hãy tham chiếu đến tham số này bằng `:hex`.

`Chú ý`: Bạn có thể tạo các hằng số (constant) cho tên bảng và tên cột.

```
@Dao
interface ColorDao {
    @Query("SELECT * FROM colors")
    fun getAll(): Array<Color>
```

```
    @Query("SELECT * FROM colors WHERE name = :name")
    fun getColorByName(name: String): LiveData<Color>

    @Query("SELECT * FROM colors WHERE hex_color = :hex")
    fun getColorByHex(hex: String): LiveData<Color>
```

Khi bạn tạo một phương thức `DAO` và đánh dấu nó với `@Insert`, `Room` sẽ sinh ra một phần cài đặt để chèn tất cả các tham số vào cơ sở dữ liệu trong một `transaction` duy nhất.

Để cho phép một hàm nhận số lượng đối số thay đổi, hãy đánh dấu tham số (thường là tham số cuối cùng) bằng từ khóa `vararg`.

```
@Dao
interface ColorDao {
    ...

    @Insert
    fun insert(vararg color: Color)

    ...
}
```

`@Update` đánh dấu một phương thức trong lớp được gắn `@Dao` là phương thức cập nhật, dùng để cập nhật `Color` đã chỉ định.

```
@Dao
interface ColorDao {
    ...

    @Update
    fun update(color: Color)

    ...
}
```

`@Delete` đánh dấu một phương thức trong lớp được gắn `@Dao` là phương thức xoá, dùng để xoá `Color` đã chỉ định khỏi cơ sở dữ liệu.

```
@Dao
interface ColorDao {
    ...

    @Delete
    fun delete(color: Color)

    ...
}
```

Cuối cùng, sau khi đã khai báo `DAO`, chúng ta có thể tạo cơ sở dữ liệu `Room`. Đánh dấu lớp với `@Database` và đưa vào danh sách các `entity`. Thuộc tính `entities` của `@Database` khai báo các đối tượng sẽ được lưu trong cơ sở dữ liệu này.

Khai báo lớp dưới dạng `abstract class` (`ColorDatabase`) kế thừa từ `RoomDatabase`. Bên trong lớp, bao gồm một phương thức `abstract` không có tham số và trả về lớp đã được đánh dấu với `@Dao`.

Bạn sẽ nhận được một cài đặt của lớp `ColorDatabase` thông qua `Room.databaseBuilder` hoặc `Room.inMemoryDatabaseBuilder`.

9.1.3. Ví dụ Room

Đây là cách lớp `ColorDatabase` được định nghĩa. Bên trong nó, chúng ta khai báo một `singleton` để lấy một instance của cơ sở dữ liệu. Nếu ứng dụng của bạn chạy trong một tiến trình duy nhất, bạn nên tuân theo mẫu thiết kế `singleton` khi khởi tạo đối tượng `Database`. Lý do là bởi vì mỗi instance của `RoomDatabase` khá tốn kém, và hiếm khi bạn cần truy cập nhiều instance trong cùng một tiến trình.

`@Volatile`: Lý do chúng ta khai báo biến `INSTANCE` là `volatile` là để giá trị của nó không bao giờ bị cache, và mọi thao tác đọc/ghi đều được thực hiện trực tiếp trong bộ nhớ chính. Điều này đảm bảo tính toàn vẹn dữ liệu (tức là bất kỳ thay đổi nào đối với instance của cơ sở dữ liệu) sẽ hiển thị ngay lập tức cho tất cả các thread khác, và ngăn chặn, ví dụ, hai thread cùng cập nhật một entity trong cache.

Đối với phần cài đặt phương thức `getInstance()` trả về một `ColorDatabase`, hãy xem ở slide tiếp theo.

```
@Database(entities = [Color::class], version = 1)
```

```
abstract class ColorDatabase : RoomDatabase() {
    abstract fun colorDao(): ColorDao
    companion object {
        @Volatile
        private var INSTANCE: ColorDatabase? = null
        fun getInstance(context: Context): ColorDatabase {
            ...
        }
    }
    ...
```

Chúng ta sử dụng `Room.databaseBuilder()` để tạo một cơ sở dữ liệu, với `application context`, lớp `ColorDatabase`, và tên cơ sở dữ liệu.

`Synchronized`: Nhiều thread có thể đồng thời yêu cầu một instance cơ sở dữ liệu, dẫn đến việc có từ hai instance trở lên thay vì chỉ một. Mặc dù điều này khó xảy ra trong ứng dụng mẫu này, nhưng hoàn toàn có thể xảy ra với các ứng dụng phức tạp hơn. Bao bọc đoạn code khởi tạo cơ sở dữ liệu bằng `synchronized` sẽ đảm bảo rằng chỉ có một thread được thực thi tại một thời điểm trong block này, và cơ sở dữ liệu chỉ được khởi tạo duy nhất một lần.

Nếu việc mất dữ liệu hiện tại là chấp nhận được khi thiếu đường dẫn migration, hãy gọi phương thức `fallbackToDestructiveMigration()` trong quá trình tạo cơ sở dữ liệu. Phương thức này báo cho `Room` rằng khi cần thực hiện migration gia tăng (incremental migration) nhưng không có đường dẫn migration được định nghĩa, thì các bảng trong cơ sở dữ liệu của ứng dụng sẽ được tạo lại một cách phá huỷ (destructive recreate).

```
fun getInstance(context: Context): ColorDatabase {
    return INSTANCE ?: synchronized(this) {
        INSTANCE ?: Room.databaseBuilder(
            context.applicationContext,
            ColorDatabase::class.java, "color_database"
        )
        .fallbackToDestructiveMigration()
        .build()
        .also { INSTANCE = it }
    }
}
```

Bây giờ hãy thêm một ít dữ liệu vào cơ sở dữ liệu.

```
val colorDao = ColorDatabase.getInstance(application).colorDao()
val newColor = Color(hex = "#6200EE", name = "purple")
colorDao.insert(newColor)
```

Đầu tiên, tạo một instance của `DAO` tên là `colorDao` bằng cách sử dụng phương thức `getInstance()`, đảm bảo rằng nó cung cấp cho chúng ta một instance `singleton`.

Tiếp theo, tạo một đối tượng `Color` mới là `newColor`, và thêm nó vào instance `DAO` thông qua phương thức `insert()`. Điều này sẽ thêm `newColor` vào trong cơ sở dữ liệu của bạn.

Bây giờ bạn có thể sử dụng đối tượng `DAO` để `insert`, `update`, `search` hoặc `query` cơ sở dữ liệu.

9.2. Lập trình bất đồng bộ

Nếu bạn cần thực hiện một số tác vụ chạy lâu, bạn không nên làm điều đó trên `main thread`, nếu không ứng dụng của bạn có thể trở nên không phản hồi với người dùng. Điều này dẫn đến nhu cầu phải thực hiện công việc một cách không đồng bộ

- Tải thông tin từ mạng
- Đồng bộ với server
- Ghi dữ liệu ra file
- Tính toán cần nhiều thời gian
- Thao tác với CSDL

Nhu cầu lập trình `async` xuất phát từ ba yếu tố chính:

- **Thời gian giới hạn để chạy tác vụ** Các frame UI phải được render trong vòng <16ms để đạt tốc độ làm mới 60 fps. Nếu việc render UI chậm hơn, hệ thống buộc phải bỏ qua frame, gây ra hiện tượng giật được gọi là `"jank"`.
- **Một số tác vụ cần nhiều thời gian để hoàn tất** Dù có thể tối ưu code, nhưng vẫn có những tác vụ không tránh khỏi việc xử lý lâu.

- **Cần kiểm soát cả cách thức và nơi mà tác vụ chạy** Ví dụ: chúng ta cần kiểm soát điều kiện mà trong đó một tác vụ lan truyền lỗi hoặc dừng thực thi.

Một trong những cách thực hiện công việc `asynchronously` là dùng `thread`. `Threading` tạo ra một đơn vị thực thi để chạy phần code tách khỏi `main thread`. Tuy nhiên, việc khởi tạo `thread` có chi phí vì hệ điều hành phải tạo và cấp phát bộ nhớ cho `thread` đó.

Một vấn đề khác với `thread` là nhiều công việc cần tính `concurrent` nhưng không (hoặc không thể) `parallel`. Ví dụ: khi tải một đơn hàng từ cơ sở dữ liệu, có thể cần 3 yêu cầu để lấy đủ dữ liệu. Dù bạn có thể dùng một `thread` cho mỗi yêu cầu, nhưng nếu yêu cầu 2 và 3 phụ thuộc yêu cầu 1, thì phần phụ thuộc này phải được bảo toàn — chúng sẽ chặn nhau theo thứ tự.

Chúng ta có thể dùng `callback`, nhưng các `nested callback` trở nên khó quản lý và xử lý lỗi cũng phức tạp.

Có nhiều lựa chọn khác, nhưng mỗi cách đều có nhược điểm và thách thức riêng. Vậy cách khuyến nghị để lập trình `asynchronous` trên `Android` là gì?

9.2.1. Coroutines

`Coroutine` là một mẫu thiết kế `concurrency` mà bạn có thể sử dụng trên `Android` để đơn giản hóa code thực thi `asynchronously`. `Coroutines` được thêm vào `Kotlin` từ phiên bản 1.3, và dựa trên các khái niệm đã được thiết lập từ các ngôn ngữ khác.

Trên `Android`, `coroutines` giúp quản lý các tác vụ chạy lâu dài mà nếu không, chúng có thể chặn `main thread` và khiến ứng dụng của bạn không phản hồi.

Có một số lợi ích quan trọng khi sử dụng `coroutines`:

- `Lightweight`: Bạn có thể chạy nhiều `coroutine` trên một `thread` nhờ hỗ trợ `suspension`, giúp không chặn `thread` nơi `coroutine` đang chạy.

- `Fewer memory leaks`: `Coroutines` có thể sử dụng `structured concurrency` để khởi chạy `coroutine` trong phạm vi cụ thể của tác vụ mà ta đang thực hiện.

- `Built-in cancellation support`: Việc huỷ bỏ (`cancellation`) được truyền tự động trong hệ thống phân cấp các `coroutine`, cho phép dừng các tác vụ không còn cần thiết.

- `Jetpack integration`: Nhiều thư viện `Jetpack` bao gồm các extension hỗ trợ đầy đủ cho `coroutines`. Một số thư viện cũng cung cấp `coroutine scope` riêng mà bạn có thể dùng cho `structured concurrency`.

Dùng từ khóa `suspend` để khai báo một hàm có thể được sử dụng trong `coroutine`. Hàm này chỉ có thể được gọi từ bên trong một `coroutine` hoặc từ một hàm `suspend` khác.

Các hàm `suspend` được hiển thị trong `Android Studio` với một biểu tượng đặc trưng ở lề trái (left gutter).

Khi một `coroutine` gọi một hàm được đánh dấu với `suspend`, thay vì chặn (`block`) cho đến khi hàm đó trả về như một lời gọi hàm bình thường, nó sẽ `suspend` việc thực thi cho đến khi kết quả sẵn sàng, sau đó tiếp tục (`resume`) từ vị trí trước đó cùng với kết quả nhận được.

Trong khi đang `suspend` để chờ kết quả, `coroutine` sẽ giải phóng `thread` mà nó đang chạy, để các hàm hoặc `coroutine` khác có thể được thực thi.

`Suspending` tiết kiệm bộ nhớ hơn so với `blocking`, đồng thời hỗ trợ nhiều tác vụ chạy đồng thời.

`suspend` và `resume` phối hợp với nhau để thay thế cho `callback`. Trong slide tiếp theo, chúng ta sẽ xem một ví dụ đơn giản về cách `suspend` và `resume` hoạt động cùng nhau.

- `suspend`
  Tạm dừng việc thực thi của `coroutine` hiện tại và lưu lại các biến cục bộ.

- `resume`
  Tự động tải lại trạng thái đã lưu và tiếp tục thực thi từ điểm mà code đã bị `suspend`.

`Room` hỗ trợ `coroutines`, vì vậy hãy thêm từ khóa `suspend` vào các phương thức trong `DAO` để đảm bảo rằng chúng chỉ có thể được gọi từ bên trong một `coroutine` hoặc từ một hàm `suspend` khác.

```
@Dao
interface ColorDao {

    @Query("SELECT * FROM colors")
    suspend fun getAll(): Array<Color>
```

```
@Insert
suspend fun insert(vararg color: Color)
@Update
suspend fun update(color: Color)
@Delete
suspend fun delete(color: Color)
```

Sử dụng `suspend` không có nghĩa là Kotlin sẽ chạy hàm trên một `background thread`. `Coroutines` có thể chạy trên `main thread`. Ví dụ: bạn có thể khởi chạy một `coroutine` trên `main thread` để phản hồi một sự kiện UI, và nếu không thực hiện một tác vụ chạy lâu, thì UI có thể được cập nhật ngay sau đó.

Kotlin có một số `dispatcher` mà bạn có thể sử dụng để chỉ định nơi `coroutine` sẽ chạy, tùy thuộc vào loại tác vụ mà bạn đang thực hiện. Chúng bao gồm: `Dispatchers.Main`, `Dispatchers.IO`, và `Dispatchers.Default`.

Dispatcher	**Mô tả**	**Ví dụ**
`Dispatchers.Main`	Chạy trên `main thread`, dành cho các tác vụ liên quan tới UI	Cập nhật UI, xử lý sự kiện người dùng
`Dispatchers.IO`	Tối ưu cho các tác vụ nhập/xuất (`I/O`), chạy trên một nhóm thread được thiết kế cho thao tác chặn (blocking)	Đọc/ghi cơ sở dữ liệu, truy cập mạng, đọc/ghi file
`Dispatchers.Default`	Tối ưu cho các tác vụ tính toán nặng (`CPU-intensive`), chạy trên nhóm thread dùng chung	Xử lý dữ liệu lớn, tính toán phức tạp, thao tác trên bộ sưu tập

Từ khóa `withContext` cho phép bạn chỉ định `dispatcher` mà đoạn code bên trong sẽ chạy. Ví dụ, `withContext(Dispatchers.IO)` sẽ chuyển việc thực thi `coroutine` sang một `I/O thread`.

Vì `withContext` bản thân nó là một hàm `suspend`, nên hàm gọi nó vẫn `main-safe` và có thể cập nhật `UI` khi cần thiết. `Coroutine` trên `main thread` sẽ được `resume` ngay khi khối `withContext` hoàn tất.

```
suspend fun get(url: String) {

 // Start on Dispatchers.Main

    withContext(Dispatchers.IO) {
```

```
        // Switches to Dispatchers.IO
        // Perform blocking network IO here
    }

    // Returns to Dispatchers.Main
}
```

Kotlin không cho phép bạn khởi chạy một coroutine mới mà không có CoroutineScope. CoroutineScope theo dõi tất cả các coroutine của bạn, và có thể huỷ toàn bộ các coroutine được khởi chạy trong nó. Nó không trực tiếp thực thi coroutine, mà chỉ đảm bảo bạn không bị mất quyền quản lý chúng.

Theo mặc định, coroutines chạy trong GlobalScope. Các thư viện Jetpack định nghĩa sẵn một số scope mà bạn có thể sử dụng trong ứng dụng của mình, chẳng hạn như viewModelScope và lifecycleScope.

Thêm các dependency được liệt kê trong đường dẫn bên dưới để có thể sử dụng các scope này trong code của bạn.

Hãy cùng xem cách bắt đầu một coroutine mới để có thể gọi các hàm suspend.

launch tạo một coroutine job mới trong phạm vi đã cho và không trả về giá trị. async cho phép bạn khởi chạy một coroutine và trả về giá trị với từ khóa await. Trong bài học này, chúng ta sẽ tập trung vào launch.

Ví dụ: khối launch khởi chạy một coroutine mới trong default coroutine scope với default dispatcher. Bên trong nó có thể gọi các hàm suspend như fetchDocs.

```
fun loadUI() {
    launch {
        fetchDocs()
    }
}
```

launch - không cần kết quả trả về
async - có thể trả về kết quả

Một viewModelScope được định nghĩa cho mỗi ViewModel trong ứng dụng. Bạn có thể truy cập CoroutineScope của ViewModel thông qua thuộc tính viewModelScope (miễn là bạn đã thêm dependency KTX phù hợp).

Bất kỳ `coroutine` nào được khởi chạy trong scope này sẽ tự động bị huỷ nếu `ViewModel` bị `cleared`. `Coroutines` đặc biệt hữu ích trong trường hợp công việc chỉ cần thực hiện khi `ViewModel` còn hoạt động.

Ví dụ: nếu bạn đang tải dữ liệu cho một layout, bạn nên ràng buộc công việc đó với `ViewModel`, để khi `ViewModel` bị xóa, công việc cũng bị huỷ tự động, tránh tiêu tốn tài nguyên không cần thiết.

```
class MyViewModel: ViewModel() {

    init {
        viewModelScope.launch {
            // Coroutine that will be canceled
            // when the ViewModel is cleared
        }
    }
    ...
```

Hãy cùng ghép tất cả các khái niệm đã học lại trong ứng dụng `ColorValue`.

Từ `ViewModel`, chúng ta có thể tương tác với `DAO`. Đối với lớp `ColorViewModel`, chúng ta kế thừa từ `AndroidViewModel` thay vì `ViewModel`, bởi vì phiên bản này có thể tham chiếu đến `application context` để khởi tạo cơ sở dữ liệu theo cách `lifecycle-aware`.

`ColorViewModel` có một hàm `save` khởi chạy một `coroutine` mới bằng `viewModelScope`, hàm này sẽ chèn màu mới vào cơ sở dữ liệu thông qua `DAO`.

```
class ColorViewModel(val dao: ColorDao, application: Application)
    : AndroidViewModel(application) {

fun save(color: Color) {
    viewModelScope.launch {
        colorDao.insert(color)
    }
}

...
```

9.3. Kiểm thử với CSDL

Thêm các thành phần phụ thuộc vào Gradle

```
android {
    defaultConfig {
        ...
        testInstrumentationRunner "androidx.test.runner
         .AndroidJUnitRunner"
        testInstrumentationRunnerArguments clearPackageData: 'true'
    }
}
dependencies {
    testImplementation 'junit:junit:4.12'
    androidTestImplementation 'androidx.test.ext:junit:1.1.0'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.1.1'
}
```

`AndroidJUnit4 runner` cung cấp cho chúng ta các tham chiếu và `context` phù hợp để chạy code đặc thù của `Android`.

`@Before` và `@After` sẽ chạy code ngay trước và ngay sau mỗi hàm kiểm thử được đánh dấu với `@Test`.

Điều đầu tiên chúng ta cần làm trong lớp kiểm thử là khai báo các đối tượng `DAO` và `database`. Chúng ta cũng đã tạo ba instance của `Color` để sử dụng về sau.

```
@RunWith(AndroidJUnit4::class)
class DatabaseTest {

    private lateinit val colorDao: ColorDao
    private lateinit val db: ColorDatabase

    private val red = Color(hex = "#FF0000", name = "red")
    private val green = Color(hex = "#00FF00", name = "green")
    private val blue = Color(hex = "#0000FF", name = "blue")

    ...
```

Trong hàm được đánh dấu với `@Before`, chúng ta tạo `database` và instance `DAO`. Hàm này sẽ được chạy trước mỗi test case trong lớp.

Để giữ cho cơ sở dữ liệu production (chứa dữ liệu thực của người dùng) không bị ảnh hưởng, chúng ta khởi tạo một `in-memory database` cho mục đích kiểm thử, và nó sẽ bị huỷ bỏ khi không còn sử dụng.

Hàm `closeDb()`, được đánh dấu với `@After`, thực hiện việc dọn dẹp sau khi mỗi test hoàn tất.

```
@Before
fun createDb() {
    val context: Context = ApplicationProvider.getApplicationContext()
    db = Room.inMemoryDatabaseBuilder(context, ColorDatabase::class.java)
        .allowMainThreadQueries()
        .build()
    colorDao = db.colorDao()
}
@After
@Throws(IOException::class)
fun closeDb() = db.close()
```

Sau khi hoàn tất các bước chuẩn bị, chúng ta có thể bắt đầu kiểm thử cơ sở dữ liệu. Trong đoạn code này, chúng ta chèn ba `Color` đã tạo vào cơ sở dữ liệu thông qua `DAO` và thử truy xuất lại chúng.

```
 @Test
    @Throws(Exception::class)
    fun insertAndRetrieve() {
        colorDao.insert(red, green, blue)
        val colors = colorDao.getAll()
        assert(colors.size == 3)
    }
```

9.4. Tóm tắt chương

Trong chương này, bạn đã học cách:

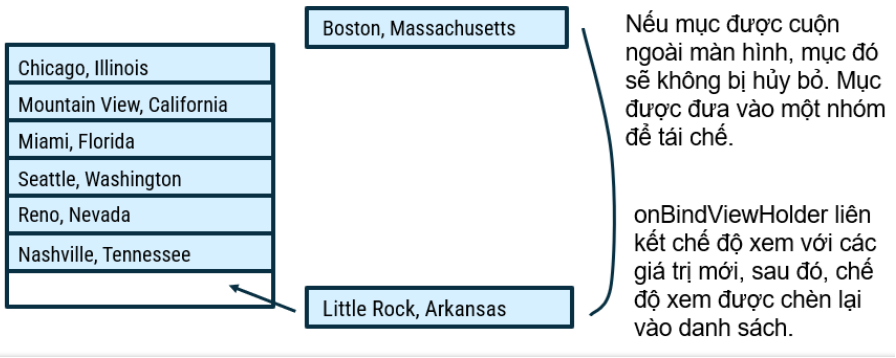
- Thiết lập và cấu hình cơ sở dữ liệu bằng thư viện `Room`

- Sử dụng `coroutines` để lập trình bất đồng bộ (`asynchronous programming`)
- Kết hợp `coroutines` với `Room`
- Kiểm thử (`test`) cơ sở dữ liệu

Chương 10
Sử dụng nâng cao RecyclerView

10.1. Nhắc lại về RecyclerView

- Điều khiển để hiển thị dữ liệu dạng danh sách
- "Tái chế" (tái sử dụng) các chế độ xem mục để cuộn hiệu quả hơn
- Có thể chỉ định bố cục mục danh sách cho từng mục trong tập dữ liệu
- Hỗ trợ hoạt ảnh và quá trình chuyển đổi



Trong bài học 5, chúng ta đã học rằng một adapter tuỳ chỉnh sẽ kế thừa từ RecyclerView.Adapter và ghi đè ba hàm:

- getItemCount
- onCreateViewHolder
- onBindViewHolder

Một ViewHolder đại diện cho một phần tử (item) trong danh sách và giữ tham chiếu đến tất cả các view bên trong layout đó.

Như sơ đồ đã minh hoạ, khi bạn có một số lượng lớn item trong danh sách (nhiều hơn số có thể hiển thị trên màn hình cùng lúc), chúng ta không cần tạo view cho những item chưa được cuộn tới.

Phương thức onCreateViewHolder() của adapter được gọi để tạo các view holder tương ứng với số lượng item có thể hiển thị trên màn hình tại một thời điểm. Sau lần

tạo ban đầu, khi bạn cuộn, hệ thống sẽ loại bỏ các view item ra khỏi màn hình khỏi hierarchy, và gọi onBindViewHolder() trên adapter để "tái sử dụng" các view item đó. Giá trị bên trong view sẽ được cập nhật để phản ánh dữ liệu của item mới (chuẩn bị xuất hiện trên màn hình). Sau đó, view item được thêm trở lại vào view hierarchy.



Ví dụ để xây dựng danh sách hiển thị các số như ảnh trên. Ta cần xây dựng lớp Adapter như sau:

```
class NumberListAdapter(var data: List<Int>):
        RecyclerView.Adapter<NumberListAdapter.IntViewHolder>() {
   class IntViewHolder(val row: View): RecyclerView.ViewHolder(row) {
       val textView = row.findViewById<TextView>(R.id.number)
   }

override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
    IntViewHolder {
   val layout = LayoutInflater.from(parent.context)
   .inflate(R.layout.item_view, parent, false)
   return IntViewHolder(layout)
}
```

```
override fun onBindViewHolder(holder: IntViewHolder, position: Int) {
   holder.textView.text = data.get(position).toString()
}
```

Chúng ta sẽ ghi đè onCreateViewHolder() để inflate layout và gọi ViewHolder, đồng thời ghi đè onBindViewHolder() để gán (bind) dữ liệu.

Trong MainActivity, chúng ta thiết lập RecyclerView. Lưu ý rằng chúng ta khởi tạo NumberListAdapter bằng cách tạo một danh sách các số từ 0 đến 100. Sau đó, chúng ta gán adapter này cho RecyclerView.

```
override fun onCreate(savedInstanceState: Bundle?) {
   super.onCreate(savedInstanceState)
   setContentView(R.layout.activity_main)

   val rv: RecyclerView = findViewById(R.id.rv)
   rv.layoutManager = LinearLayoutManager(this)

   rv.adapter = NumberListAdapter(IntRange(0,100).toList())
}
```

Cũng giống như bất kỳ view nào khác, chúng ta có thể thêm click listener cho toàn bộ item view trong danh sách hoặc cho một view con bên trong nó. Bởi vì các view holder được tái sử dụng, chúng ta chỉ cần thiết lập click listener một lần khi ViewHolder được tạo, chứ không phải mỗi lần nó được bind (hoặc re-bind). Khi một item được click, chúng ta có thể thực hiện một hành động, chẳng hạn như hiển thị thông báo Toast với giá trị lấy từ TextView.

Để bắt sự kiện click vào các mục trong danh sách. Ta sẽ không bắt sự kiện ở View, không bắt trong MainActivity mà bắt trong Adapter của RecyclerView. Trong file: NumberListAdapter.kt, cài đặt hàm sau:

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
IntViewHolder{
val layout = LayoutInflater.from(parent.context)
.inflate(R.layout.item_view, parent, false)
    val holder = IntViewHolder(layout)
    holder.row.setOnClickListener {
        // Do something on click
    }
```

```
    return holder
}
```

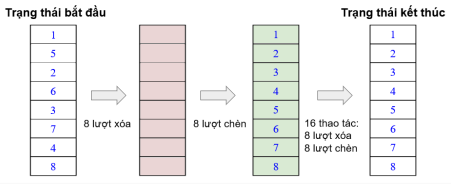
ListAdapter

- RecyclerView.Adapter
  - Hủy bỏ dữ liệu giao diện người dùng trong mỗi lần cập nhật
  - Có thể tốn kém và lãng phí tài nguyên
- ListAdapter
  - Tính toán sự khác biệt giữa nội dung đang hiển thị và nội dung cần hiển thị
  - Các thay đổi được tính toán trên một luồng background, không ảnh hưởng đến luồng UI

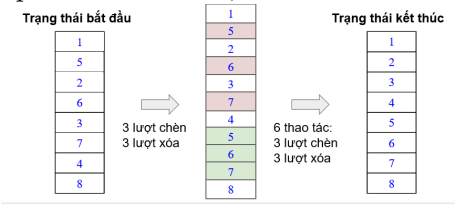
Đối với Adapter kế thừa trực tiếp từ RecyclerView.Adapter, mỗi khi cần cập nhật dữ liệu nguồn, chúng ta sẽ loại bỏ toàn bộ danh sách dữ liệu và tạo lại danh sách đó. Việc này khá tốn kém và lãng phí. Đặc biệt nếu nguồn dữ liệu cũ và nguồn dữ liệu mới là giống nhau hoặc gần như giống nhau. Thay vào đó, hãy cân nhắc dùng ListAdapter.

Ví dụ về việc sắp xếp danh sách sau đây cho thấy hiệu quả của ListAdapter so với RecyclerView.Adapter.

Nếu sử dụng RecyclerView.Adapter, cần 16 lượt thao tác để tạo ra danh sách mới đã được sắp xếp.



Nếu sử dụng ListAdapter chỉ cần 6 lượt thao tác:



Để chuyển đổi sang ListAdapter, bạn cần sửa lại code đã viết ở phần trước (sử dụng RecyclerView.Adapter). Cụ thể trong lớp NumberListAdapter, chúng ta sẽ kế thừa ListAdapter, thay vì RecyclerView.Adapter.

```
    class NumberListAdapter: ListAdapter<Int,
        NumberListAdapter.IntViewHolder>(RowItemDiffCallback()) {

    class IntViewHolder(val row: View):RecyclerView.ViewHolder(row) {
        val textView = row.findViewById<TextView>(R.id.number)
    }

    ...
```

Giờ ta thấy cần phải truyền vào một đối tượng mới tên là RowItemDiffCallback(); mục đích của hàm này để báo cho ListAdapter rằng các mục trong danh sách mới và cũ có thay đổi không.

DiffUtil.ItemCallback xác định một callback để tính toán sự khác biệt giữa hai phần tử không-null trong danh sách. Nó có hai hàm trừu tượng cần được ghi đè: areContentsTheSame() và areItemsTheSame(). Mặc dù hai phương thức này có vẻ giống nhau, nhưng việc có cả hai mang lại cho bạn nhiều quyền kiểm soát hơn về cách dữ liệu được xử lý. Ví dụ, bạn có thể có một tập dữ liệu trong đó hai item trong danh sách trông giống hệt nhau đối với người dùng, nhưng thực tế lại đại diện cho hai đối tượng khác nhau trong dữ liệu nguồn.

Lớp RowItemDiffCallback được định nghĩa như sau:

```
class RowItemDiffCallback : DiffUtil.ItemCallback<Int>() {

   override fun areItemsTheSame(oldItem: Int, newItem: Int): Boolean {
       return oldItem == newItem
   }

   override fun areContentsTheSame(oldItem: Int, newItem: Int): Boolean {
       return oldItem == newItem
   }
}
```

Cụ thể hơn, areItemsTheSame() được gọi để kiểm tra xem oldItem và newItem có đại diện cho cùng một đối tượng hay không. Ví dụ, nếu các item của bạn có ID duy nhất, phương thức này nên kiểm tra sự bằng nhau của các ID đó.

areContentsTheSame() được gọi để kiểm tra xem oldItem và newItem có cùng dữ liệu hay không. Khi DiffUtil được sử dụng với RecyclerView.Adapter, bạn nên trả

về kết quả cho biết liệu hai item có cùng biểu diễn trực quan (visual representation) hay không.

Trong trường hợp danh sách của chúng ta được xây dựng từ một danh sách số nguyên (integers), cả hai hàm areItemsTheSame() và areContentsTheSame() đều có cùng một đoạn mã.

10.2. Liên kết nâng cao (Advanced binding)

Trong phần tiếp theo, chúng ta sẽ tìm hiểu cách triển khai Data Binding với RecyclerView Adapter.

```
class IntViewHolder private constructor(val binding: ItemViewBinding):
        RecyclerView.ViewHolder(binding.root) {
    companion object {
        fun from(parent: ViewGroup): IntViewHolder {
            val layoutInflater = LayoutInflater.from(parent.context)
            val binding = ItemViewBinding.inflate(layoutInflater,
            return IntViewHolder(binding)
        }
    }
}
```

Giả sử chúng ta đã thêm Data Binding vào file layout của list item. Bây giờ hãy cùng xem các thay đổi cần thực hiện trong NumberListAdapter.

Trong Kotlin, nếu chúng ta muốn một hàm hoặc thuộc tính gắn liền với lớp thay vì các instance của nó, ta khai báo nó trong companion object. Bên trong companion object của lớp IntViewHolder, định nghĩa một hàm tên là $from$ để tạo ViewHolder. Chúng ta sẽ sử dụng hàm này sau. Hàm này sẽ inflate item view bằng ItemView-Binding, là một lớp binding được tự động sinh ra từ file layout $item_view.xml$ trong ứng dụng của chúng ta.

Tiếp theo, chúng ta chỉnh sửa code của ViewHolder để nhận một đối tượng ItemViewBinding làm tham số đầu vào, rồi truyền binding.root vào constructor của lớp cha RecyclerView.ViewHolder.

Tại sao chúng ta lại truyền binding làm tham số cho ViewHolder? Bởi vì chúng ta sẽ cần nó để thiết lập các binding adapter ở phần sau của bài học. Binding adapter cho phép chúng ta đóng gói các chức năng tuỳ chỉnh và gắn chúng với một attribute.

Dùng ViewHolder trong ListAdapter

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
        IntViewHolder {
    return IntViewHolder.from(parent)
}

override fun onBindViewHolder(holder: NumberListAdapter.IntViewHolder,
        position: Int) {
    holder.binding.num = getItem(position)
}
```

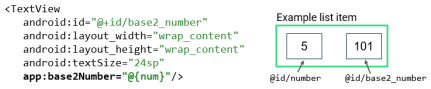
Bây giờ, trong ListAdapter, khi cần tạo một ViewHolder mới, chúng ta có thể sử dụng hàm trợ giúp *from* đã tạo trước đó (IntViewHolder.from(parent)).

Khi cần bind một ViewHolder với dữ liệu mới, chúng ta sẽ cập nhật biến *num* trong binding bằng item hiện tại trong tập dữ liệu. (Biến num đã được khai báo bằng thẻ data trong file *item_view.xml* thông qua Data Binding.)

Binding adapters Trong các bài giảng trước, chúng ta đã tìm hiểu cách sử dụng Transformations trên LiveData để tính toán dữ liệu mới. Một phần công việc đó có thể được chuyển sang file layout XML, bằng cách tạo ra các attribute (thuộc tính) cụ thể được đánh giá (evaluate) tại thời gian chạy.

Bất kỳ attribute nào trong một thẻ view đều có thể được ghi đè bởi một binding adapter. Đây là cách để đóng gói code nhằm xử lý cách một thuộc tính được diễn giải. Bạn có thể ghi đè các attribute trong namespace android hoặc tạo attribute của riêng mình. Bạn có thể chỉ định phương thức nào sẽ được gọi và cung cấp logic binding tùy chỉnh. Các binding adapter nên thực hiện các lời gọi framework thích hợp để gán giá trị cho View.

Để minh họa, chúng ta thêm một TextView trong layout của item trong danh sách như sau:



Thuộc tính tùy chỉnh - custom atrribute

Để minh hoạ điều này, chúng ta đã thêm một TextView thứ hai (có resource ID là *base2_number*) vào layout của item, nhằm hiển thị cùng một số nhưng dưới dạng nhị phân (base 2). Để ngắn gọn, toàn bộ file XML layout của item sẽ không được liệt kê ở đây.

Sơ đồ minh họa một item trong danh sách nếu nó đại diện cho số 5. TextView

thứ hai sẽ hiển thị giá trị nhị phân của nó (101).

Quay lại với phần thảo luận về binding adapter, chúng ta cần một attribute cho View muốn thay đổi — trong trường hợp này là TextView. Binding tùy chỉnh sẽ nằm trong namespace app để tránh xung đột với các attribute do framework cung cấp. Lưu ý rằng chúng ta không cần chỉ định thêm android:text cho TextView vì hàm binding sẽ tự động thực hiện điều đó khi app:base2number được gán.

Thêm binding adapter

```
@BindingAdapter("base2Number")
fun TextView.setBase2Number(item: Int) {
    text = Integer.toBinaryString(item)
}
```

Trong lớp *NumberListAdapter.kt* thêm

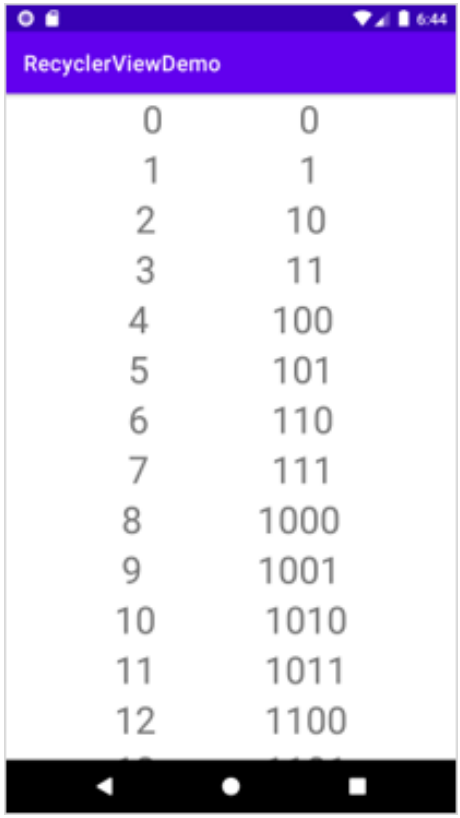
```
override fun onBindViewHolder(holder: NumberListAdapter.IntViewHolder, position:
    holder.binding.num = getItem(position)
    holder.binding.executePendingBindings()
}
```

Chúng ta khai báo một hàm mở rộng (extension function) trên lớp TextView với từ khóa set, theo sau là tên của attribute (setBase2Number), để Android có thể tận dụng cơ chế tự động lựa chọn phương thức. Hàm này cũng phải nhận tham số phù hợp — trong ví dụ này là Int. Hàm setBase2Number() sẽ trực tiếp chỉnh sửa nội dung text của TextView gọi nó.

Chúng ta hoàn toàn có thể chọn một tên hàm tùy chỉnh khác nếu muốn (xem chi tiết trong hướng dẫn Binding Adapters).

Lưu ý: Hãy nhớ rằng hàm mở rộng (extension function) cho phép chúng ta thêm chức năng mới vào một lớp Kotlin mà không cần kế thừa từ lớp đó.

Khi bind ViewHolder, một thói quen tốt là gọi executePendingBindings(). Lệnh này đảm bảo framework thực hiện đầy đủ các tính toán cần thiết để xác định không gian hiển thị cho item, vì việc thay đổi một biến binding không phải lúc nào cũng được áp dụng ngay lập tức. Trong đoạn code ví dụ này — nơi chúng ta chỉ tính toán giá trị nhị phân (base 2) — việc gọi không thực sự cần thiết. Tuy nhiên, nếu bạn có một BindingAdapter xử lý việc tải ảnh hoặc các quá trình tốn thời gian khác, thì việc gọi executePendingBindings() sẽ rất quan trọng.



Sau khi áp dụng binding adapter, ứng dụng sẽ hiển thị như sau: Mỗi item trong danh sách chứa một số nguyên và biểu diễn nhị phân tương ứng ở phía bên phải của nó.

10.3. Nhiều loại item view (Multiple item view types)

Thêm một loại item view mới Dưới đây là các bước cần thiết để hỗ trợ một kiểu (type) khác cho RecyclerView của bạn. Nhiều bước trong số này giống với những gì chúng ta đã làm trước đây, chỉ cần một vài chỉnh sửa nhỏ vào mã hiện có.

- Tạo một file XML layout mới cho list item.
- Chỉnh sửa adapter để lưu/nhận biết kiểu item mới.
- Ghi đè getItemViewType trong adapter.
- Tạo một lớp ViewHolder mới cho kiểu item này.
- Bổ sung điều kiện trong onCreateViewHolder và onBindViewHolder để xử lý kiểu mới.

Khai báo layout item màu mới

Tạo file layout *color_item_view.xml* cho kiểu list item mới này. Thêm Data Binding vào bên trong file layout để khai báo biến Color, và chúng ta sẽ tham chiếu biến này trong các TextView.

```
<layout ...>
   <data>
       <variable
           name="color"
           type="android.graphics.Color" />
   </data>
   <androidx.constraintlayout.widget.ConstraintLayout ...>
       <TextView
           ...
           android:backgroundColor="@{color.toArgb()}" />
       <TextView
           ...
           android:text="@{color.toString()}" />
   </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```

Kiểu view mới Adapter cần biết hai loại item view:

- Item hiển thị số
- Item hiển thị màu

```
enum class ITEM_VIEW_TYPE { NUMBER, COLOR }
```

Chỉnh sửa hàm getItemViewType() để trả về kiểu thích hợp (dạng Int).

```
override fun getItemViewType(position: Int): Int
```

Adapter cần biết về hai loại item view: một item hiển thị số và một item hiển thị màu. Ở đây chúng ta đã sử dụng một lớp enum, nhưng bạn cũng có thể dùng các hằng số Int bất biến nếu muốn.

Sau đó, chỉnh sửa phương thức getItemViewType() của adapter để trả về kiểu thích hợp (dạng Int) dựa trên item tại vị trí được chỉ định.

Ghi đè hàm getItemViewType

Trong lớp: *NumberListAdapter.kt* sửa lại như sau:

```
override fun getItemViewType(position: Int): Int {
    return when(getItem(position)) {
        is Int -> ITEM_VIEW_TYPE.NUMBER.ordinal
        else -> ITEM_VIEW_TYPE.COLOR.ordinal
    }
}
```

Đoạn code này sẽ kiểm tra dữ liệu tại một vị trí nhất định và trả về một số nguyên cho biết kiểu view cần sử dụng.

Định nghĩa ViewHolder mới

```
class ColorViewHolder private constructor(val binding: ColorItemViewBinding):     Recycler

    companion object {
        fun from(parent: ViewGroup): ColorViewHolder {
            val layoutInflater = LayoutInflater.from(parent.context)
            val binding = ColorItemViewBinding.inflate(layoutInflater,                pare
            return ColorViewHolder(binding)
        }
    }
}
```

Vì chúng ta đã định nghĩa một file layout mới tên là color_item_view.xml và đã bật Data Binding trong ứng dụng, nên lớp ColorItemViewBinding sẽ được tự động sinh ra. Hãy sử dụng ColorItemViewBinding để inflate view của item màu trong danh sách. Chúng ta cũng nên định nghĩa một ColorViewHolder mới cho kiểu item view này.

Cập nhật onCreateViewHolder

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
    RecyclerView.ViewHolder {
    return when(viewType) {
        ITEM_VIEW_TYPE.NUMBER.ordinal -> IntViewHolder.from(parent)
        else -> ColorViewHolder.from(parent)
```

```
    }
}
```

Trong onCreateViewHolder(), chúng ta sẽ kiểm tra kiểu view (view type) và tạo ViewHolder phù hợp.

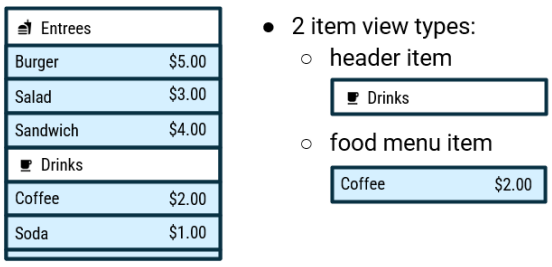
Cập nhật onBindViewHolder()

```
override fun onBindViewHolder(holder: RecyclerView.ViewHolder, position: Int) {
    when (holder) {
        is IntViewHolder -> {
            holder.binding.num = getItem(position) as Int
            holder.binding.executePendingBindings()
        }
        is ColorViewHolder -> {
            holder.binding.color = getItem(position) as Color
            holder.binding.executePendingBindings()
        }
    }
}
```

Trong onBindViewHolder(), chúng ta sử dụng kiểu lớp của ViewHolder để gán giá trị cho các biến được expose trong binding, sau đó gọi executePendingBindings() để đảm bảo thông tin kích thước được cập nhật chính xác.

10.4. Headers

Header là một ví dụ điển hình cho lý do tại sao bạn có thể muốn có nhiều loại item view. Đôi khi bạn có thể muốn thêm một header vào danh sách item trong RecyclerView. Hoặc bạn cũng có thể muốn tách các item trong danh sách thành nhiều nhóm khác nhau với nhiều header. Hãy cùng xem một ví dụ.



Trong ví dụ này, RecyclerView đang hiển thị một danh sách các món ăn trong menu, với các header dùng để phân loại từng nhóm món. Có 2 loại item view trong

RecyclerView này. Một header view được tạo từ một ImageView và một TextView. Trong khi đó, một item của menu món ăn được tạo từ hai TextView để hiển thị tên món và giá của nó.

Các view vẫn được tái sử dụng, nhưng điều quan trọng là tại mỗi vị trí cần loại ViewHolder nào. Các header chỉ được tái sử dụng ở những vị trí cần header, trong khi các item menu chỉ được tái sử dụng ở những vị trí cần item menu.

Khi triển khai header trong RecyclerView, bạn có thể:

- Đại diện cho header trực tiếp trong tập dữ liệu (ví dụ: phần tử thứ 0 trong tập dữ liệu là category Entrees).
- Hoặc triển khai logic trong adapter để trả về một ViewHolder khác ở những chỉ số nhất định (ví dụ: index 0), sau đó hiển thị các item còn lại dựa trên offset đó (tức là món Burger vẫn ở vị trí 0 trong tập dữ liệu, nhưng sẽ được hiển thị ở vị trí 1 trong RecyclerView). Cách thứ hai có thể hơi phức tạp khi phải quản lý offset nếu bạn bắt đầu có nhiều header trong danh sách.

Dù theo cách nào, việc có thể hiển thị header trong danh sách là rất hữu ích khi bạn cần xây dựng các danh sách phức tạp hơn.

10.5. Grid Layout

Trong rất nhiều tình huống, bạn có thể gặp phải là cần hiển thị dữ liệu dưới dạng lưới (grid).

Từ trước đến giờ ta vẫn dùng RecyclerView để hiển thị dữ liệu dạng danh sách dọc, tuy nhiên RecyclerView hoàn toàn có thể dùng để hiển thị dữ liệu dạng lưới.

Để hiển thị dữ liệu dưới dạng lưới trong RecyclerView, chúng ta chỉ cần thay đổi layoutManager trong code. Khi khởi tạo GridLayoutManager, tham số đầu tiên là Context, và tham số thứ hai là số lượng item mà chúng ta muốn hiển thị trong một hàng.

Trong hàm onCreate của MainActivity, sau khi có tham chiếu đến RecyclerView, ta có thể lựa chọn chế độ hiển thị cho nó:

- Hiển thị dạng danh sách: recyclerView.layoutManager = LinearLayoutManager(this)
- Hiển thị dạng lưới: recyclerView.layoutManager = GridLayoutManager(this, 2)

GridLayoutManager sắp xếp các item trong một lưới giống như một bảng gồm các hàng và cột. Hướng cuộn có thể là dọc hoặc ngang. Mặc định, mỗi item chiếm 1 span, nhưng bạn có thể thay đổi số span mà một item chiếm bằng cách cung cấp một SpanSizeLookup tùy chỉnh.

Giả sử tất cả các phần tử đều cùng một loại item, bạn có thể tính số lượng span khả dụng bằng cách lấy số dp khả dụng trong chiều được chỉ định, rồi chia cho kích thước item cộng với các ràng buộc layout theo chiều đó.

Thiết lập kích thước span cho một item Tạo một instance của SpanSizeLookup và ghi đè hàm getSpanSize(position):

```
val manager = GridLayoutManager(this, 2)
    manager.spanSizeLookup = object : GridLayoutManager.SpanSizeLookup() {
    override fun getSpanSize(position: Int): Int {
        return when (position) {
            0,1,2 -> 2
            else -> 1
        }
    }
}
```

GridLayoutManager cho phép bạn xác định một item chiếm bao nhiêu span bằng cách cung cấp một instance SpanSizeLookup tùy chỉnh và ghi đè getSpanSize(position) để chỉ định số span mà item tại vị trí đó sẽ chiếm. Việc sử dụng getSpanSize() đặc biệt quan trọng khi làm việc với GridLayoutManager.

Lưu ý rằng layout manager chỉ có thể sử dụng vị trí (position) trong dataset để xác định kích thước span (ví dụ: cứ cách một item thì có span size bằng X, hoặc cứ mỗi item thứ n thì có span size bằng Y).

Trong ví dụ, đoạn code thiết lập span size cho 3 phần tử đầu tiên bằng 2, và tất cả các phần tử còn lại bằng 1.

Chương 11
Kết nối Internet

11.1. Android Permision

Trong Android để bảo vệ quyền riêng tư của người dùng, một số tính năng cần phải được người dùng cho phép mới có thể thực hiện. Ví dụ: Ứng dụng phải yêu cầu quyền truy cập để sử dụng các dữ liệu nhạy cảm của người dùng (chẳng hạn như danh bạ và SMS), cũng như một số tính năng hệ thống nhất định (chẳng hạn như camera và quyền truy cập internet).

Để khai báo các quyền (Permision) sử dụng thẻ <uses-permission> trong file AndroidManifest.xml

Các quyền được cấp cho ứng dụng của bạn

- Quyền có thể được cấp trong quá trình cài đặt hoặc trong khi ứng dụng chạy, tùy thuộc vào cấp độ bảo vệ.
- Mỗi quyền đều có một cấp độ bảo vệ: normal, signature hoặc dangerous.
- Đối với các quyền được cấp trong khi ứng dụng chạy, hệ thống sẽ hiển thị hộp thoại để người dùng quyết định cho phép hoặc từ chối quyền truy cập của ứng dụng.

Android sử dụng các cấp độ bảo vệ (protection levels) cho quyền dựa trên mức độ rủi ro mà hệ thống đánh giá đối với người dùng. Cấp độ bảo vệ sẽ ảnh hưởng đến việc có cần yêu cầu quyền lúc runtime hay không.

- Normal permissions: áp dụng cho những trường hợp ứng dụng cần truy cập dữ liệu hoặc tài nguyên bên ngoài sandbox của nó, nhưng hầu như không gây rủi ro cho quyền riêng tư của người dùng hoặc sự hoạt động của ứng dụng khác. Hệ thống sẽ tự động cấp các quyền này cho ứng dụng trong quá trình cài đặt. Người dùng sẽ không được nhắc để cấp quyền, và họ cũng không thể thu hồi các quyền này.
- Signature permissions: là tập hợp quyền đặc biệt, chỉ có thể được sử dụng bởi các ứng dụng được ký bằng cùng một chứng chỉ với ứng dụng đã định nghĩa quyền đó. Ví dụ: một nhóm ứng dụng trong cùng một tài khoản nhà phát triển

có thể phối hợp với nhau (và vẫn giữ quyền an toàn trước ứng dụng bên thứ ba) nếu chúng được ký cùng chứng chỉ. Hệ thống sẽ cấp quyền này trong quá trình cài đặt nếu ứng dụng đáp ứng yêu cầu.

- Dangerous permissions: áp dụng cho các trường hợp ứng dụng muốn truy cập dữ liệu hoặc tài nguyên có liên quan đến thông tin cá nhân của người dùng, hoặc có khả năng ảnh hưởng đến dữ liệu được lưu trữ của người dùng hay hoạt động của ứng dụng khác.

Ví dụ, trong file AndroidManifest.xml như sau:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.sampleapp">
    <uses-permission android:name="android.permission.USE_BIOMETRIC" />
    <application>
        <activity
            android:name=".MainActivity" ... >
            ...
        </activity>
    </application>
</manifest>
```

Trên đây là một ví dụ về một ứng dụng yêu cầu quyền USE_BIOMETRIC, được chỉ định trong thẻ uses-permission. Quyền này cho phép ứng dụng sử dụng các phương thức sinh trắc học mà thiết bị hỗ trợ, bao gồm cả phần cứng vân tay. Quyền USE_BIOMETRIC có cấp độ bảo vệ normal và sẽ được cấp nếu bạn đồng ý cài đặt ứng dụng. Đối với bất kỳ quyền normal nào, chỉ cần khai báo trong file manifest là đủ, và code của bạn có thể mặc định rằng ứng dụng sẽ được cấp quyền này.

Nếu ứng dụng của bạn cần thực hiện các thao tác mạng (ví dụ: tải nội dung), bạn phải khai báo các quyền này trong file AndroidManifest.xml. Quyền truy cập Internet thuộc nhóm normal, vì vậy chỉ cần thêm các dòng trên vào file manifest là đủ. Ngược lại, với các quyền thuộc nhóm dangerous, bạn sẽ phải thực hiện thêm nhiều bước xử lý khác.

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

Yêu cầu các quyền dangerous Các quyền dangerous được yêu cầu và cấp trong khi ứng dụng đang chạy (runtime). Người dùng sẽ được nhắc khi họ lần đầu

tiên cố gắng truy cập vào một tính năng cần quyền đó. Nếu người dùng không bao giờ sử dụng tính năng đó, thì họ sẽ không bị nhắc chấp nhận quyền. Việc triển khai runtime permissions là bắt buộc khi thiết bị chạy Android 6.0 (API level 23) trở lên và ứng dụng có targetSdkVersion từ 23 trở lên.

Tuy nhiên, nếu ứng dụng của bạn đặt targetSdkVersion thấp hơn hoặc thiết bị chạy phiên bản Android cũ hơn, bạn có thể gặp hành vi trước đây — tức là các quyền dangerous được cấp ngay trong quá trình cài đặt. Cụ thể, điều này xảy ra nếu thiết bị chạy Android 5.1.1 (API level 22) trở xuống, hoặc ứng dụng có targetSdkVersion bằng 22 hoặc thấp hơn (dù chạy trên bất kỳ phiên bản Android nào). Lúc cài đặt, người dùng sẽ được nhắc chấp nhận hoặc từ chối tất cả các quyền dangerous mà ứng dụng yêu cầu. Nếu người dùng từ chối, hệ thống sẽ hủy việc cài đặt ứng dụng.

Trong phạm vi bài học này, chúng ta sẽ giả định rằng các quyền dangerous được yêu cầu tại runtime, vì đây là hành vi mới nhất của nền tảng.



Trên đây là một ví dụ về việc yêu cầu quyền dangerous trong lúc ứng dụng chạy (runtime), cụ thể là quyền truy cập vào danh bạ của người dùng. Khi quyền này bị từ chối nhiều lần hoặc nếu người dùng chọn don't ask again (trên các phiên bản Android cũ), điều đó có nghĩa là người dùng không muốn bị nhắc lại về quyền này

nữa.

Lưu ý rằng ngay cả khi người dùng đã cấp quyền mà ứng dụng yêu cầu, bạn cũng không thể luôn tin tưởng rằng quyền đó sẽ tồn tại. Người dùng có thể bật hoặc tắt từng quyền riêng lẻ trong phần cài đặt hệ thống. Vì vậy, bạn luôn cần kiểm tra và yêu cầu quyền ở runtime để tránh lỗi khi chạy (ví dụ: SecurityException).

Bắt đầu từ Android 11, người dùng có thêm lựa chọn chỉ cấp quyền cho một lần sử dụng duy nhất (one-time permission). Là lập trình viên, bạn không cần phải xử lý quyền này theo cách khác. Bạn chỉ cần tiếp tục tuân theo best practice đã nêu ở trên: kiểm tra và sau đó yêu cầu quyền.

Cách sử dụng khai báo permision chuẩn nhất Cách tốt nhất khi làm việc với quyền ứng dụng:

- **Chỉ sử dụng những quyền thật sự cần thiết cho ứng dụng để hoạt động.** Tuỳ thuộc vào cách bạn sử dụng quyền, có thể có những cách khác để đạt được điều bạn cần (ví dụ: dùng system intents, identifiers, chạy nền cho cuộc gọi) mà không phải truy cập dữ liệu nhạy cảm.

- **Chú ý đến các quyền mà thư viện (library) yêu cầu.** Khi bạn thêm một thư viện, bạn cũng kế thừa các yêu cầu quyền của nó. Bạn cần biết rõ mình đang thêm gì, thư viện đó yêu cầu quyền nào và các quyền đó được dùng cho mục đích gì.

- **Minh bạch.** Khi bạn yêu cầu quyền, hãy nói rõ bạn sẽ truy cập vào cái gì và tại sao, để người dùng có thể đưa ra quyết định sáng suốt. Cung cấp thông tin này cùng với hộp thoại yêu cầu quyền, bao gồm lúc cài đặt, khi chạy ứng dụng, hoặc khi cập nhật quyền.

- **Thể hiện rõ ràng việc truy cập hệ thống.** Cung cấp chỉ báo liên tục khi bạn truy cập các khả năng nhạy cảm (ví dụ: camera hoặc micro). Việc làm rõ ràng cho người dùng biết khi nào bạn thu thập dữ liệu sẽ tránh được cảm giác rằng bạn đang thu thập dữ liệu một cách bí mật.

11.2. Kết nối, sử dụng các tài nguyên mạng

Retrofit

- Thư viện networking biến HTTP API của bạn thành một interface Kotlin và Java.

- Cho phép xử lý các request và response thành các đối tượng (objects) để ứng dụng của bạn có thể sử dụng.
  - Cung cấp hỗ trợ cơ bản để phân tích (parse) các kiểu phản hồi phổ biến, chẳng hạn như XML và JSON.
  - Có thể được mở rộng để hỗ trợ các kiểu phản hồi khác.

Tại sao lại sử dụng Restrofit

- Được xây dựng dựa trên các thư viện tiêu chuẩn, như OkHttp, cung cấp
  - Hỗ trợ HTTP/2
  - Tái sử dụng kết nối (connection pooling)
  - Bộ nhớ đệm phản hồi (response caching) và bảo mật nâng cao
- Giúp lập trình viên không phải viết phần mã khung (scaffolding) cần thiết để chạy một request

Thêm các thư viện phụ thuộc trong gradle

```
implementation "com.squareup.retrofit2:retrofit:2.9.0"
implementation "com.squareup.retrofit2:converter-moshi:2.9.0"
implementation "com.squareup.moshi:moshi:$moshi_version"
implementation "com.squareup.moshi:moshi-kotlin:$moshi_version"
kapt "com.squareup.moshi:moshi-kotlin-codegen:$moshi_version"
```

11.3. Kết nối đến một dịch vụ web

Các phương thức trong HTTP Một dịch vụ web HTTP sẽ có các phương thức để bạn truy xuất (retrieve) và thay đổi (mutate) dữ liệu. Các phương thức này thường được thể hiện dưới dạng các HTTP verb: GET, POST, PUT và DELETE.

- GET
- POST
- PUT
- DELETE

Định nghĩa một dịch vụ Restrofit

```
interface SimpleService {
    @GET("posts")
    suspend fun listAll(): List<Post>

    @GET("posts/{userId}")
    suspend fun listByUser(@Path("userId") userId:String): List<Post>

    @GET("posts/search")  // becomes post/search?filter=query
    suspend fun search(@Query("filter") search: String): List<Post>

    @POST("posts/new")
    suspend fun create(@Body post : Post): Post
}
```

Trong mã nguồn ứng dụng, dịch vụ (service) Retrofit là nơi bạn khai báo tất cả các endpoint HTTP dự định sử dụng. Dưới đây là ví dụ về giao diện (interface) SimpleService.

Mỗi endpoint của web API mà bạn muốn truy cập cần được biểu diễn bằng một phương thức trong interface, kèm annotation chỉ rõ loại yêu cầu (dựa trên phương thức/động từ HTTP) và đường dẫn URL tương đối. Đồng thời, hãy khai báo các tham số liên quan đến endpoint và kiểu trả về phù hợp ngay trong chữ ký phương thức.

Hàm listPosts() truy xuất danh sách bài viết từ một endpoint của API có đường dẫn kết thúc bằng /posts. URL cơ sở (base URL) sẽ được khai báo ở phần sau.

Bạn có thể đã để ý một số annotation khác được sử dụng:

- @Path: Cho phép tạo hàm làm việc với URL động. Đối số phương thức userId sẽ được chèn vào endpoint của API có dạng /posts/userId.
- @Body: Cho phép gửi nội dung trong phần thân (request body) của HTTP request. Ví dụ, nội dung của một bài viết mới sẽ được gửi kèm trong request body.
- @Query: Cho phép gắn thêm các tham số truy vấn (query parameters) vào URL. Ví dụ, dùng cho endpoint /posts/search.

Lưu ý: Chúng ta sẽ sử dụng Retrofit cùng với coroutines, vì vậy các hàm này được đánh dấu bằng từ khóa suspend.

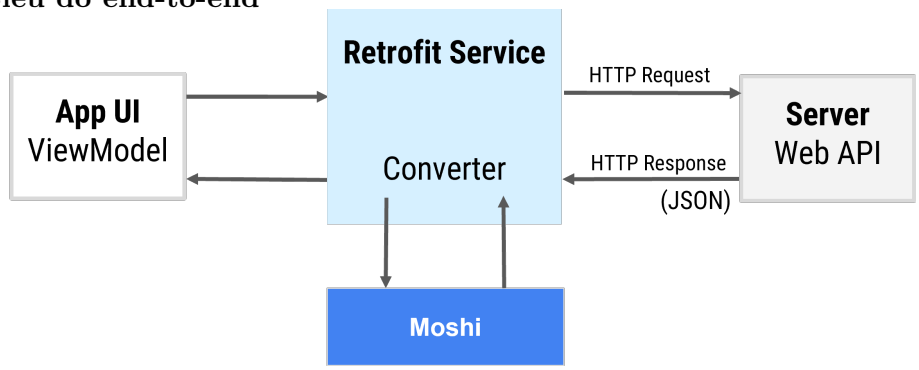
Tạo một đối tượng Retrofit để truy cập mạng.

```
val retrofit = Retrofit.Builder()
.baseUrl("https://example.com")
.addConverterFactory(...)
.build()

val service = retrofit.create(SimpleService::class.java)
```

Để tạo được service thực tế, trước hết bạn cần tạo một đối tượng Retrofit bằng Retrofit.Builder() với URL cơ sở (base URL) và các bộ chuyển đổi (converter) tùy chọn dùng để kiểm soát cách yêu cầu được xử lý. Đối tượng Retrofit sau đó sẽ sử dụng interface của service để tạo ra một triển khai cụ thể của service đó.

Biểu đồ end-to-end



Sơ đồ end-to-end này minh họa cách ứng dụng gửi một yêu cầu HTTP và nhận lại phản hồi. Tiếp theo, chúng ta sẽ nói về cách phân tích (parse) phản hồi bằng thư viện như Moshi.

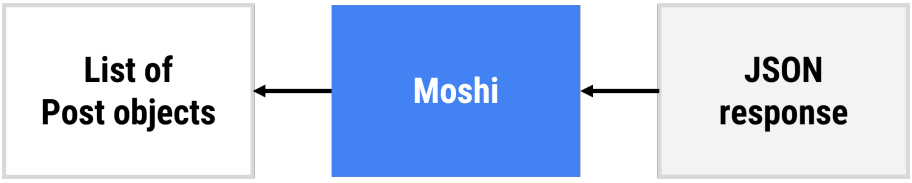
Converter.Factory Converter.Factory dùng để chuyển từ response type sang các đối tượng lớp (class object):

- JSON (Gson or Moshi)
- XML (Jackson, SimpleXML, JAXB)
- Protocol buffers
- Dữ liệu vô hướng (dữ liệu nguyên thủy, dạng đóng hộp, và String)

Converter.Factory là một lớp của Retrofit giúp chuyển đổi từ kiểu phản hồi (response type) sang đối tượng lớp (class objects). Các mô-đun chuyển đổi (converter modules) này đã sẵn có để bạn sử dụng với Retrofit. Ngoài ra, bạn cũng có thể kế thừa (extend) Converter.Factory để tạo converter của riêng mình.

Moshi

- Moshi là thư viện cho phép parse JSON sang đối tượng và ngược lại.
- Để dùng, cần thêm thư viện phụ thuộc vào file gradle
- Cấu hình sử dụng Moshi trong Restrofit



Trong trường hợp ứng dụng mẫu của chúng ta, sơ đồ này minh họa cách Moshi giúp phân tích (parse) phản hồi JSON thành danh sách các đối tượng Post mà ứng dụng có thể sử dụng.

Moshi JSON encoding

```
@JsonClass(generateAdapter = true)
    data class Post (
    val title: String,
    val description: String,
    val url: String,
    val updated: String,
    val thumbnail: String,
    val closedCaptions: String?)
```

Đây là phần hiện thực (implementation) cho lớp Post trong mã nguồn. Việc bổ sung hỗ trợ Moshi cho một lớp đơn giản như tạo một entity trong Room—chỉ cần thêm annotation @JsonClass. Cơ chế này sử dụng reflection để sinh mã phục vụ việc mã hóa/tuần tự hóa đối tượng thành JSON. Chúng ta đặt generateAdapter = true để tự động tạo một adapter đảm nhiệm việc ánh xạ dữ liệu ở phía sau (behind the scenes).

Ví dụ JSON code

```
{
    "title":"Android Jetpack: EmojiCompat",
    "description":"Android Jetpack: EmojiCompat",
    "url":"https://www.youtube.com/watch?v=sYGKUtM2ga8",
    "updated":"2018-06-07T17:09:43+00:00",
    "thumbnail":"https://i4.ytimg.com/vi/sYGKUtM2ga8/hqdefault.jpg"
}
```

Trên đây là một đoạn phản hồi JSON do dịch vụ web trả về. Moshi sẽ chuyển (parse) dữ liệu này thành các đối tượng để ứng dụng sử dụng.

Thiết lập Restrofit và Moshi

```
private val moshi = Moshi.Builder()
    .add(KotlinJsonAdapterFactory())
    .build()

val retrofit = Retrofit.Builder()
        .addConverterFactory(MoshiConverterFactory.create(moshi))
        .baseUrl(BASE_URL)
        .build()

object API {
    val retrofitService : SimpleService by lazy {
        retrofit.create(SimpleService::class.java)
    }
}
```

Quay lại mã cụ thể, đoạn trên minh họa cách khởi tạo một đối tượng service kết nối tới web service của chúng ta, đồng thời truyền MoshiConverterFactory để hỗ trợ phân tích (parse) phản hồi.

Sử dụng Restrofit với Coroutines

```
viewModelScope.launch {
    Log.d("posts", API.retrofitService.searchPosts("query"))
}
```

Tại đây, chúng ta sử dụng hàm của Retrofit trong viewModelScope để truy xuất danh sách bài viết dựa trên truy vấn tìm kiếm.

11.4. Hiển thị danh sách ảnh từ máy chủ

Glide

- Thư viện để load ảnh của bên thứ 3
- Chú trọng vào hiệu năng để cuộn ảnh mượt mà hơn
- Hỗ trợ ảnh, video, và ảnh động GIF

Thêm thư viện hỗ trợ sau trong gradle:

```
implementation "com.github.bumptech.glide:glide:$glide_version"
```

Đoạn mã trên gắn Glide với vòng đời của đối tượng sở hữu (chẳng hạn như Fragment hoặc Activity), và tải hình ảnh từ URL đã chỉ định vào ImageView tương ứng.

```
Glide.with(fragment)
    .load(url)
    .into(imageView);
```

Tùy chỉnh request với RequestOptions

- Áp dụng để cắt ảnh (crop)
- Áp dụng các phép dịch chuyển
- Thiết lập các tùy chọn cho ảnh hiển thị tạm thời (placeholder) hoặc ảnh khi xảy ra lỗi (error image).
- Thiết lập chính sách caching

Bạn có thể tùy chỉnh một request bằng cách sử dụng RequestOptions và áp dụng nhiều tùy chọn cùng nhau theo dạng chuỗi. Ví dụ, bạn có thể vừa áp dụng thao tác cắt ảnh (crop), vừa thiết lập ảnh hiển thị tạm thời (placeholder) cho ảnh.

Ví dụ RequestOptions

```
@BindingAdapter("imageUrl")
fun bindImage(imgView: ImageView, imgUrl: String?) {
    imgUrl?.let {
        val imgUri = imgUrl.toUri().buildUpon().scheme("https").build()

        Glide.with(imgView)
            .load(imgUri)
            .apply(RequestOptions()
                .placeholder(R.drawable.loading_animation)
                .error(R.drawable.ic_broken_image))
            .into(imgView)
    }
}
```

Trên đây là một ví dụ về RequestOptions trong một request của Glide, được sử dụng trong ngữ cảnh với Binding Adapter.

Giả sử có một thuộc tính tùy chỉnh trên ImageView, gọi là imgUrl, dùng để chỉ định URL của ảnh cần tải từ mạng. Ảnh sau đó sẽ được nạp vào ImageView tương ứng thông qua Glide.

Trong request của Glide, chúng ta chỉ định RequestOptions, bao gồm ảnh hiển thị tạm thời (placeholder) cho đến khi ảnh thực được tải từ mạng, hoặc ảnh thay thế được hiển thị khi xảy ra lỗi.

Lưu ý rằng thay vì truyền trực tiếp lifecycle owner vào Glide, chúng ta truyền vào chính imgView. Điều này là hợp lệ vì Glide sẽ tự động lan truyền (propagate) ngược lên cho đến khi tìm thấy lifecycle owner của view.

11.5. Tổng kết chương

Trong chương này chúng ta đã học các mục sau:

- Khai báo các quyền mà ứng dụng của bạn cần trong AndroidManifest.xml.
- Sử dụng ba mức bảo vệ cho quyền (permissions): normal, signature và dangerous (đối với quyền dangerous, hệ thống sẽ yêu cầu người dùng cấp quyền khi ứng dụng chạy).
- Sử dụng thư viện Retrofit để gọi các API dịch vụ web từ ứng dụng.
- Sử dụng thư viện Moshi để phân tích (parse) phản hồi JSON thành các đối tượng lớp (class objects).
- Tải và hiển thị hình ảnh từ Internet bằng thư viện Glide.

Tài liệu tham khảo

[1] H. Rademacher, *Higher Mathematics from an Elementary point of view*, Birkhauser, 1983.

[2] Martin Aigner and Gunter M. Ziegler, *Proofs from the book*, Springer, 1999.

[3] T. Andreescu, R. Gelca, *Mathematical olympiad challenges*, Birkhauser, 2002.

[4] Loren C. Larson, *Problem-Solving through problems*, Springer-Verlag, 1983.

[5] L. E. Dickson, *New first course in the theory of equations* John Wiley & Sons, 1946.

[6] Judita Cofman, *What to solve? Problems and Suggestions for Young Mathematicians*, Clarendon Press-Oxford, 1989.

[7] Nguyễn Hữu Điển, *Phương pháp Dirichle và ứng dụng*, NXBKHKT, 1999.

[8] Nguyễn Hữu Điển, *Phương pháp Quy nạp toán học*, NXBGD, 2000.

[9] Nguyễn Hữu Điển, *Những phương pháp điển hình trong giải toán phổ thông*, NXBGD, 2001.

[10] Nguyễn Hữu Điển, *Những phương pháp giải bài toán cực trị trong hình học*, NXBKHKT, 2001.

[11] Nguyễn Hữu Điển, *Sáng tạo trong giải toán phổ thông*, NXBGD, 2002.

[12] Nguyễn Hữu Điển, *Đa thức và ứng dụng*, NXBGD, 2003.

[13] Nguyễn Hữu Điển, *Giải phương trình vô định nghiệm nguyên*, NXBĐHQG, 2004.

[14] Nguyễn Hữu Điển, *Giải toán bằng phương pháp đại lượng bất biến*, NXBGD, 2004.